

RETAIL DATA INTEGRATION PLATFORM

Azure Data Factory — Build & Implementation Walkbook

A screenshot-led technical record of how the ADF solution was designed, built and proven

Document Type	Technical Build Walkbook for the RDIP Azure Data Factory Solution
Platform	Microsoft Azure Data Factory — factory "datafactnan66"
Storage Account	storenan66 (ADLS Gen2)
Sources Covered	Azure SQL (Orders) · Vendor REST API (Products) · Blob CSV (Stores)
Curated Output	dbo.SalesSummary (Azure SQL Database)
Prepared By	Data Engineering Team
Status	Build evidence captured during development & validation

Contents

1. Why This Document Exists
2. Solution Architecture at a Glance
3. Storage Foundation
4. Metadata Layer — Linked Services & Datasets
5. Incremental Load Design — the Watermark Control Table
6. Ingestion Layer — Landing the Three Sources
7. Incremental Load Pipeline — Step-by-Step
8. Transformation Layer — the DF_CleanAndJoin Data Flow
9. Deployment — Publishing the Factory
10. Orchestration — the Master Pipeline
11. Scheduling — Automated Daily Execution
12. Monitoring — Operational Evidence
13. Data Quality & Additional Validation Evidence
14. Final Curated Output — Proving the Business Outcome
15. Requirements Traceability
16. Closing Notes & Recommendations

1. Why This Document Exists

This walkbook is a build record for the Retail Data Integration Platform — a screen-by-screen account of how the Azure Data Factory solution was actually designed, assembled, and proven inside the datafactnan66 factory. Where a requirements document tells you *what* a platform is supposed to do, this document shows *how* it was actually done: every container, dataset, pipeline, and transformation step that was configured, along with the reasoning behind each decision.

The document is organised as a build journal rather than a specifications list. Each section follows the sequence a data engineer would naturally work through in practice — provisioning storage, wiring up the metadata layer, designing the incremental-load mechanism, building the ingestion pipelines, authoring the transformation logic, wiring orchestration and scheduling, and finally proving the result through monitoring and direct SQL evidence. Every screenshot captured below comes straight from the working factory during development and is annotated with what it demonstrates and which part of the overall design it supports — so a reader can follow the platform from an empty storage account through to a fully populated, query-ready curated table without needing to reproduce the build themselves.

2. Solution Architecture at a Glance

Three source systems feed the platform: an Azure SQL Database standing in for the on-premises Orders system, a vendor REST API for the Product Catalog, and CSV drops in Blob Storage for the Store Master. All three are landed into a single ADLS Gen2 account (storenan66) that is organised into four containers acting as the medallion-style zones for the pipeline:

landing — the raw drop zone for files arriving from the API and file-based sources, before ADF has touched them.

raw — a Parquet-standardised copy of every source (orders, products, stores), written by the ingestion pipelines and used as the single read surface for transformation.

processed — the intermediate ‘silver’ output of the Mapping Data Flow (sales), kept as Parquet for reuse and audit.

\$logs — the storage account's own diagnostic log container (created automatically by Azure).

On top of that storage layer sit five pipelines (three per-source ingestion pipelines, one transformation pipeline, and one master orchestrator), a single reusable Mapping Data Flow (DF_CleanAndJoin), a SQL watermark-control table that drives incremental loading, and a curated Azure SQL table (SalesSummary) that Power BI ultimately reads from. The remainder of this document walks through each of these components with the corresponding screen captures.

3. Storage Foundation

The first build step was provisioning the ADLS Gen2 account and laying out the container structure that the rest of the pipelines depend on. Hierarchical namespace was enabled at creation time so that the containers could be treated as a true folder hierarchy rather than a flat blob namespace.

Figure 1. Storage account overview (storenan66)

The screenshot shows the Microsoft Azure portal interface for the 'storenan66' storage account. The left sidebar contains navigation links: Overview, Activity log, Tags, Diagnose and solve problems, Access Control (IAM), Data migration, Events, Storage browser, Partner solutions, Resource visualizer, Data storage, and Containers. The main area is titled 'Overview' and shows account details under the 'Essentials' tab. Key information includes: Resource group (move) : NANDINLDE, Location : eastus, Subscription (move) : Azure subscription 1, Subscription ID : ccf7674d-2fed-4484-8086-e39221351d47, Disk state : Available, and Tags (edit) : Add tags. On the right, a table lists account properties: Performance (Standard), Replication (Locally redundant storage (LRS)), Account kind (StorageV2 (general purpose v2)), Provisioning state (Succeeded), and Created (9/3/2026, 11:23:58 AM). Below this, the 'Properties' tab is active, showing 'Data Lake Storage' settings: Hierarchical namespace (Enabled), Default access tier (Hot), and Blob anonymous access (Disabled). A 'Security' section on the right shows 'Require secure transfer for REST API operations' (Enabled) and 'Storage account key access' (Enabled).

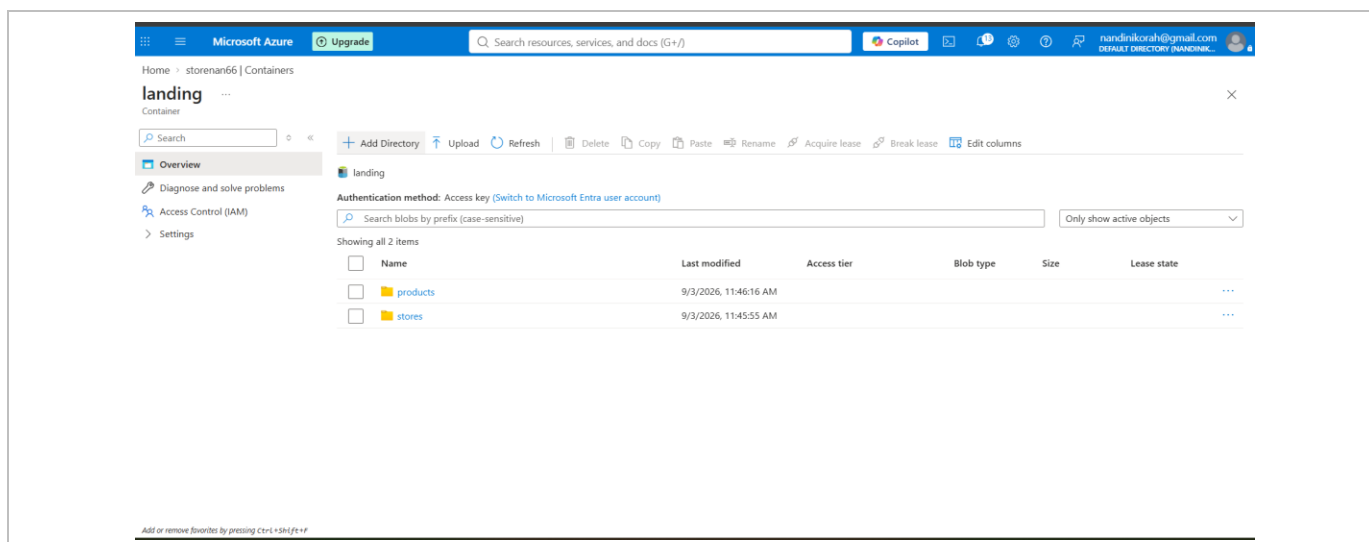
The account is provisioned as StorageV2 (general purpose v2) with Data Lake Storage / hierarchical namespace enabled, Locally Redundant Storage, and the Hot access tier — an appropriate, low-cost baseline for a batch, daily-refresh workload rather than a globally distributed or geo-redundant one.

Figure 2. "raw" container structure

The screenshot shows the 'raw' container structure in the Microsoft Azure portal. The left sidebar shows the 'Containers' section with 'raw' selected. The main area displays a table of subfolders. The table has columns: Name, Last modified, Access tier, Blob type, Size, and Lease state. The subfolders listed are 'orders', 'products', and 'stores', all created on 9/3/2026. Above the table, there are controls for 'Add Directory', 'Upload', 'Refresh', 'Delete', 'Copy', 'Paste', 'Rename', 'Acquire lease', 'Break lease', and 'Edit columns'. The 'Authentication method' is set to 'Access key (Switch to Microsoft Entra user account)'. A search bar is present with the placeholder 'Search blobs by prefix (case-sensitive)'. A dropdown menu shows 'Only show active objects'.

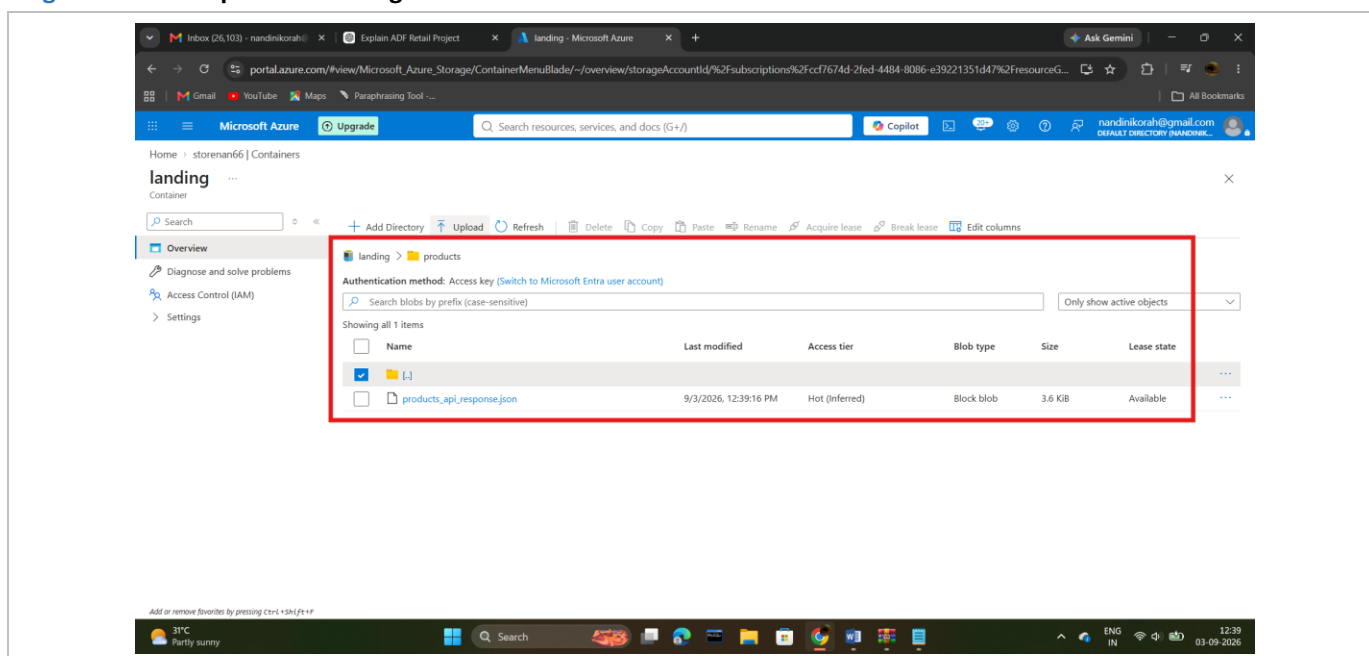
The raw container holds three subfolders — orders, products and stores — one per source system. Each ingestion pipeline writes exclusively into its own subfolder, which keeps the raw zone easy to reason about and avoids any pipeline being able to accidentally overwrite another source's data.

Figure 3. "landing" container



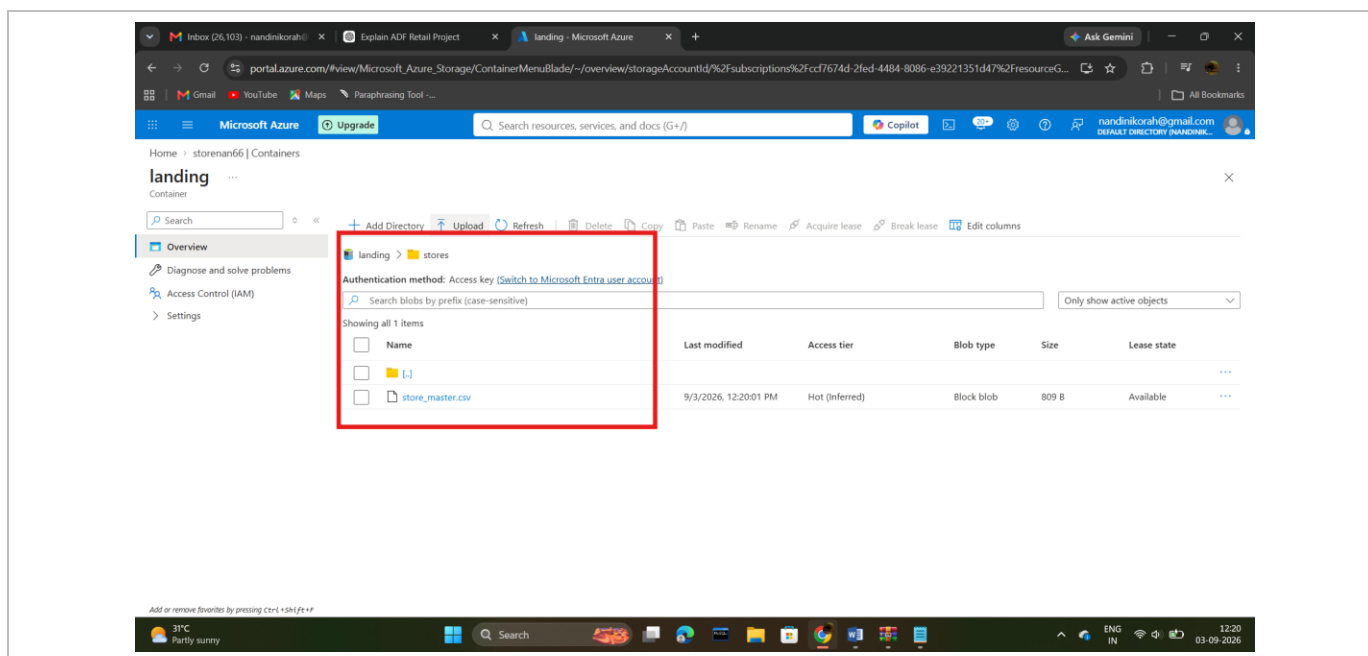
Landing is the pre-ADF drop zone. It currently holds a products and a stores subfolder, each containing the file exactly as it arrived from the source — the vendor's JSON API response and the store master CSV — before any schema standardisation happens.

Figure 4. Landed product catalog file



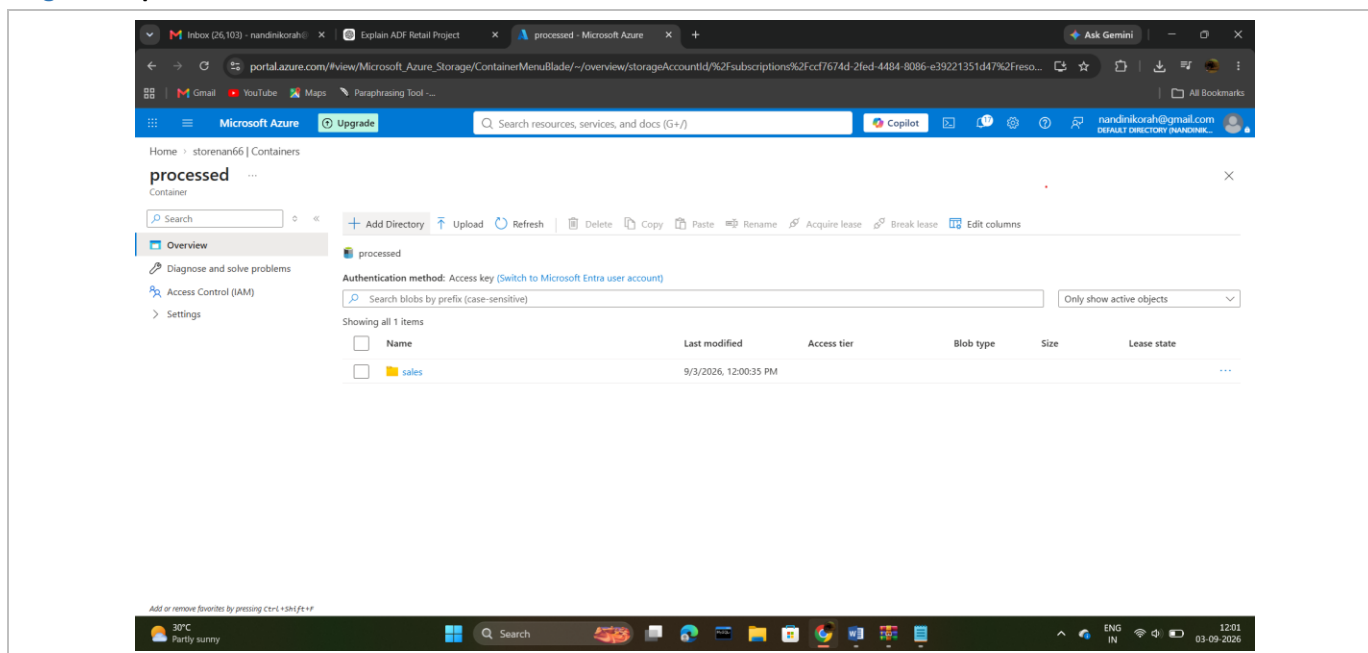
products_api_response.json sitting under landing/products, roughly 3.6 KiB, confirms the vendor payload has arrived and is available for the ingestion pipeline to pick up and convert to Parquet in the raw zone.

Figure 5. Landed store master file



store_master.csv under *landing/stores* — this is the file the Store Master ingestion pipeline reads on each run; because it is a small reference file it is reloaded in full rather than incrementally.

Figure 6. "processed" container



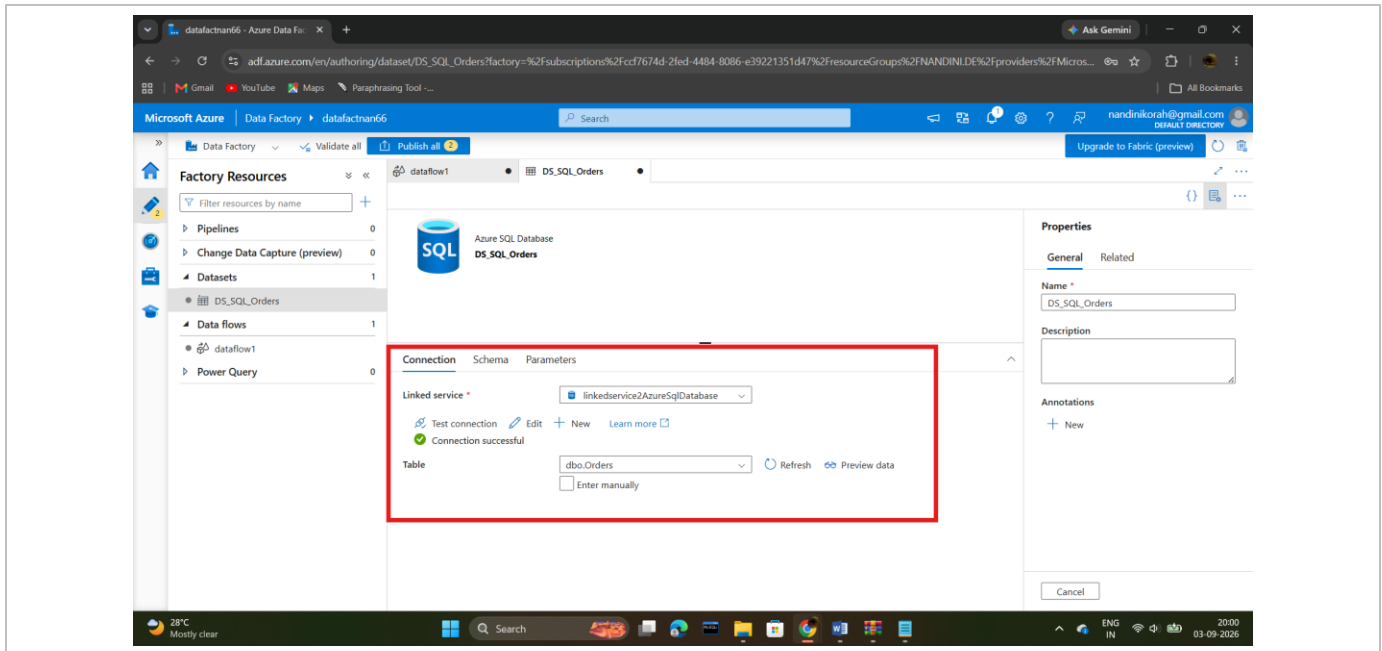
processed/sales is the landing point for the Mapping Data Flow's Parquet output — an intermediate, already-cleansed copy of the joined sales data kept for traceability and reuse, distinct from the final curated table that lands in Azure SQL Database.

4. Metadata Layer — Linked Services & Datasets

Every pipeline activity in ADF reads from or writes to a dataset, and every dataset points at a linked service. Building this metadata layer first — before any pipeline logic — meant the ingestion and transformation pipelines

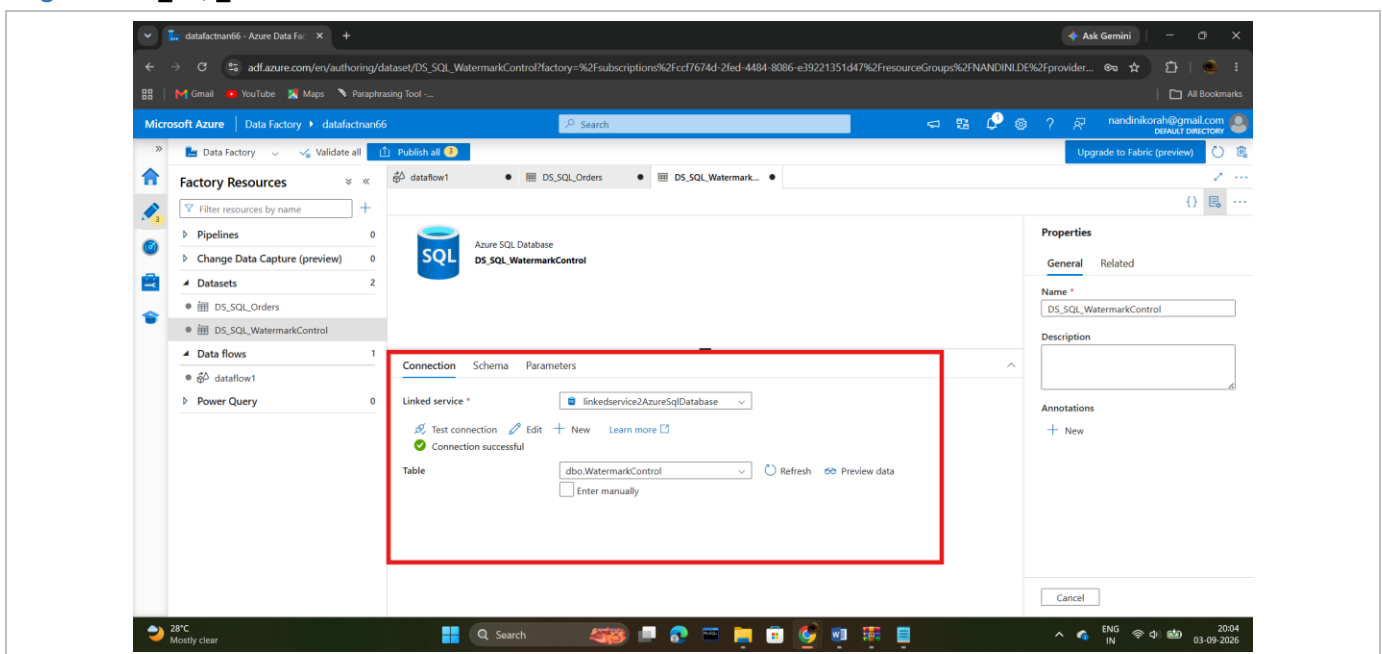
could simply be wired to existing, already-tested datasets rather than each pipeline re-defining its own connection. Two linked services underpin the whole factory: linkedservice2AzureSqlDatabase for the Orders/Watermark/SalesSummary tables, and AzureDataLakeStorage1 for every file-based dataset in landing, raw and processed.

Figure 7. DS_SQL_Orders



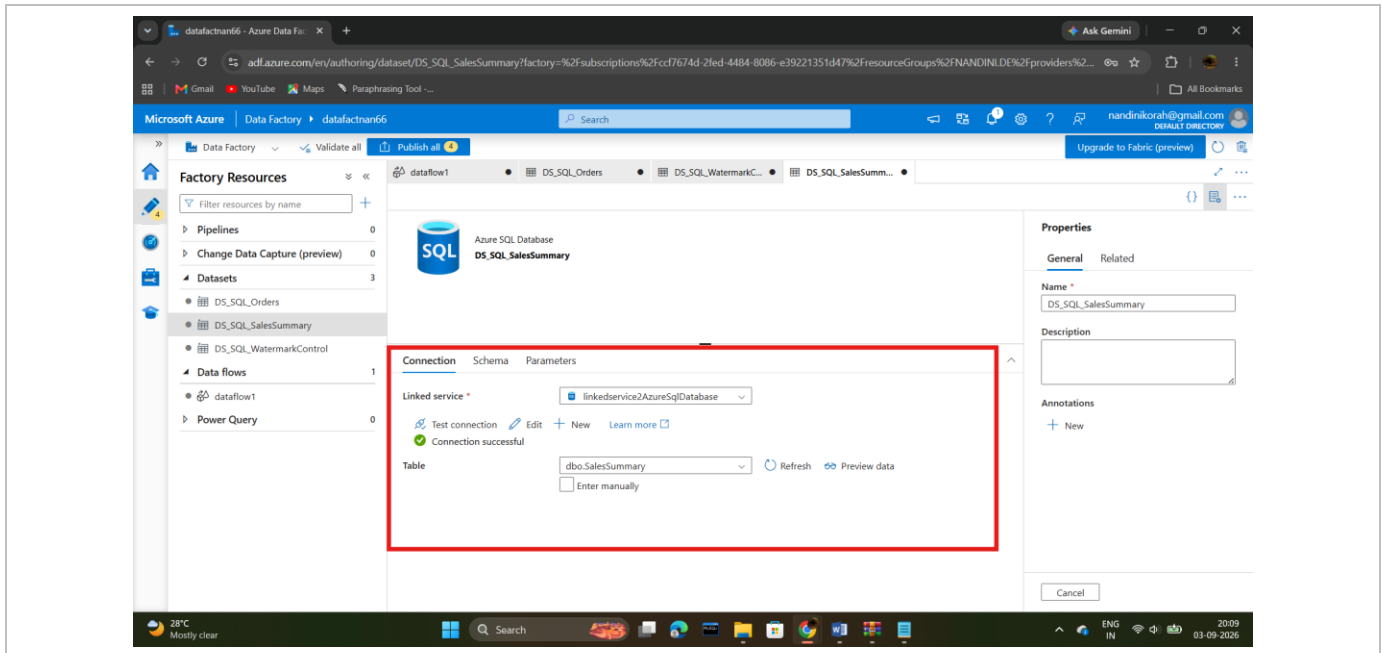
Points at `dbo.Orders` via `linkedService2AzureSqlDatabase`. "Test connection" reports *Connection successful*, confirming the factory has the correct credentials and network path to the SQL database before any pipeline tries to use it.

Figure 8. DS_SQL_WatermarkControl



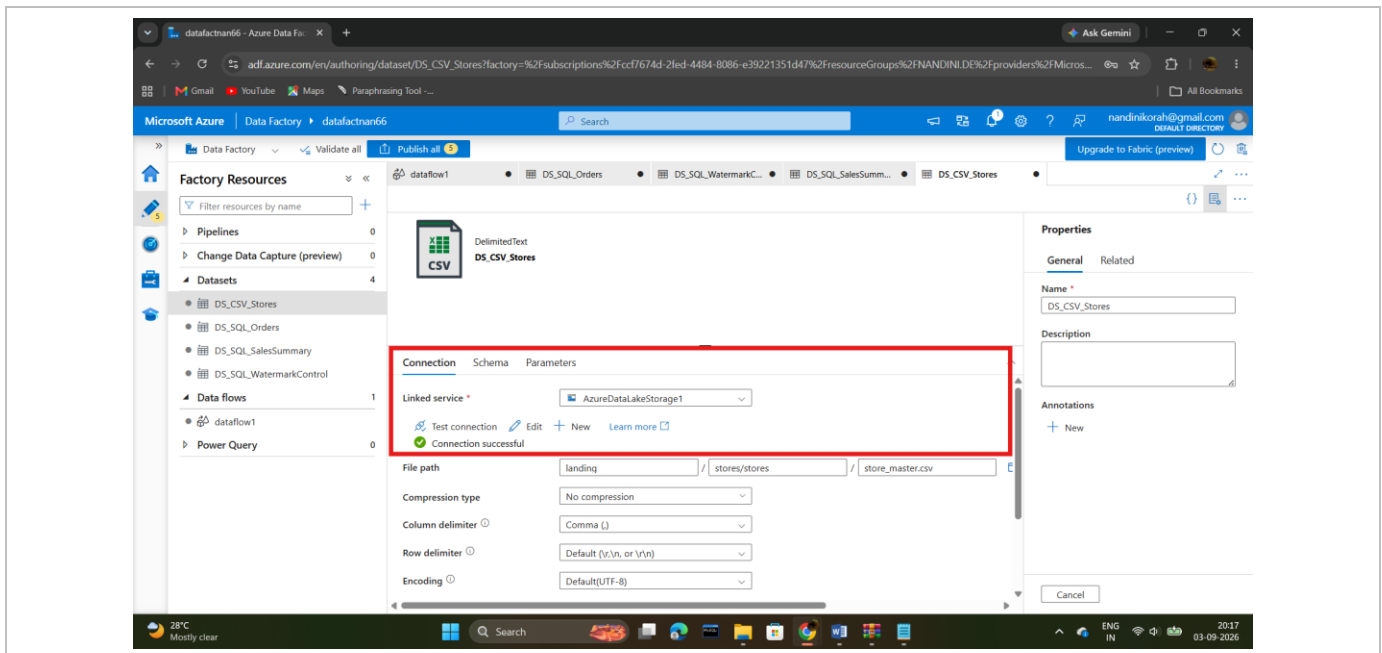
The dataset behind the incremental-load control table (`dbo.WatermarkControl`). Every Lookup activity that reads or updates watermark state goes through this single dataset definition.

Figure 9. DS_SQL_SalesSummary



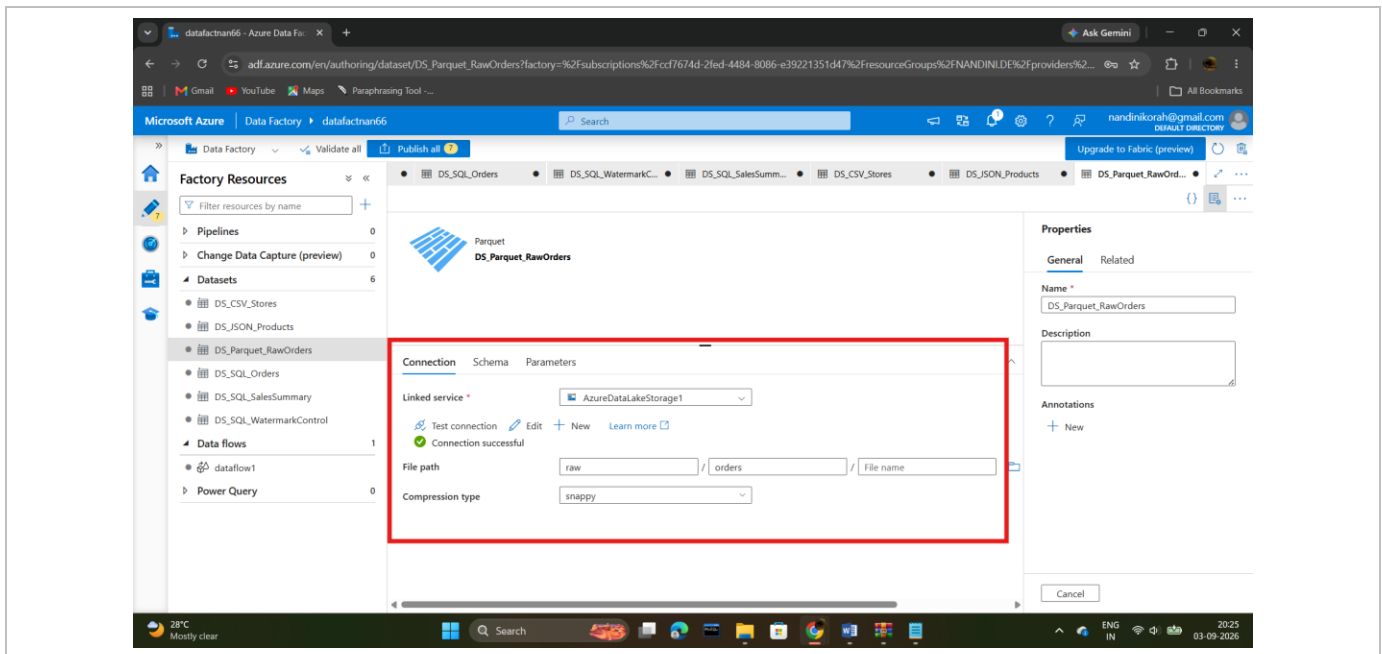
The sink dataset for the curated output table, *dbo.SalesSummary* — the table that Power BI is intended to query directly once the platform is live.

Figure 10. DS_CSV_Stores



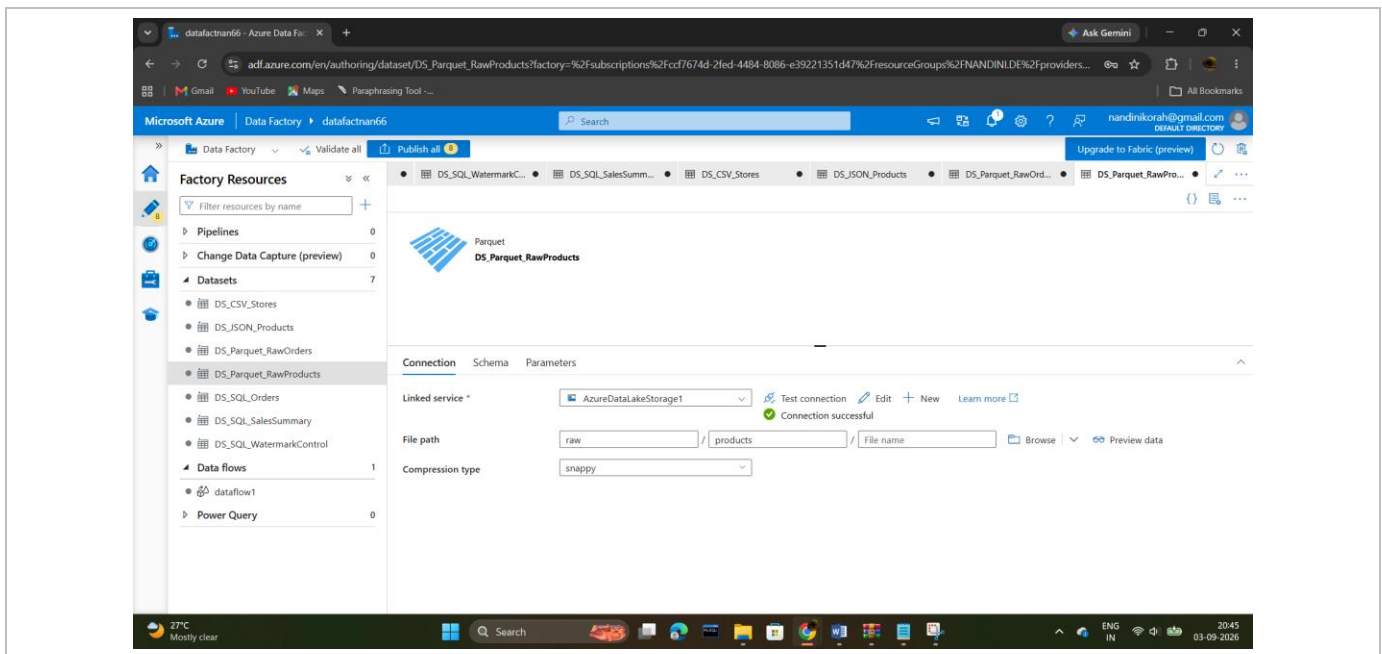
A *DelimitedText* dataset over *landing/stores/stores/store_master.csv*, comma-delimited with the first row as header — the source dataset for the Store Master ingestion copy activity.

Figure 11. DS_Parquet_RawOrders



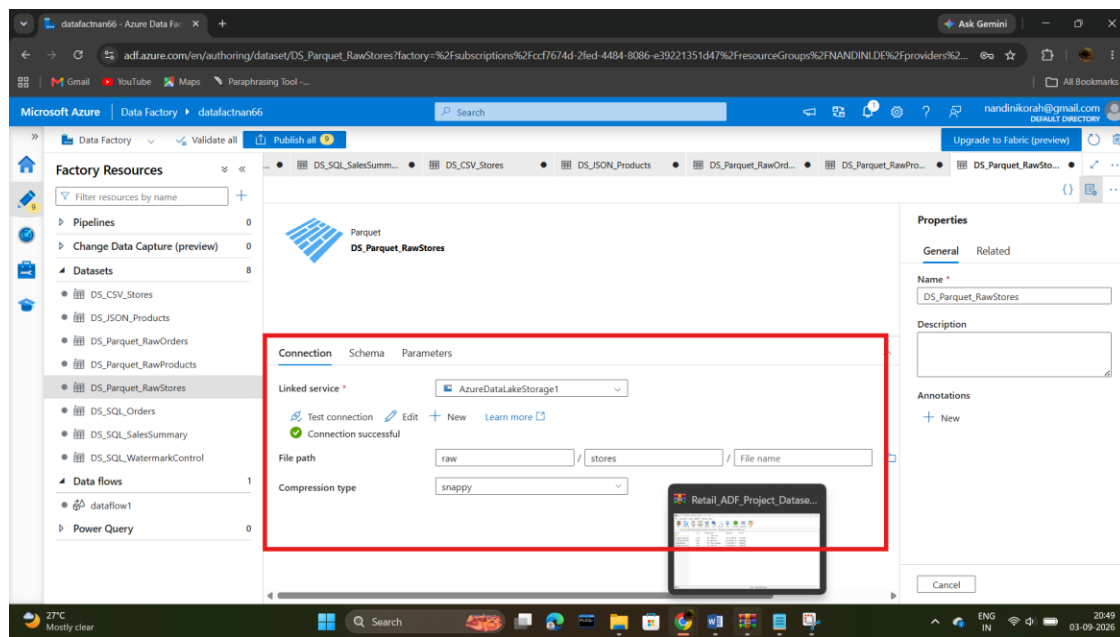
Parquet dataset (snappy compression) pointing at raw/orders. This is the sink for the Orders ingestion pipeline and, later, the source that the Mapping Data Flow reads from.

Figure 12. DS_Parquet_RawProducts



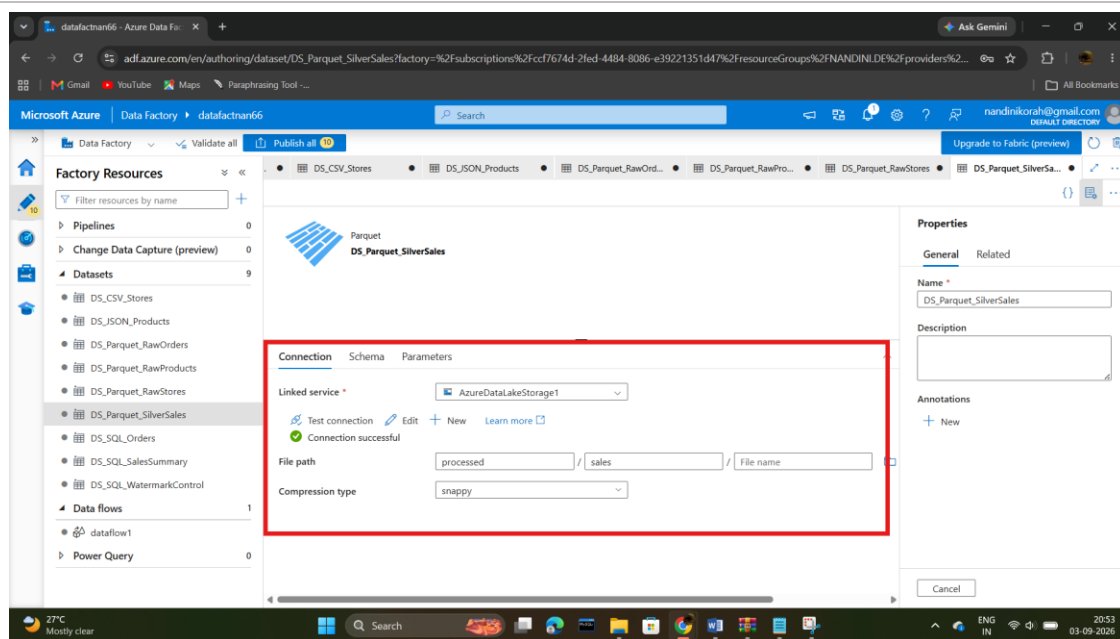
The equivalent Parquet dataset for the product catalog, pointing at raw/products — the target of the Products ingestion copy activity.

Figure 13. DS_Parquet_RawStores



Parquet dataset over raw/stores, populated by the Store Master ingestion pipeline from the CSV source dataset above.

Figure 14. DS_Parquet_SilverSales



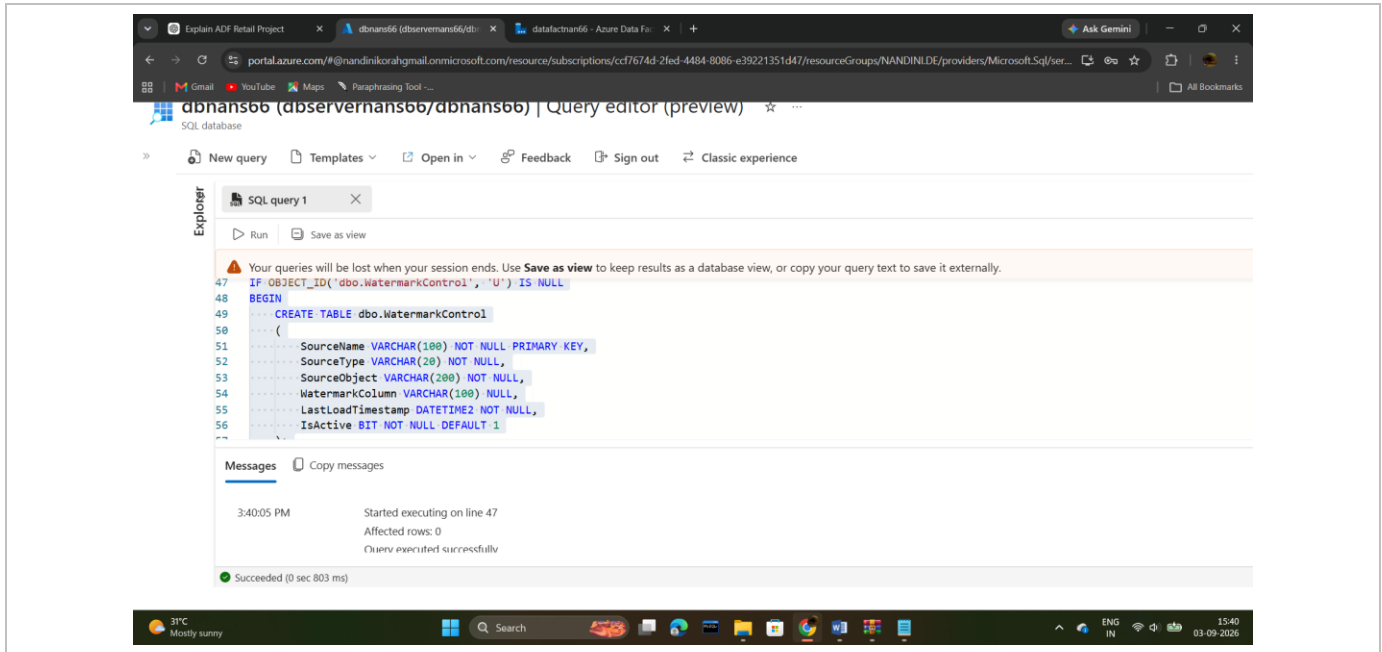
Parquet dataset over processed/sales — the intermediate output target used by the data flow's secondary sink, separate from the final Azure SQL curated table.

5. Incremental Load Design — the Watermark Control Table

Business requirement BR-04 calls for loading only new or changed records since the last successful run. Rather than hard-coding a "last run" value inside each pipeline, the design centralises watermark state in a single SQL control table, `dbo.WatermarkControl`, with one row per source. This is what makes the framework metadata-

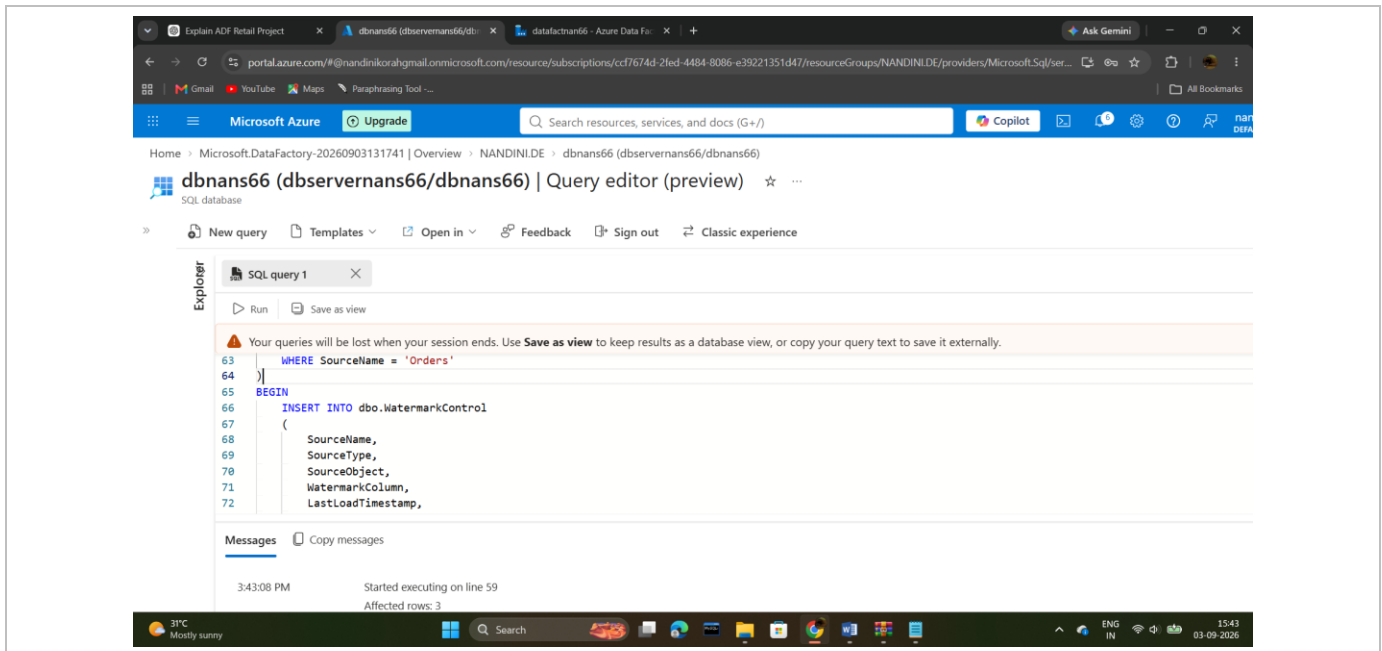
driven and, per BR-10, extensible to new sources without new pipeline code — onboarding a fourth source is, in principle, a new row in this table plus a parameterised pipeline run rather than a rebuild.

Figure 15. WatermarkControl DDL



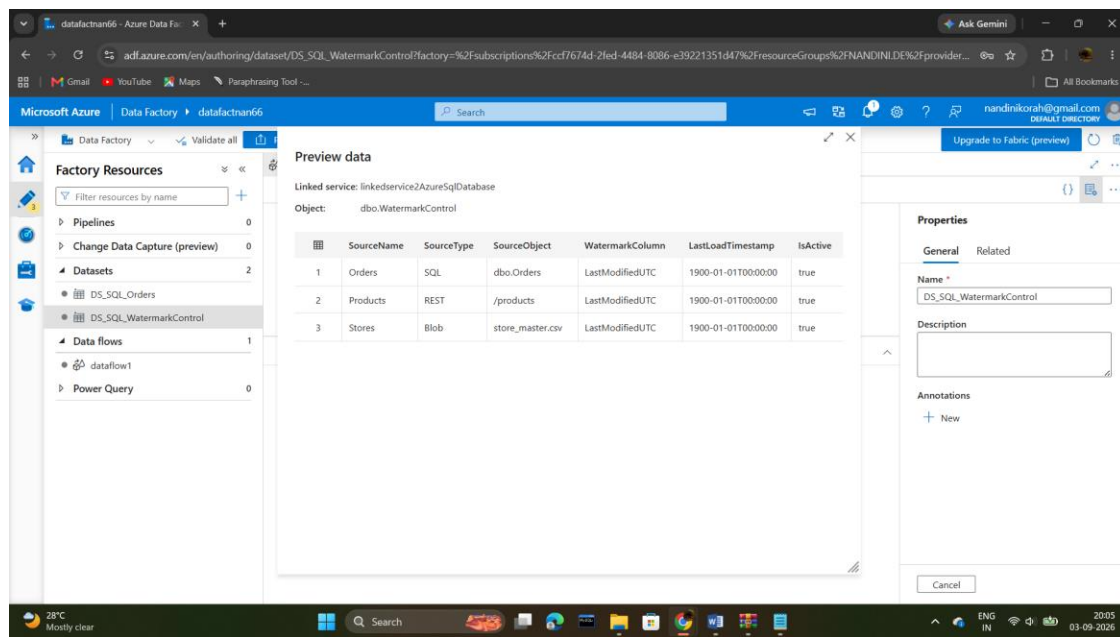
dbo.WatermarkControl is created with *SourceName* as the primary key, plus *SourceType*, *SourceObject*, *WatermarkColumn*, *LastLoadTimestamp* (DATETIME2) and an *IsActive* flag — enough metadata to drive the incremental-extraction logic generically across all three sources.

Figure 16. Seeding the baseline watermark rows



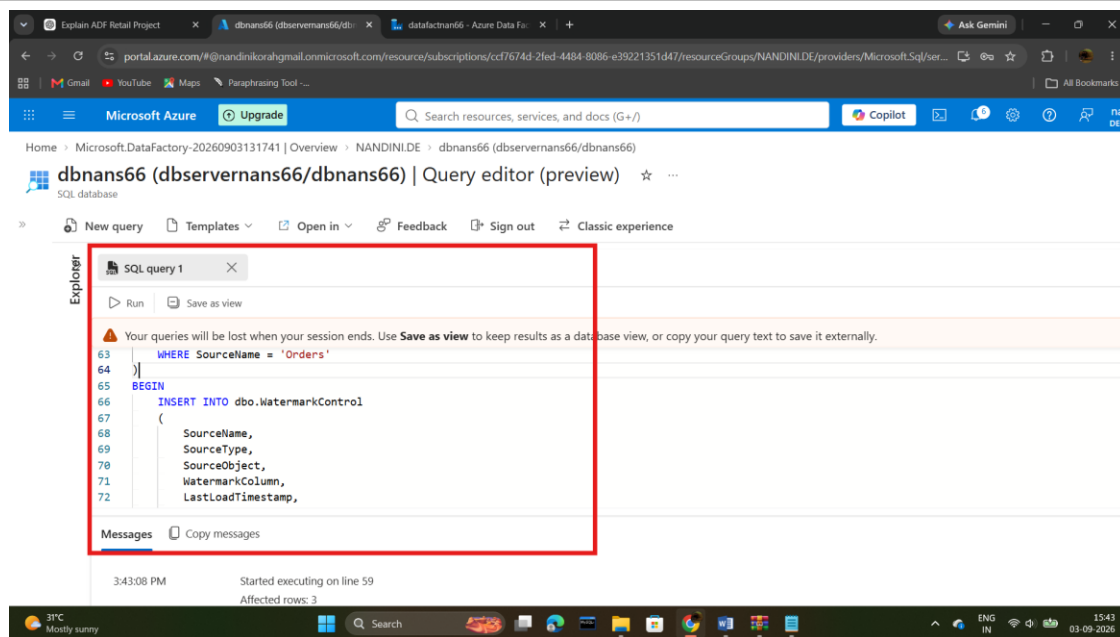
Three rows are inserted — *Orders*, *Products*, *Stores* — each defaulted to an early baseline timestamp so that the very first run of each pipeline performs a full load, after which every subsequent run becomes genuinely incremental.

Figure 17. Preview of the seeded control table



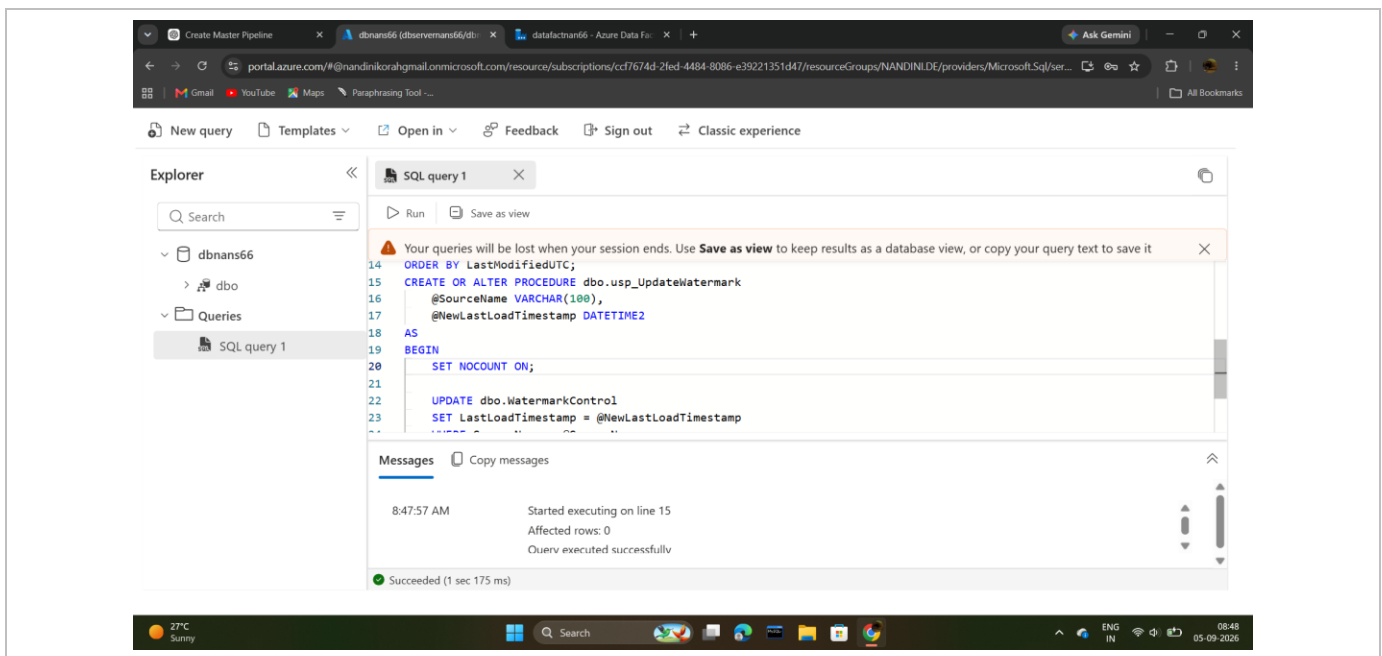
ADF's own "Preview data" pane against `DS_SQL_WatermarkControl` confirms all three sources are present and flagged `IsActive = true`, with their respective `WatermarkColumn (LastModifiedUTC)` recorded.

Figure 18. Authoring `usp_UpdateWatermark`



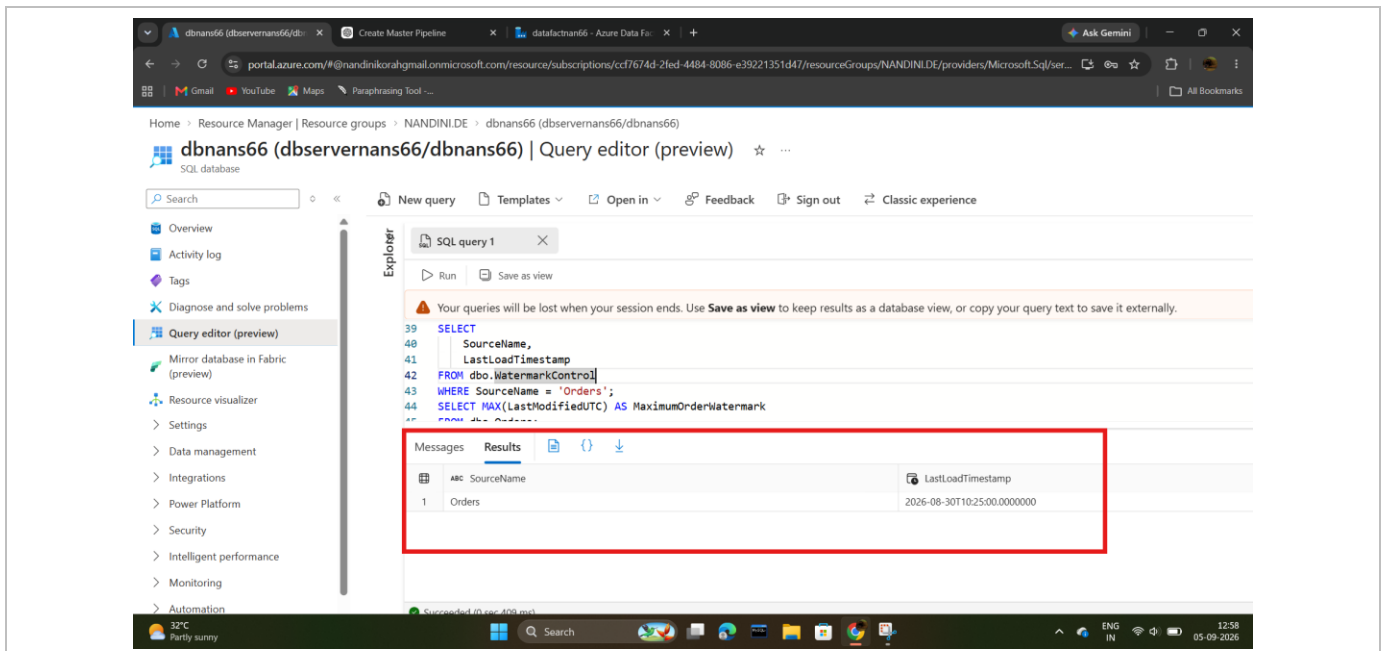
A stored procedure, parameterised on `@SourceName` and `@NewLastLoadTimestamp`, centralises how the watermark is advanced after a successful load — keeping that logic out of pipeline expressions and in one reusable, testable T-SQL object.

Figure 19. `usp_UpdateWatermark` executing successfully



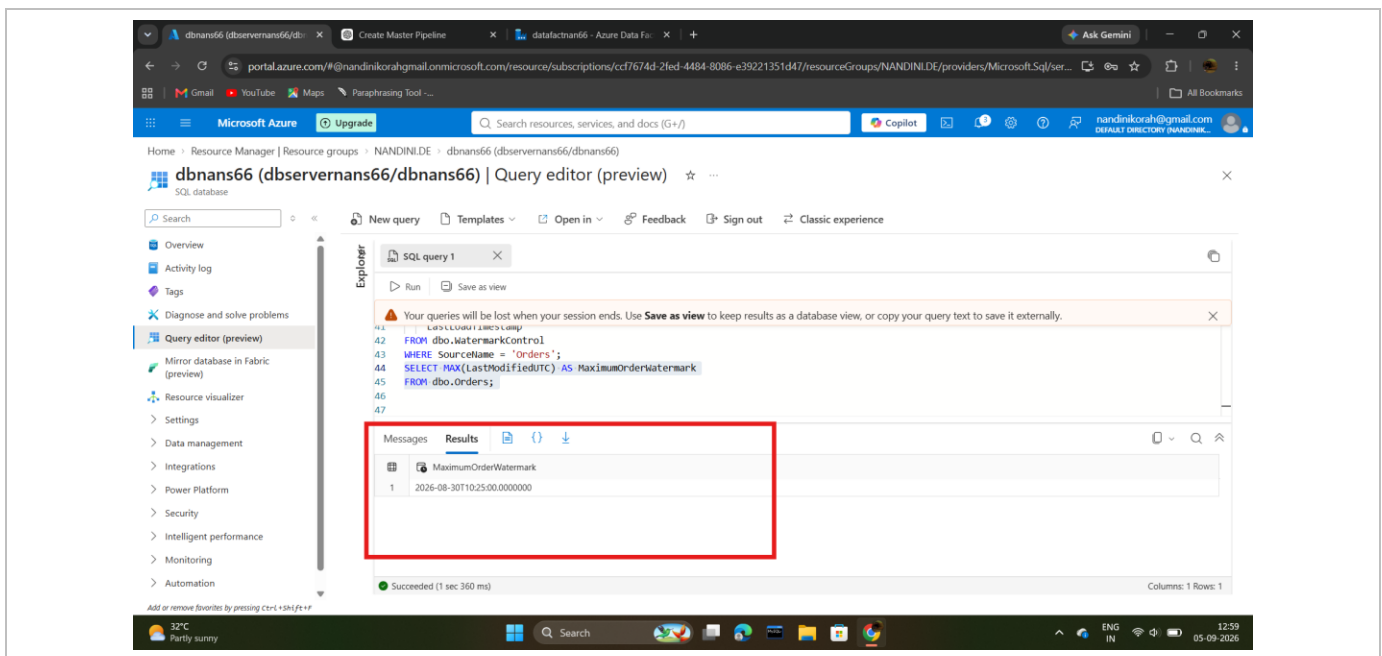
The stored procedure runs against `dbo.WatermarkControl` and returns "Query executed successfully" with the UPDATE statement targeting `LastLoadTimestamp` — this is the activity the master pipeline invokes at the end of every ingestion cycle.

Figure 20. Reading the prior watermark for Orders



A direct SQL query against `WatermarkControl` confirms the stored "old" watermark for the `Orders` source (2026-08-30T10:25:00) — this is the same value the pipeline's `Lookup_Old_Watermark` activity retrieves at runtime.

Figure 21. Establishing the "new" watermark from the source

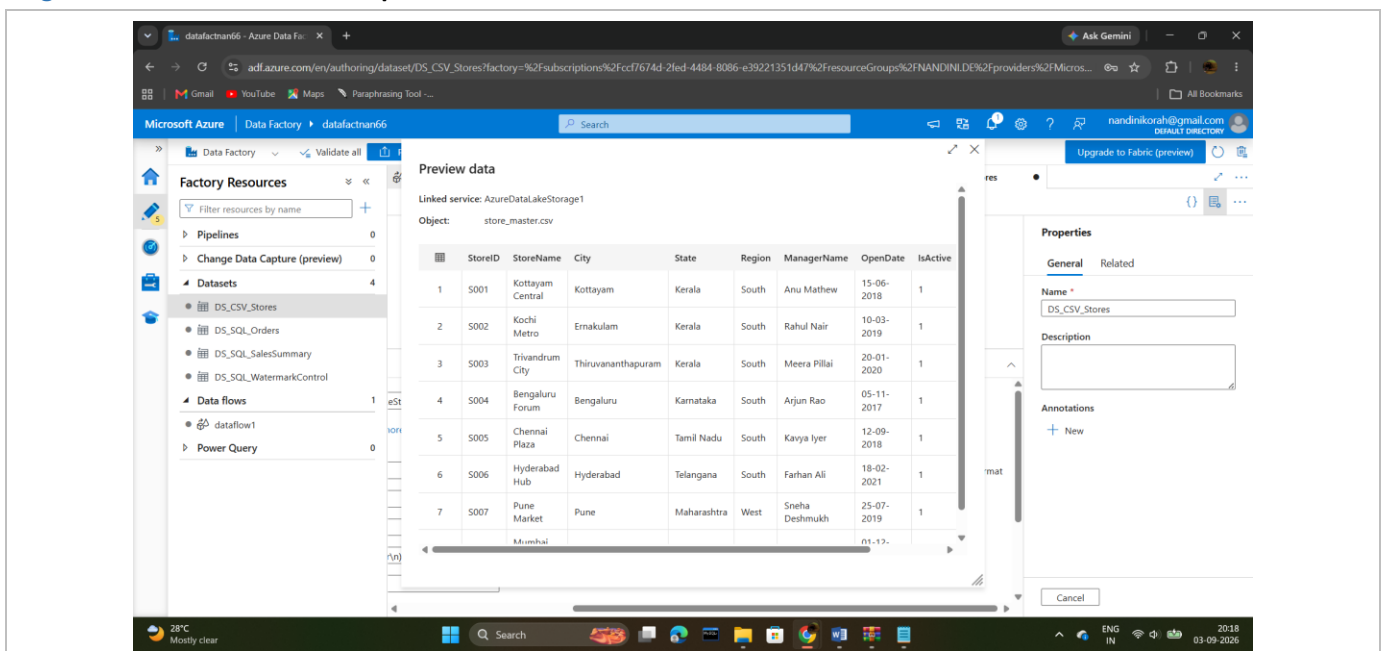


SELECT MAX(LastModifiedUTC) against dbo.Orders gives the ceiling for the current run. Comparing this against the value in step 20 is exactly what the pipeline does dynamically inside Lookup_New_Watermark.

6. Ingestion Layer — Landing the Three Sources

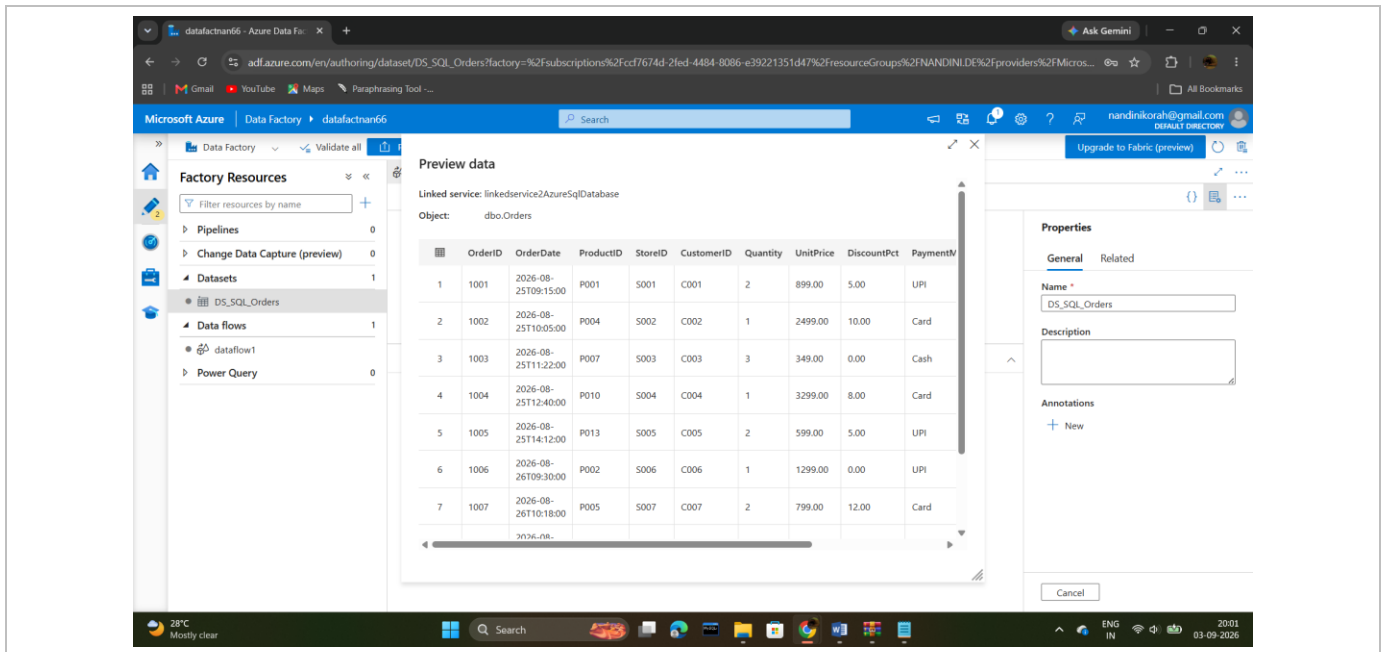
Three parallel copy-activity pipelines — CP_Orders_To_Raw, CP_Products_To_Raw and CP_Stores_To_Raw — move data from its native source format into standard Parquet files in the raw zone. Products and Stores are reloaded in full on each run (their volumes are small reference-style datasets); Orders is the one source large and fast-changing enough to warrant the watermark-based incremental pattern detailed in the previous section.

Figure 22. Store master source preview



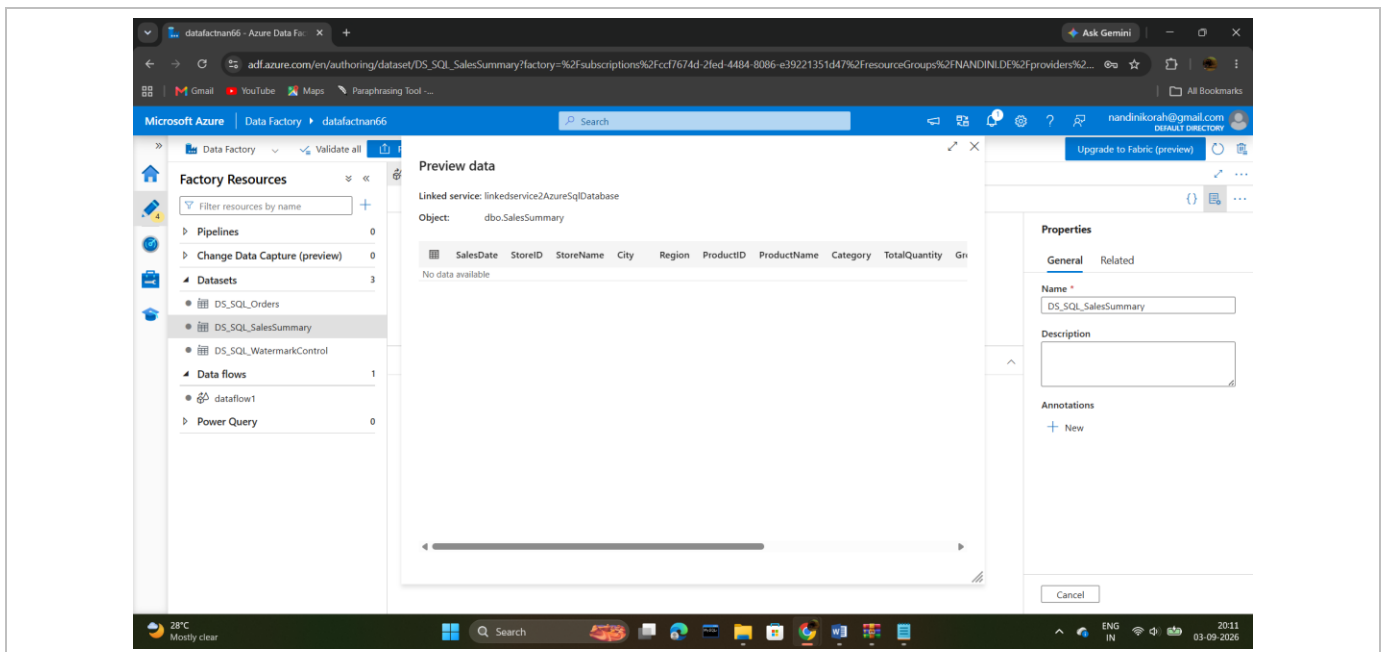
Live preview of `store_master.csv` showing `StoreID`, `StoreName`, `City`, `State`, `Region`, `ManagerName`, `OpenDate` and `IsActive` — this schema is what the Stores ingestion pipeline copies through unchanged into Parquet.

Figure 23. Orders source preview



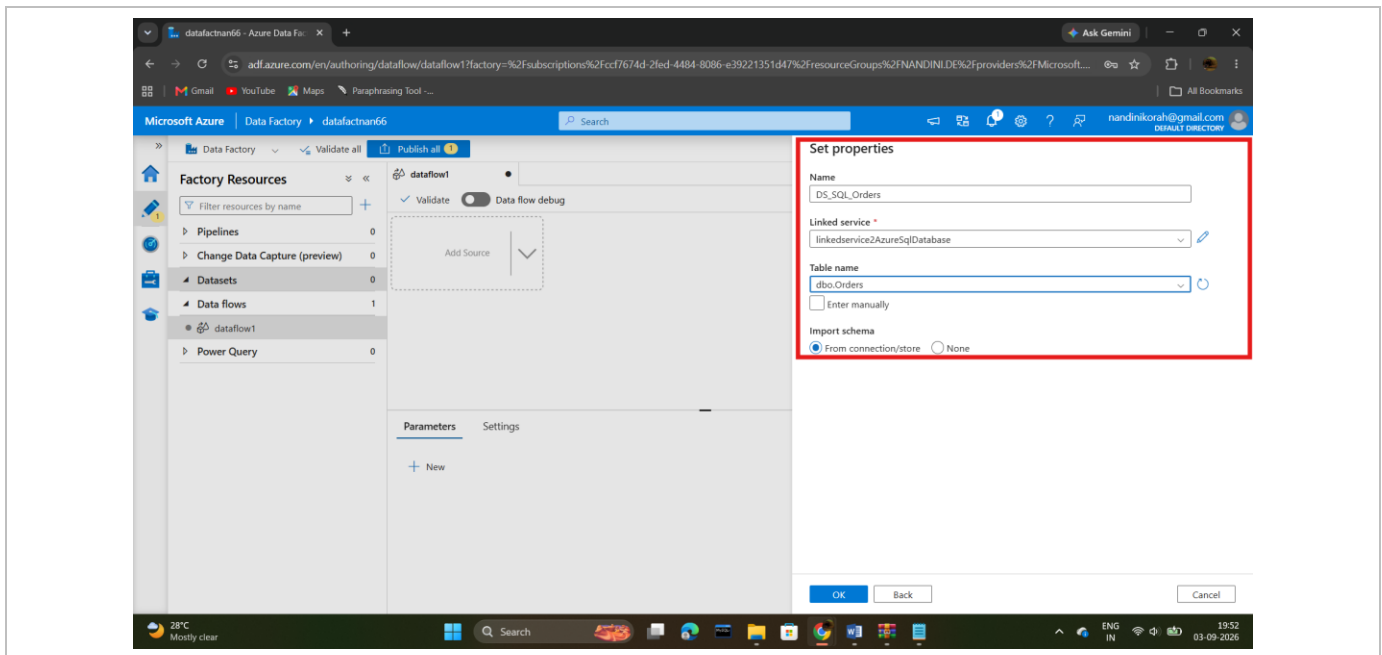
Preview of `dbo.Orders` shows the full transactional grain: `OrderID`, `OrderDate`, `ProductID`, `StoreID`, `CustomerID`, `Quantity`, `UnitPrice`, `DiscountPct` and `PaymentMethod` — the columns the rest of the pipeline cleanses and enriches downstream.

Figure 24. Curated sink table before the first run



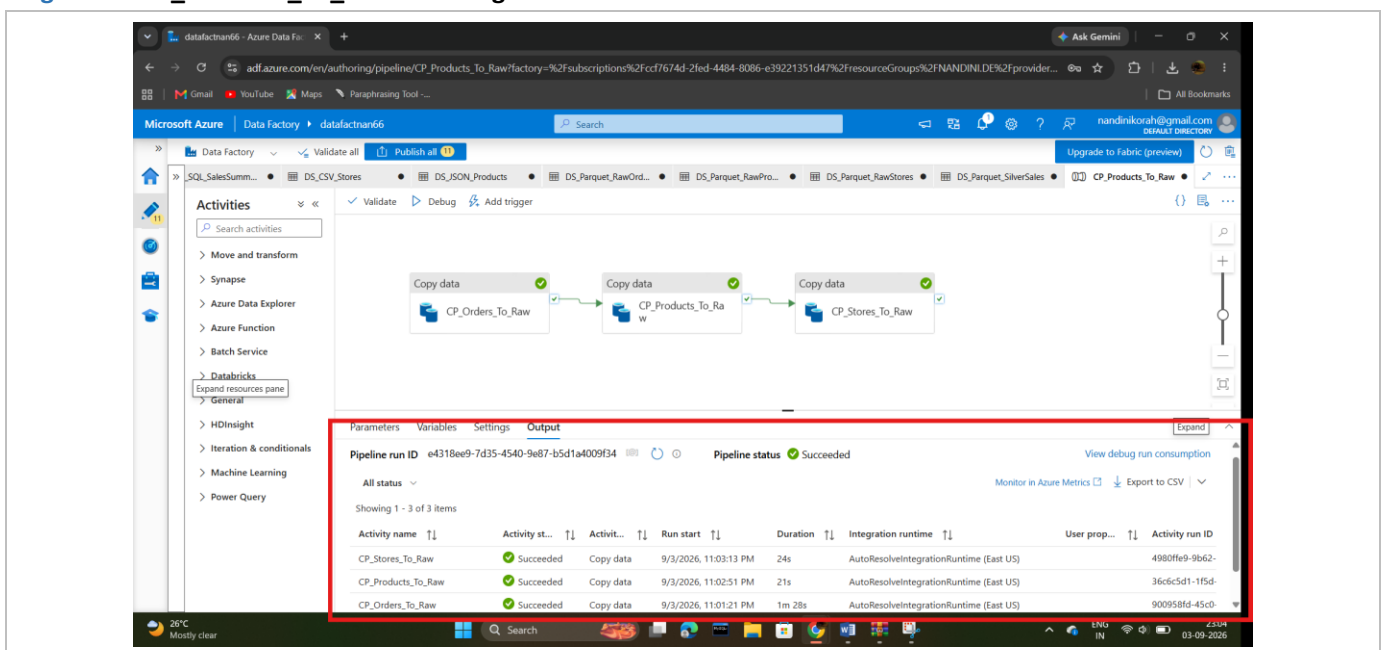
Before the transformation pipeline has executed even once, `dbo.SalesSummary` legitimately shows "No data available" — captured deliberately to document the starting state that the later result screenshots (see Section 12) are compared against.

Figure 25. Creating the Orders dataset



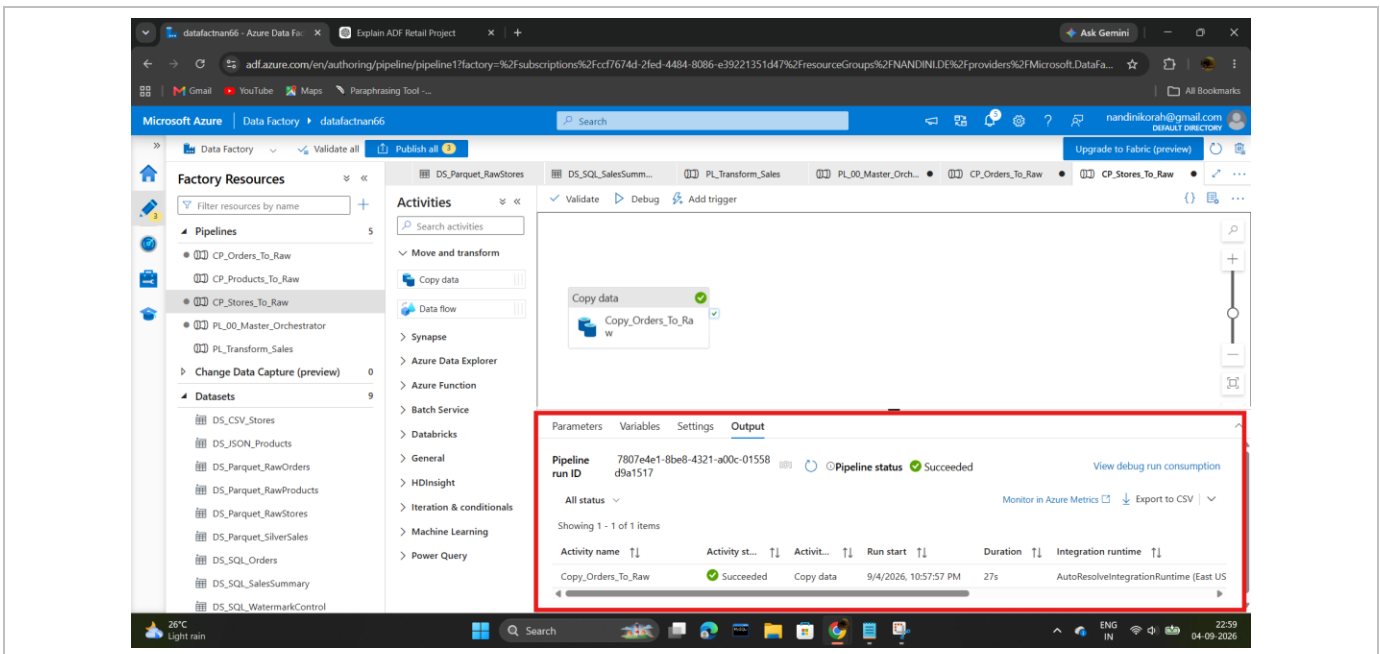
The "Set properties" wizard used to create `DS_SQL_Orders`, with schema imported directly from the connected store rather than typed manually — reducing the chance of a column mismatch between the dataset definition and the underlying table.

Figure 26. CP_Products_To_Raw — debug success



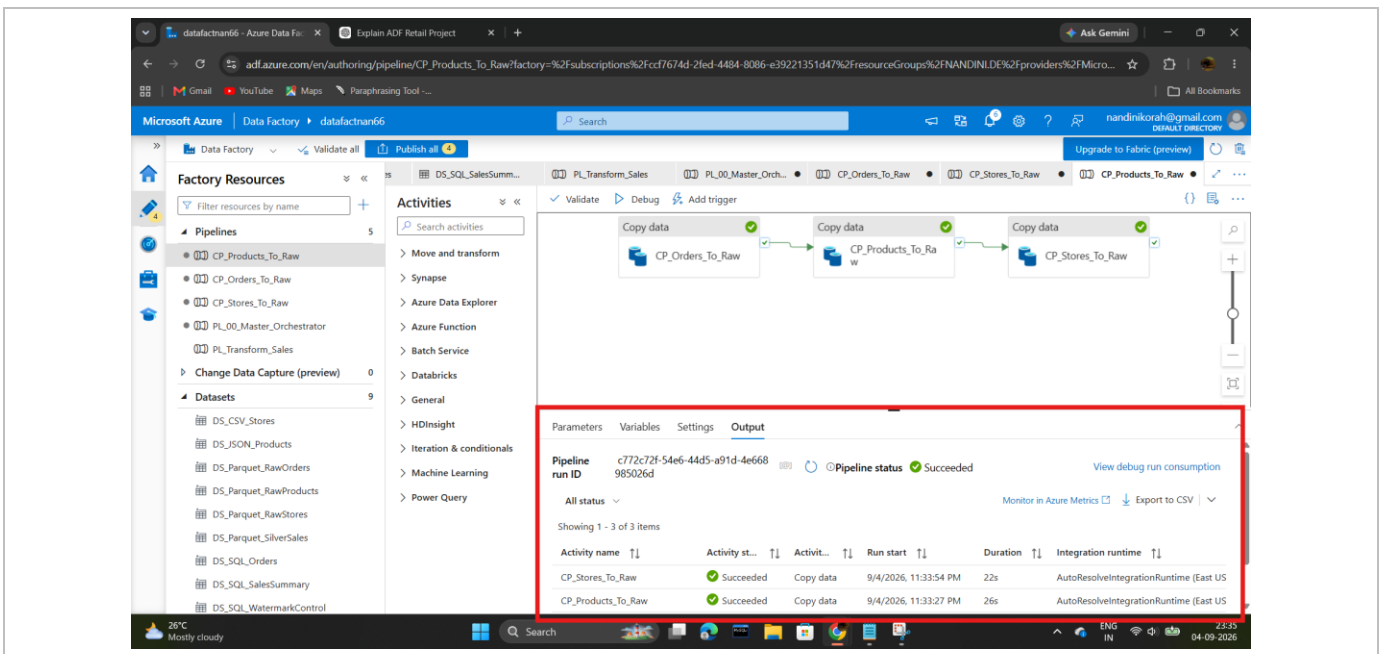
A debug run of the three chained copy activities (Orders, Products, Stores) inside this pipeline group all show Succeeded, each in well under 30 seconds — proof the metadata layer and linked services were wired up correctly before promoting to a scheduled run.

Figure 27. CP_Stores_To_Raw — debug success



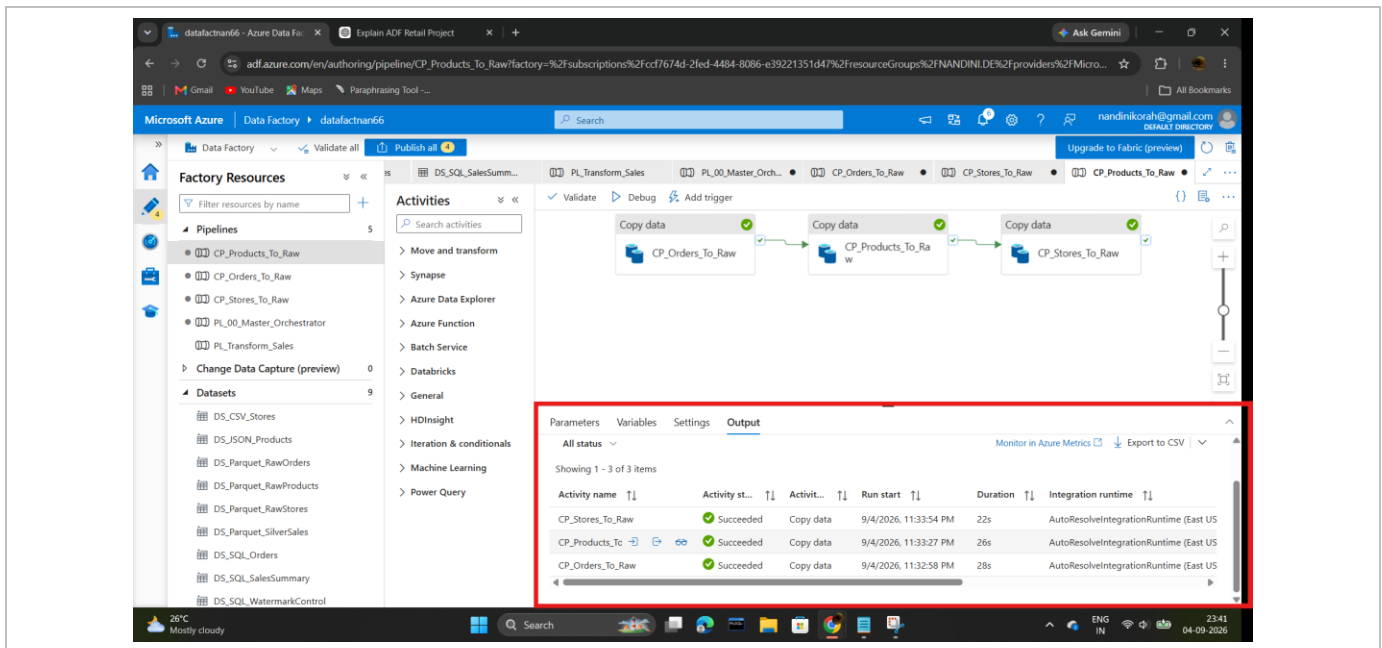
The Store Master copy activity completing in 24 seconds. Because this pipeline does a full reload each time, consistent short run times are the expected, healthy pattern.

Figure 28. CP_Orders_To_Raw — debug success



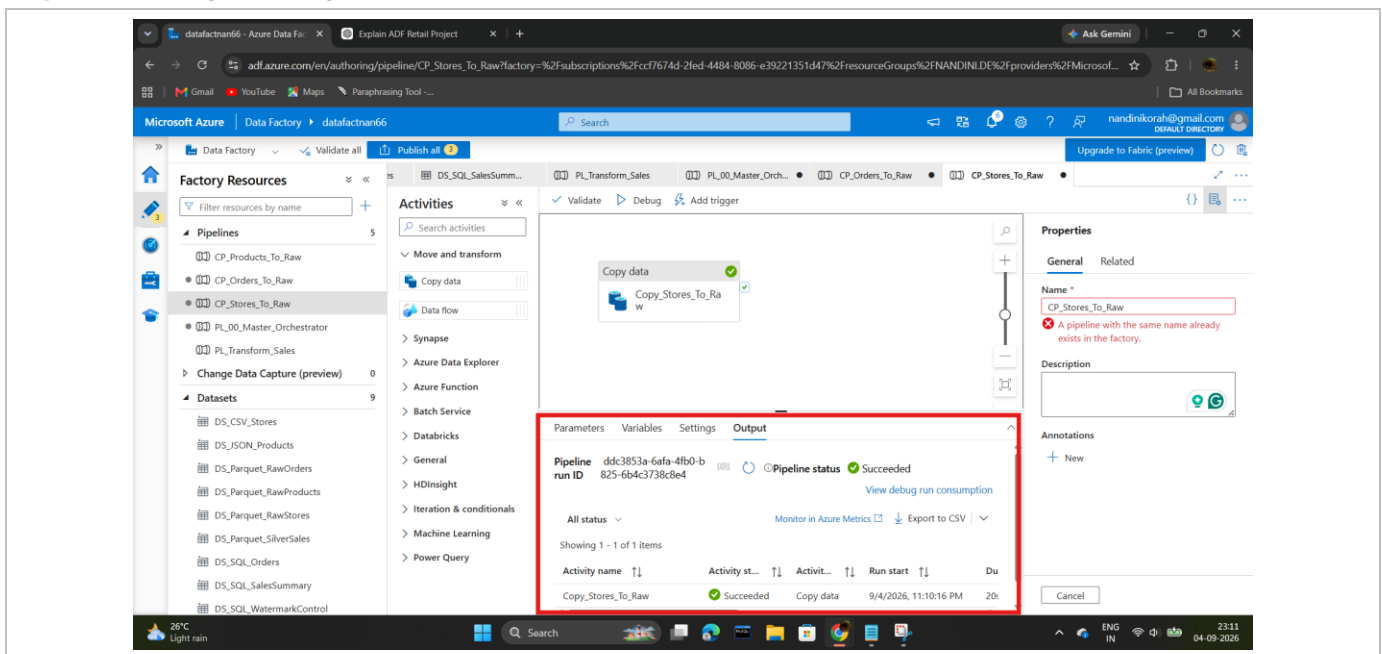
The Orders copy activity (Copy_Orders_To_Raw) succeeding as part of the incremental pipeline — this is one step inside the larger Lookup → Lookup → Copy → Set-variable chain described in Section 7.

Figure 29. Monitoring tab — three ingestion activities



The Output tab lists all three copy activities for this run with their individual durations (28s, 26s, 22s) and the AutoResolveIntegrationRuntime (East US) that executed them, giving an auditable per-activity timing record.

Figure 30. A caught naming error

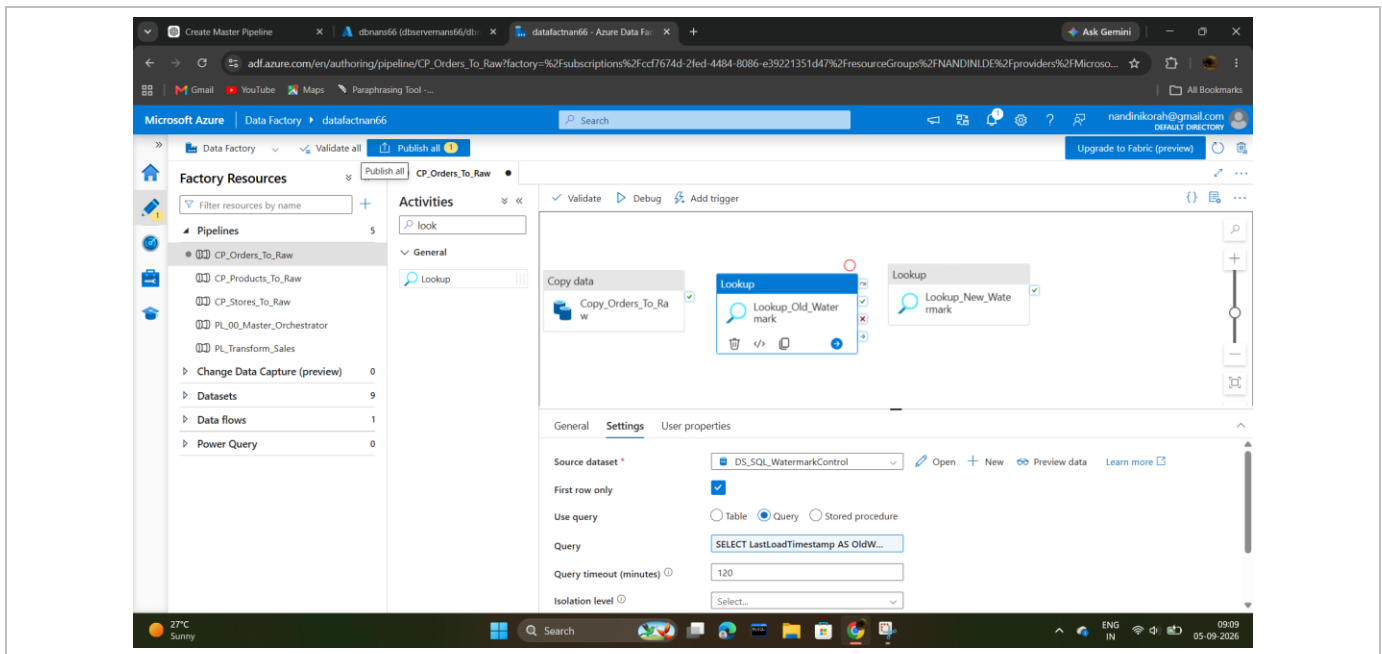


While iterating on the Stores pipeline, ADF's validation correctly blocked a save with "A pipeline with the same name already exists in the factory." This screenshot is included deliberately: it illustrates ADF's built-in guardrails and the kind of naming-collision issue that a governance/naming-convention step (per BR-08's promotion-across-environments requirement) needs to prevent.

7. Incremental Load Pipeline — Step-by-Step

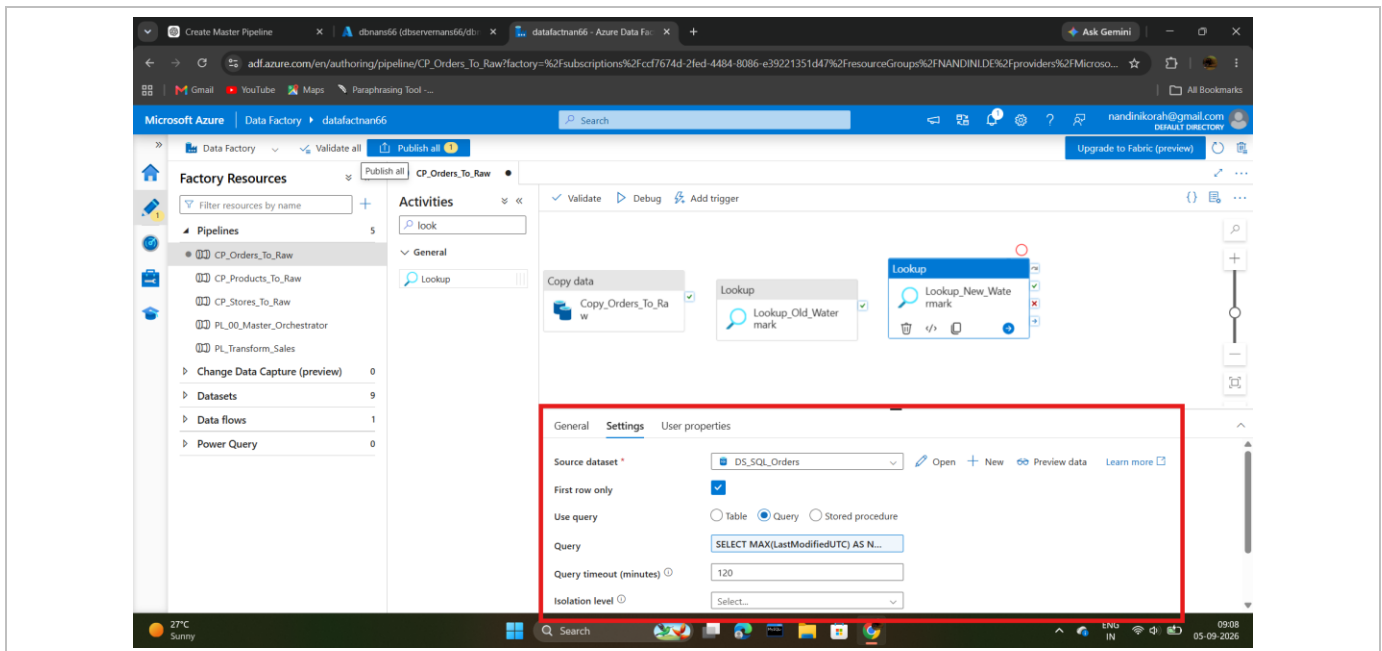
CP_Orders_To_Raw is the most involved of the three ingestion pipelines because it implements the watermark pattern end-to-end: read the old watermark, read the new ceiling from the source, pull only the delta, write it with a traceable filename, and then persist the new watermark for next time.

Figure 31. Lookup_Old_Watermark activity

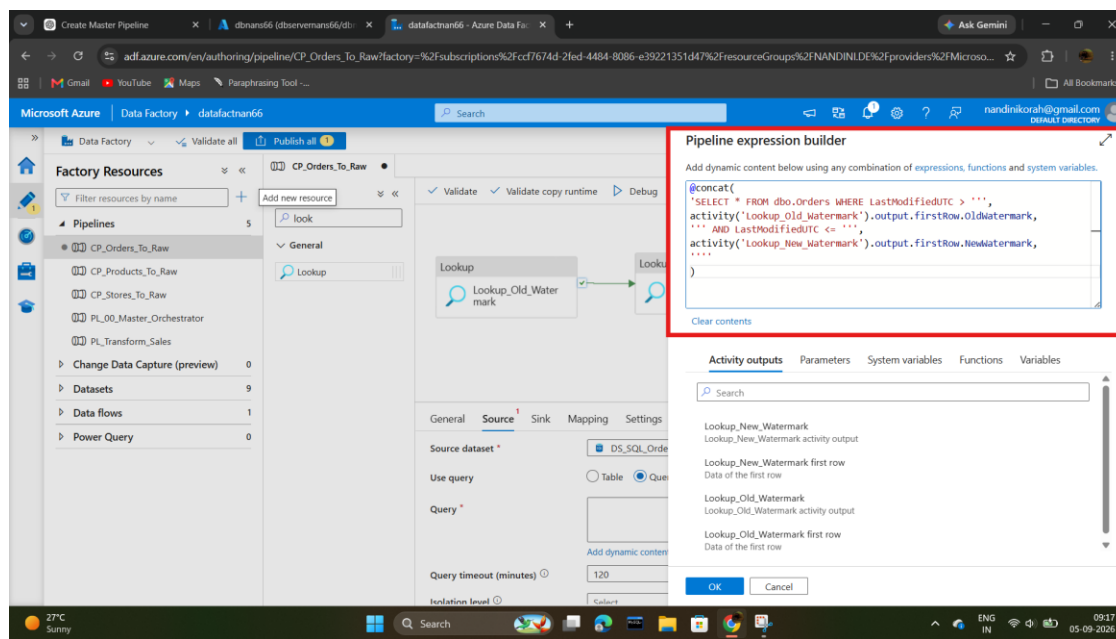


Configured against `DS_SQL_WatermarkControl` with "First row only" checked and the query `SELECT LastLoadTimestamp AS OldWatermark ... WHERE SourceName = 'Orders'` — this is the lower bound of the delta window.

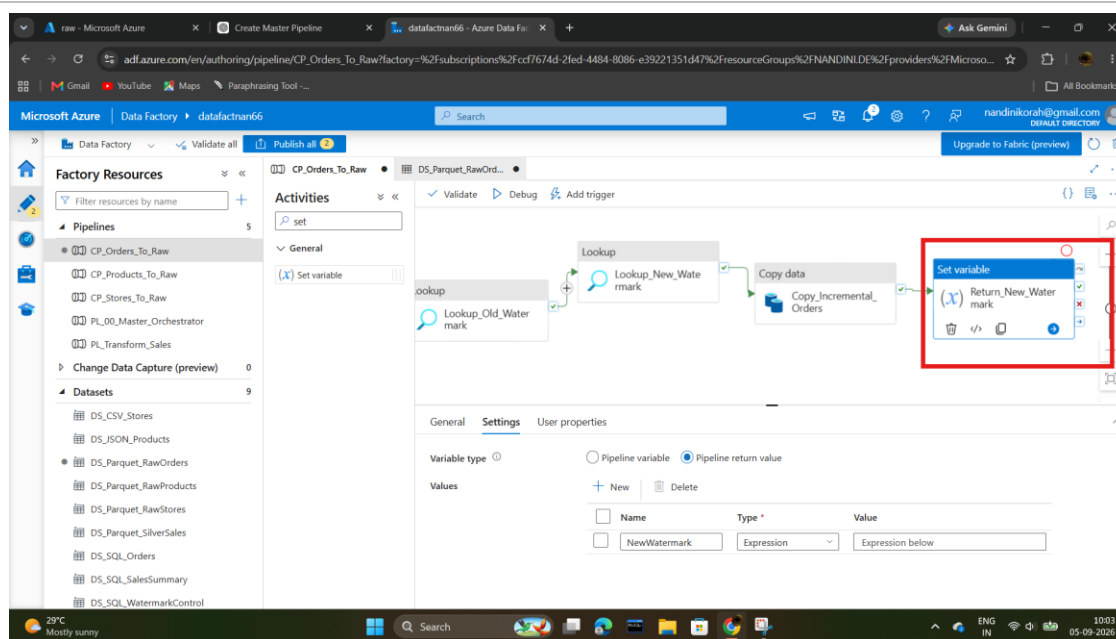
Figure 32. Lookup_New_Watermark activity



Configured against `DS_SQL_Orders` with `SELECT MAX(LastModifiedUTC) AS NewWatermark` — this is the upper bound, computed fresh from the source on every run rather than from a static configuration value.

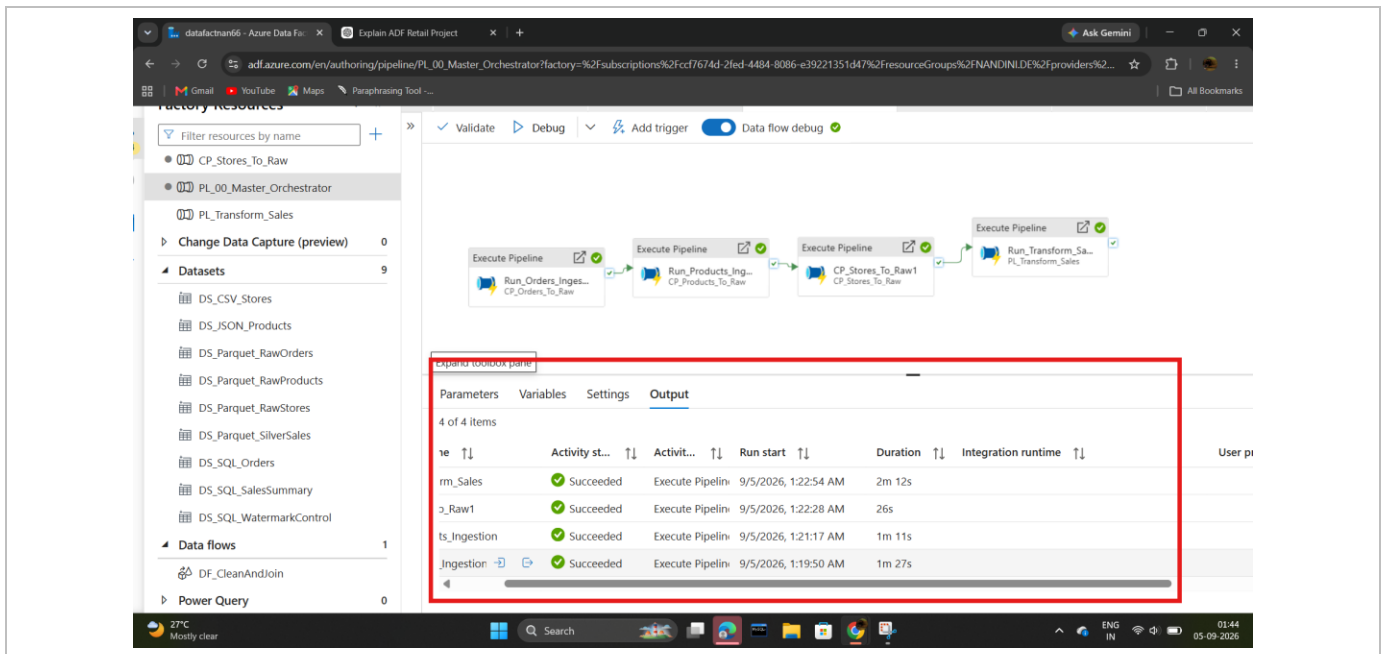
Figure 33. Building the dynamic delta query

The Pipeline Expression Builder concatenates a *SELECT* statement that filters *LastModifiedUTC > OldWatermark* AND *LastModifiedUTC <= NewWatermark*, referencing both lookup outputs by activity name — this single expression is what turns a static copy activity into an incremental one.

Figure 34. Set variable — Return_New_Watermark

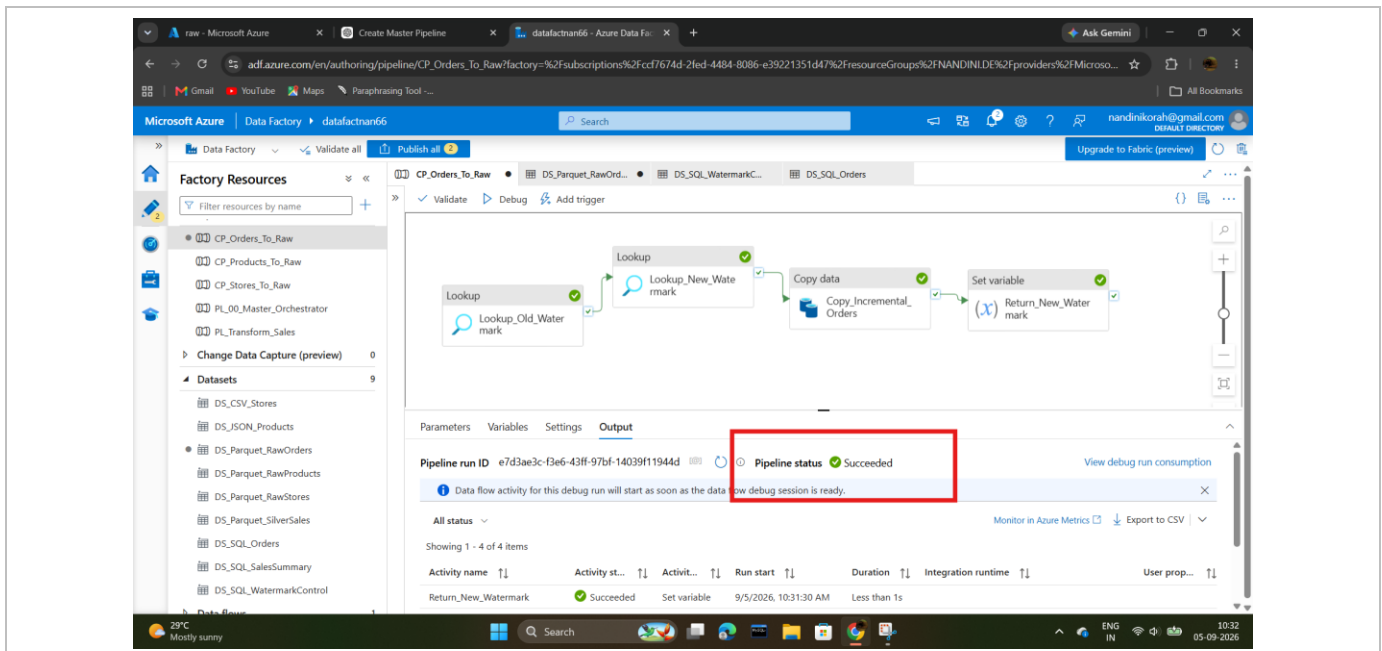
A pipeline-return-value variable captures the *NewWatermark* output so the calling pipeline (the master orchestrator) can pass it on to the stored procedure that finally commits it to *WatermarkControl*.

Figure 35. Dynamic, traceable output filenames



The sink's filename is generated as `concat('orders_', formatDateTime(NewWatermark, 'yyyyMMddHHmmssffffff'))` so that every incremental Parquet file in `raw/orders` is uniquely named and sortable by the watermark that produced it — useful for troubleshooting exactly which run wrote which file.

Figure 36. Full incremental pipeline — all four steps succeeded

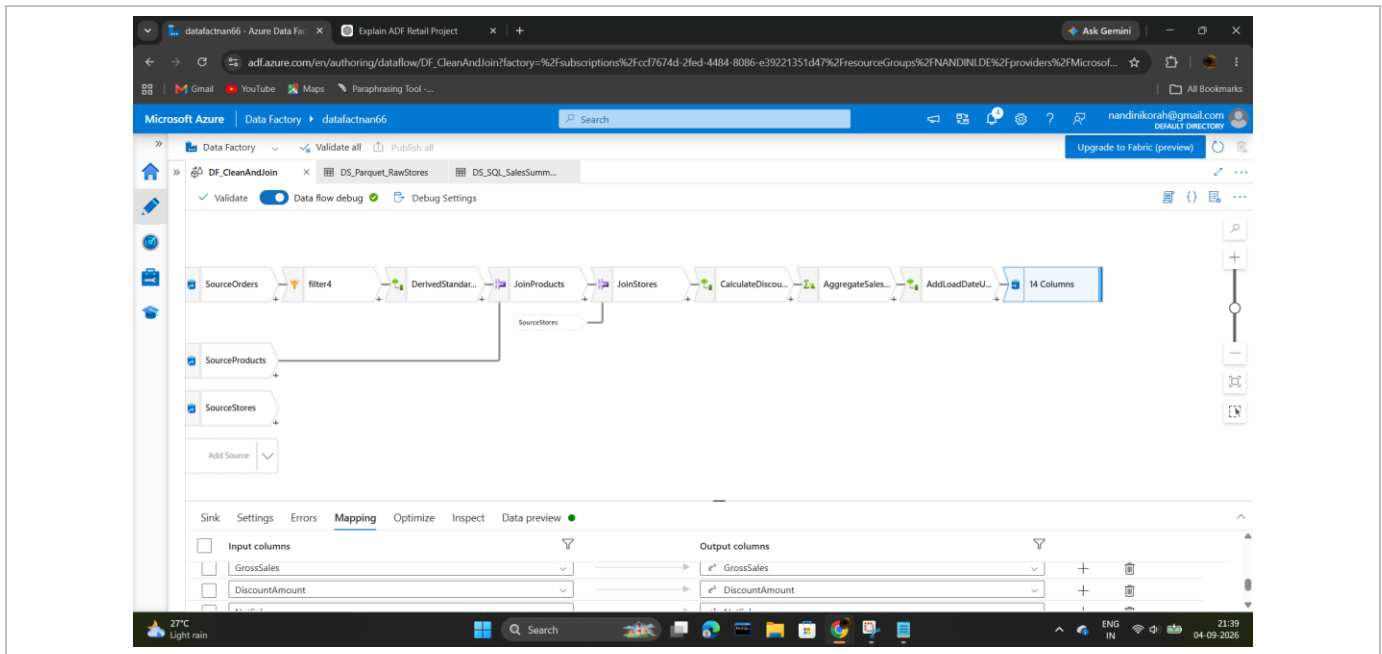


The complete chain — `Lookup_Old_Watermark` → `Lookup_New_Watermark` → `Copy_Incremental_Orders` → `Return_New_Watermark` — executed end-to-end with every step green, confirming the incremental design works as intended before it was wired into the master orchestrator.

8. Transformation Layer — the DF_CleanAndJoin Data Flow

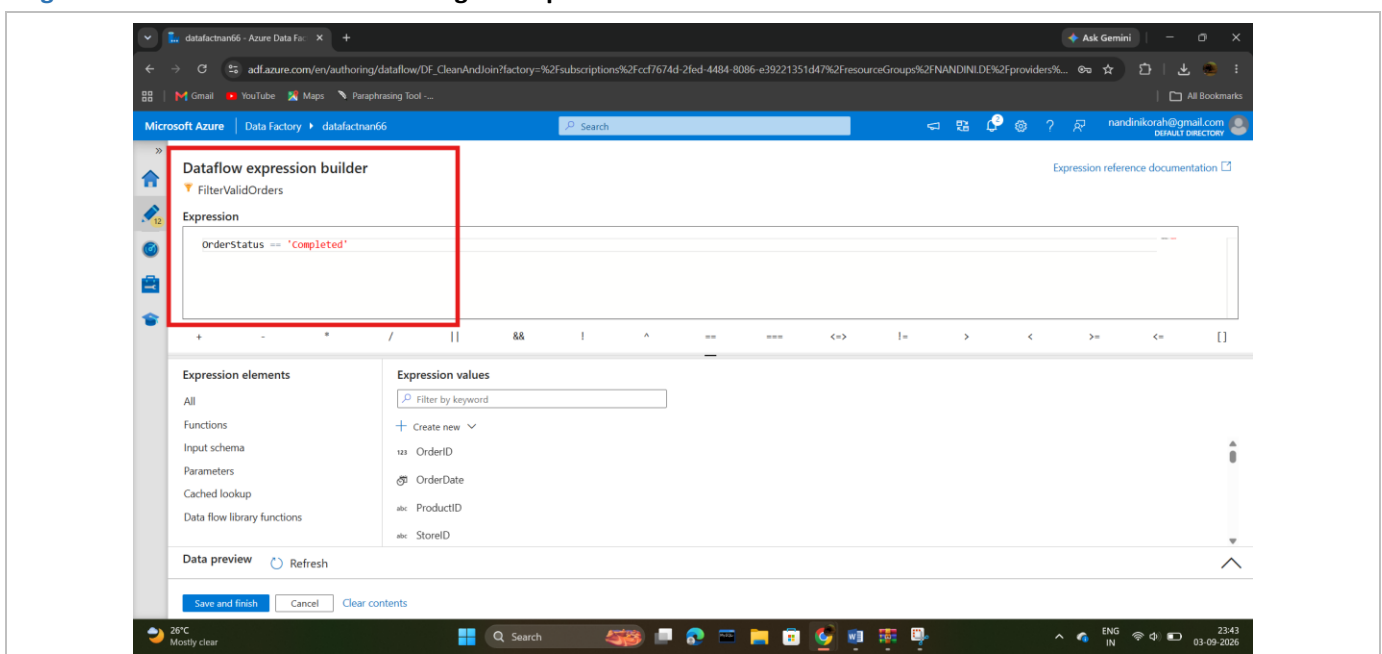
All cleansing, standardisation and joining (BR-05) happens inside a single reusable Mapping Data Flow, DF_CleanAndJoin, so that the transformation logic lives in one place regardless of which pipeline invokes it. The flow reads the three raw Parquet datasets, applies two parallel filtering branches on Orders (one feeding the join, one feeding an aggregate), enriches with derived columns, joins in Products and Stores reference data, aggregates, and finally writes both an intermediate Parquet copy and the curated SQL sink.

Figure 37. Full data flow canvas



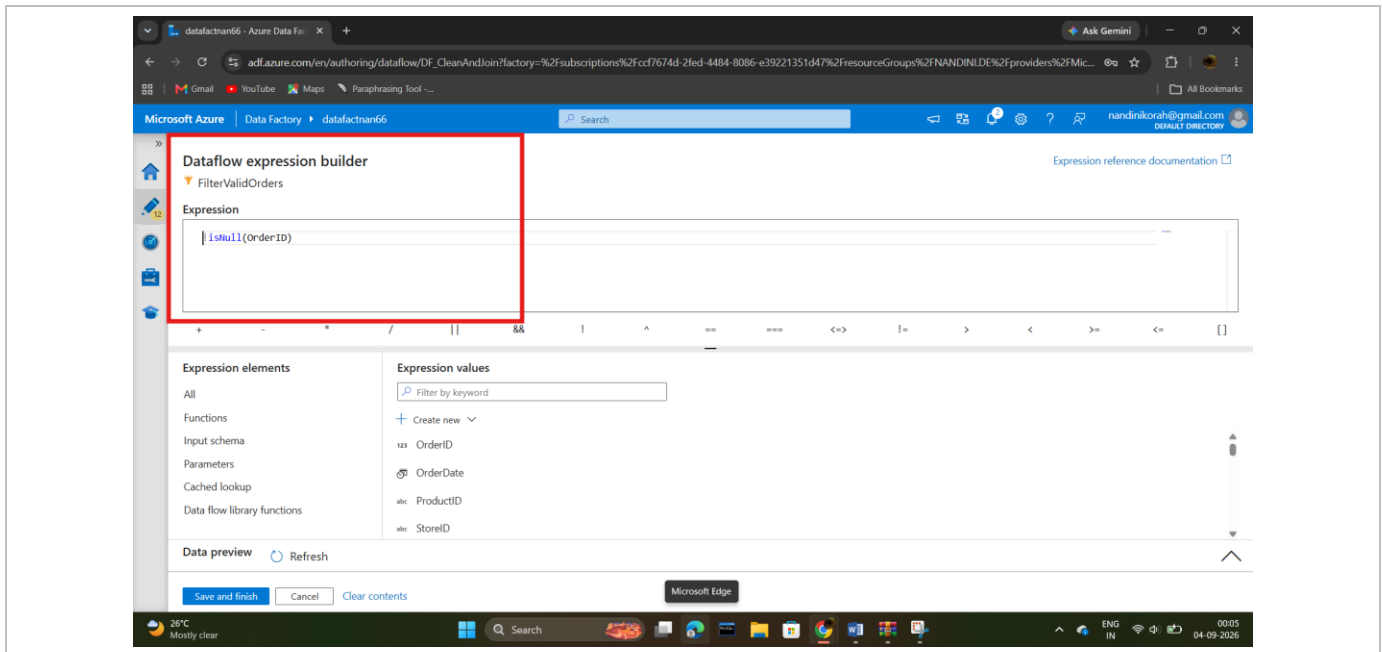
SourceOrders feeds two branches — one through filter4 → DerivedStandardize → JoinProducts → JoinStores → CalculateDiscount → AggregateSales → AddLoadDateUTC → a 14-column sink, the other through filter3 for a separate quantity aggregate. The Mapping tab on the right confirms GrossSales and DiscountAmount are wired through to the output schema.

Figure 38. FilterValidOrders — starting the expression



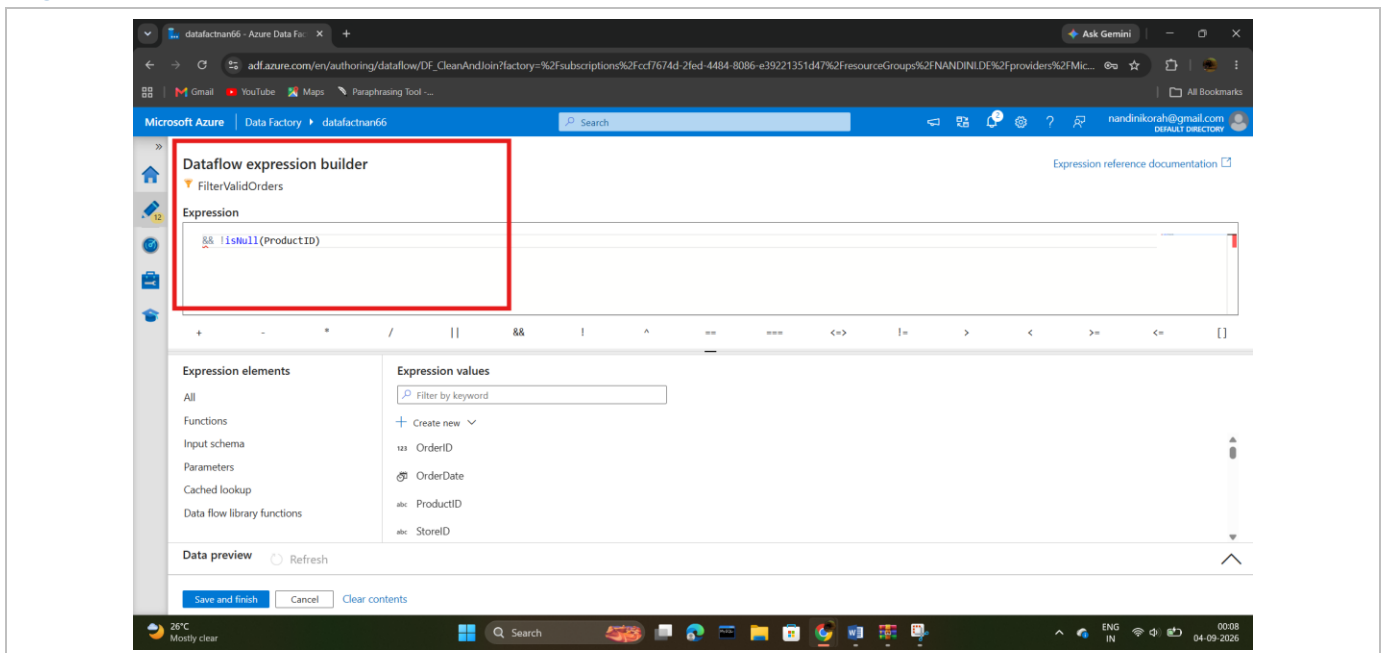
The first predicate written, *isNull(OrderID)*, rejects rows with no order identifier — the most basic row-validity check before anything more specific is layered on.

Figure 39. FilterValidOrders — adding ProductID validity



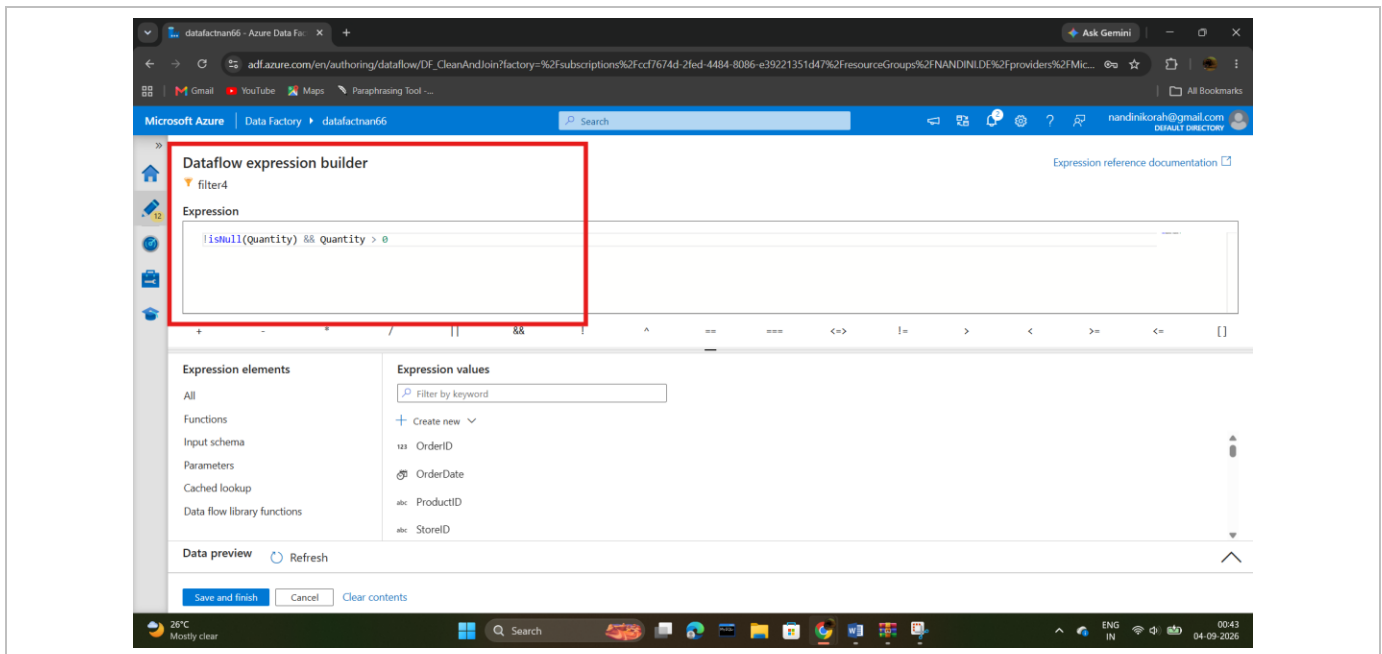
The expression is extended to *&& isNull(ProductID)*, so a row must have both a valid order key and a valid product reference to survive into the join.

Figure 40. FilterValidOrders — final business rule



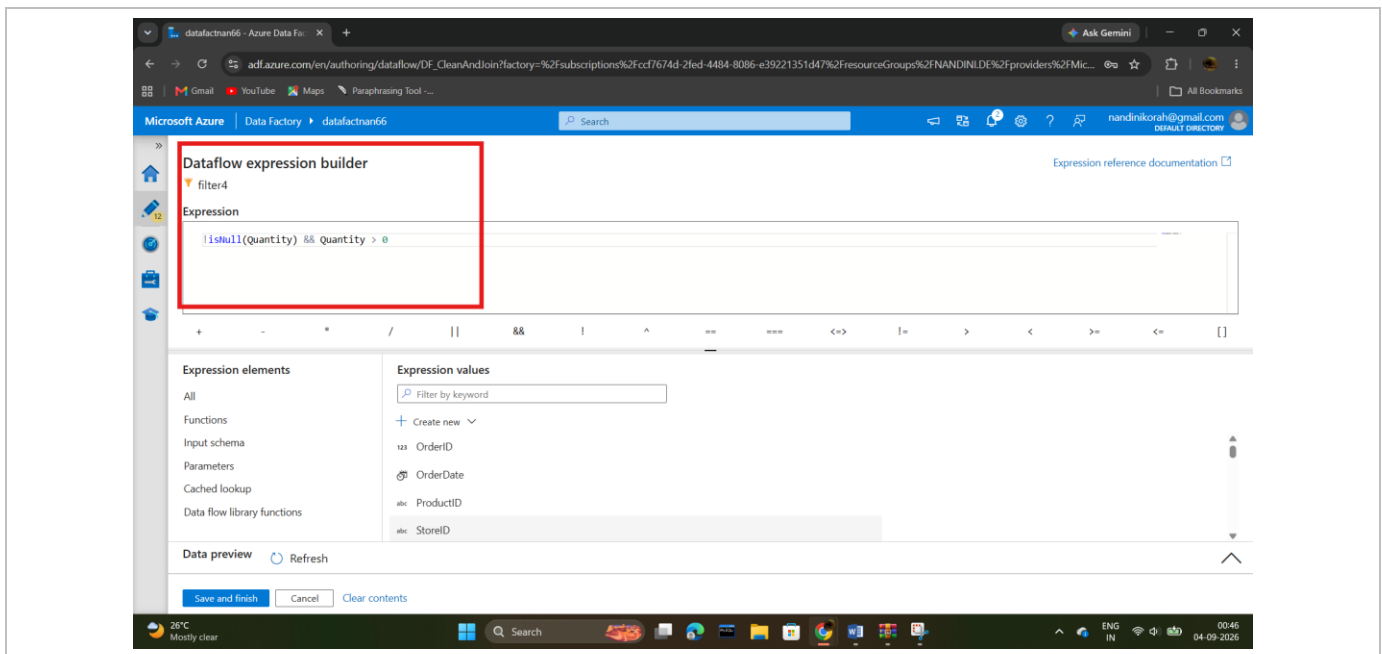
The completed filter also requires *OrderStatus == 'Completed'*, ensuring the curated Sales Summary only ever reflects finalised transactions rather than pending or cancelled orders.

Figure 41. filter4 — validating Quantity



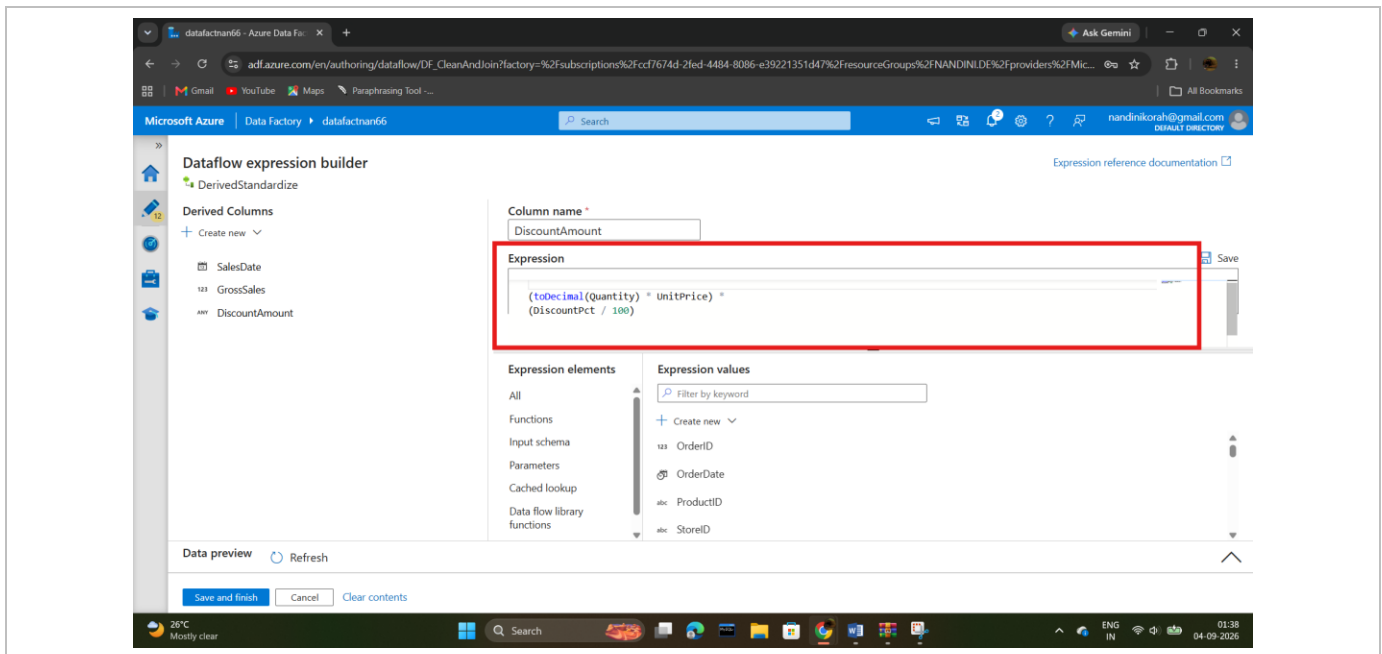
!isNull(Quantity) && Quantity > 0 removes any row with a missing or non-positive quantity — a simple but important data-quality guard directly supporting BR-11's row-level validation requirement.

Figure 42. filter4 — confirmed against the input schema



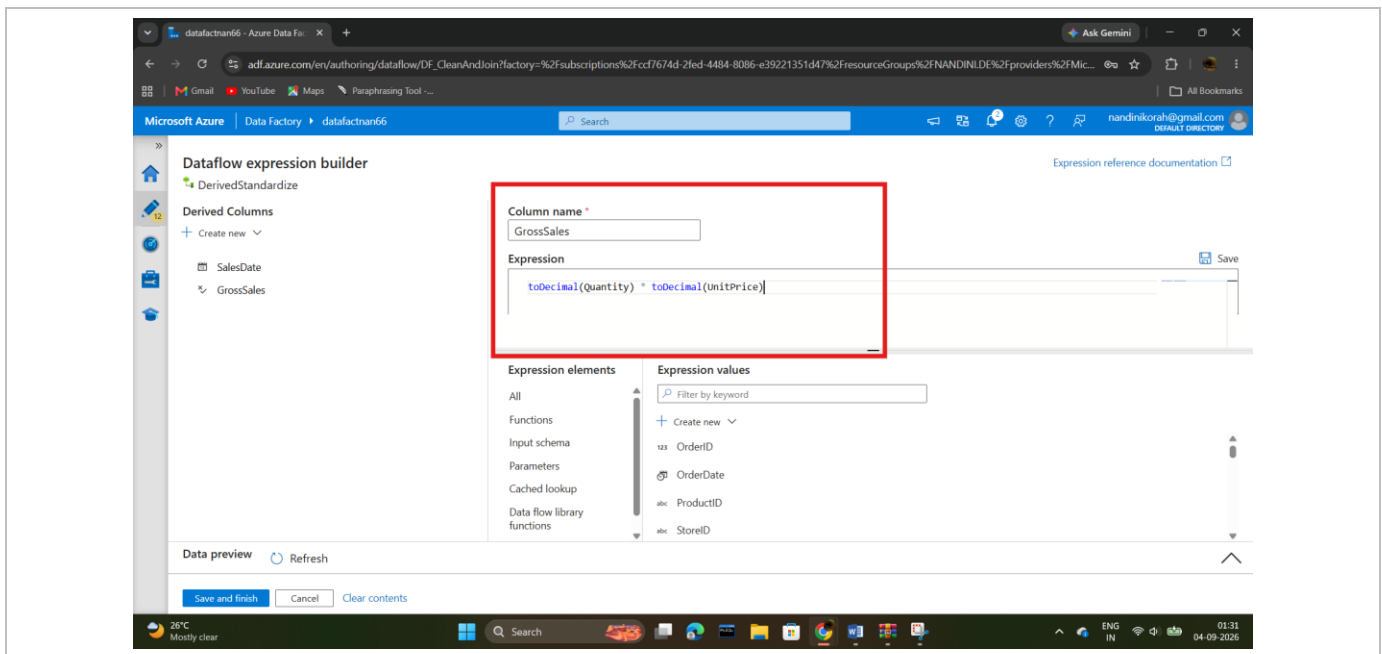
The same expression re-validated once the upstream schema is fully resolved, with the filter icon on the canvas turning green to indicate no expression errors remain.

Figure 43. Derived column — DiscountAmount



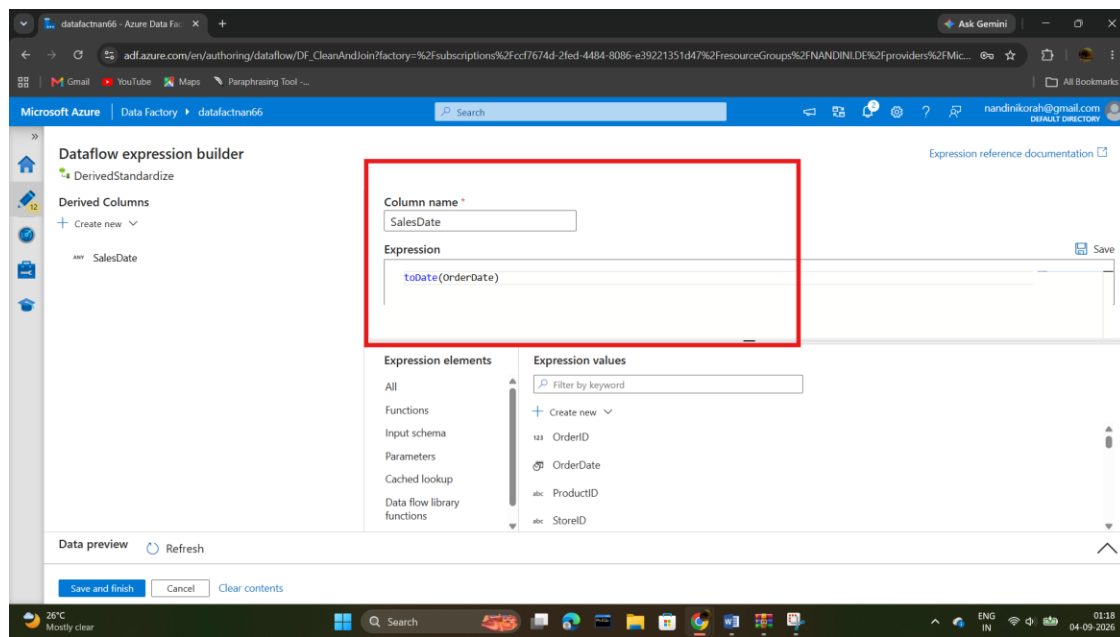
$(toDecimal(Quantity) * UnitPrice) * (DiscountPct / 100)$ calculates the monetary discount applied to each line, explicitly casting Quantity to decimal first to avoid integer-division surprises.

Figure 44. Derived column — GrossSales



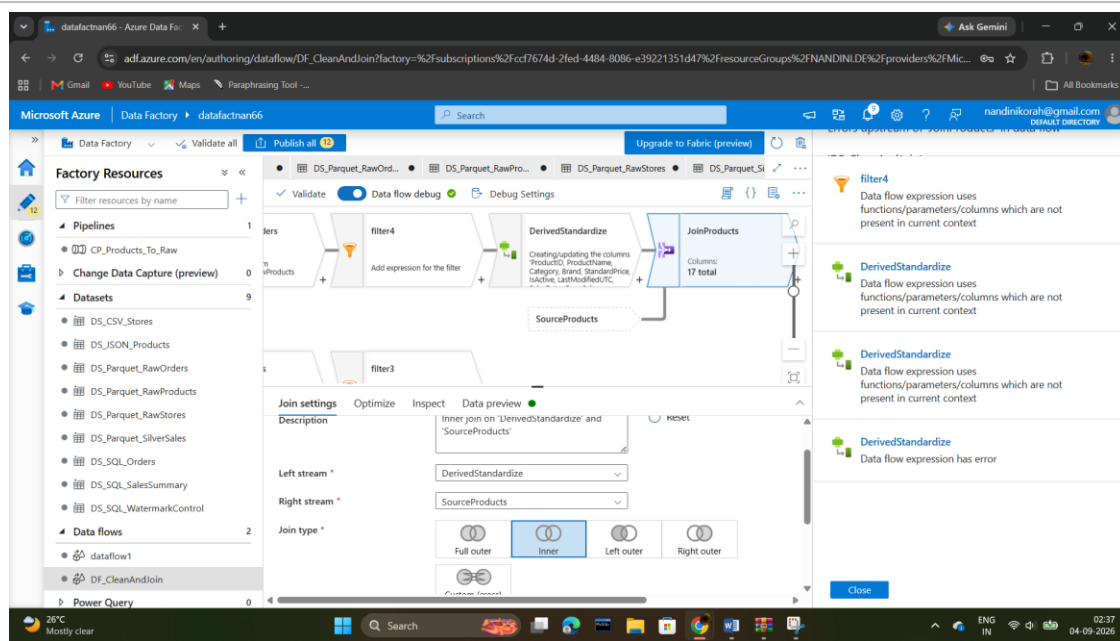
$toDecimal(Quantity) * toDecimal(UnitPrice)$ computes the pre-discount line value that downstream aggregation and reporting are built on.

Figure 45. Derived column — SalesDate



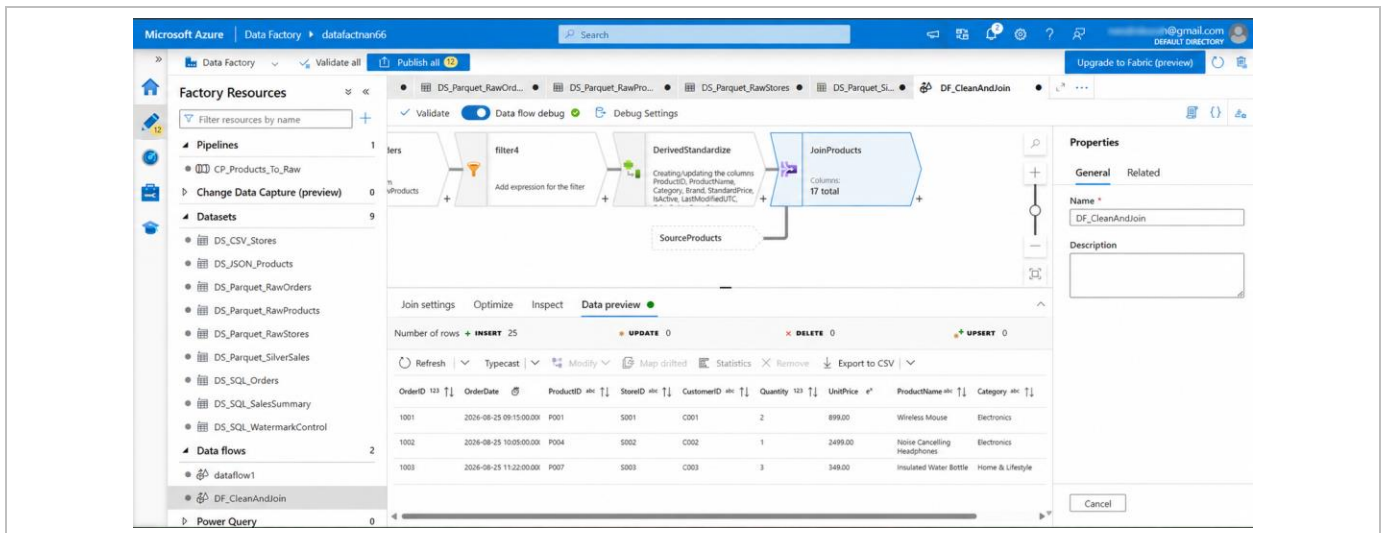
toDate(OrderDate) truncates the full order timestamp down to a calendar date, giving Power BI a clean grouping key that is independent of the exact time of day an order was placed.

Figure 46. A join in a broken intermediate state



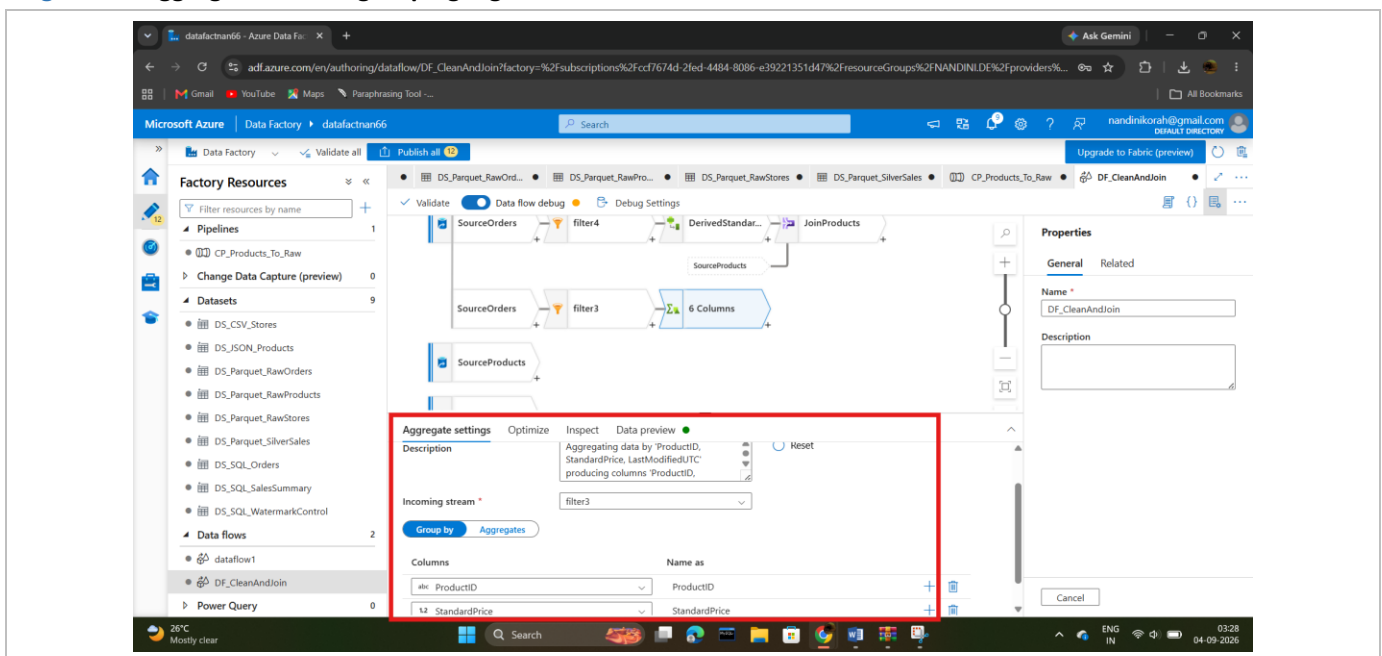
Captured mid-build: the data-flow validation panel flags DerivedStandardize and filter4 as "using functions/parameters/columns which are not present in current context." Kept here deliberately to show that the finished flow (next screenshot) is the result of iterative debugging, not a first-time-correct design.

Figure 47. JoinProducts — corrected and previewing live data



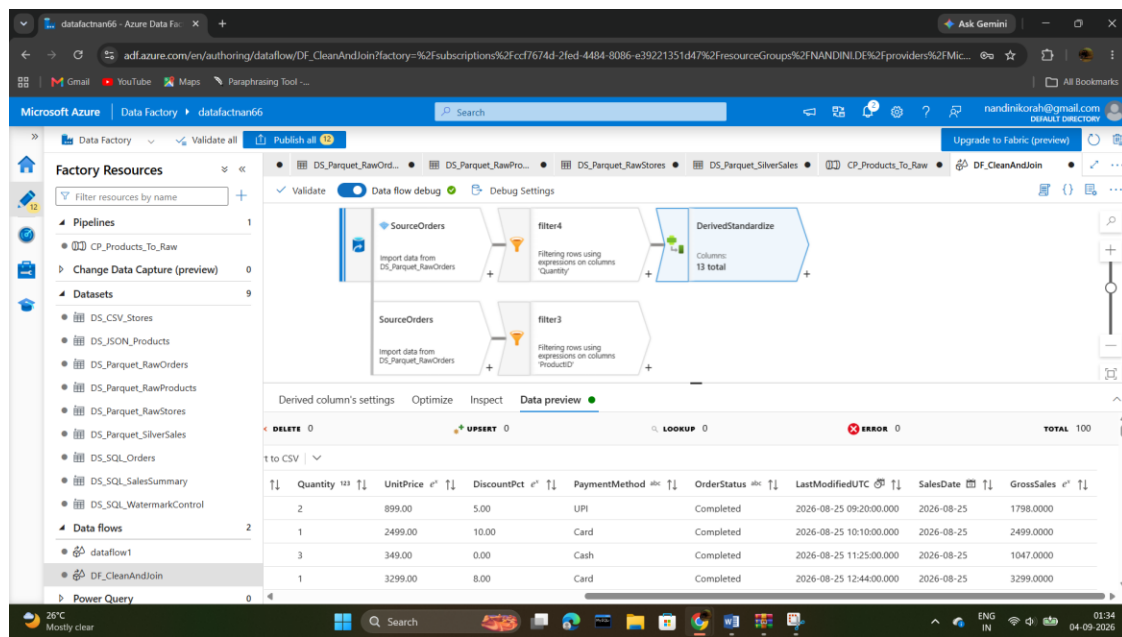
After resolving the upstream schema issue, the inner join between *DerivedStandardize* and *SourceProducts* produces a clean live preview — 25 rows inserted, sample columns including *OrderID*, *ProductID*, *StoreID*, *CustomerID*, *ProductName* and *Category* all populated correctly.

Figure 48. AggregateSales — grouping logic



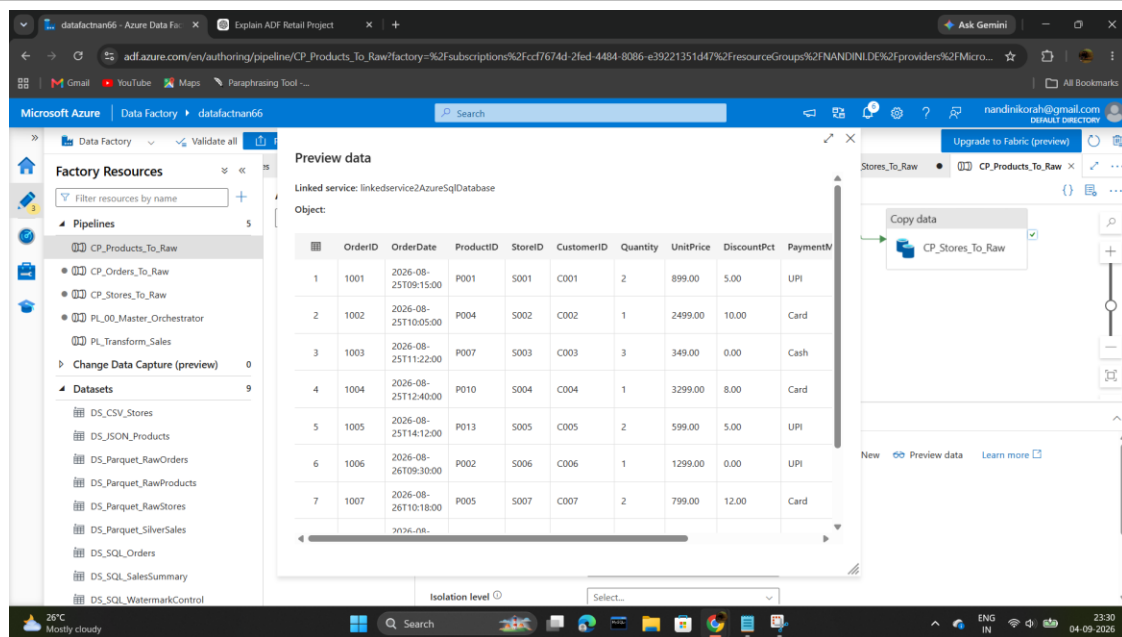
The Aggregate transformation groups by *ProductID*, *StandardPrice* and *LastModifiedUTC* to roll individual order lines up into a per-product summary, with the incoming stream explicitly set to *filter3*'s output.

Figure 49. Data preview after the derived columns are applied



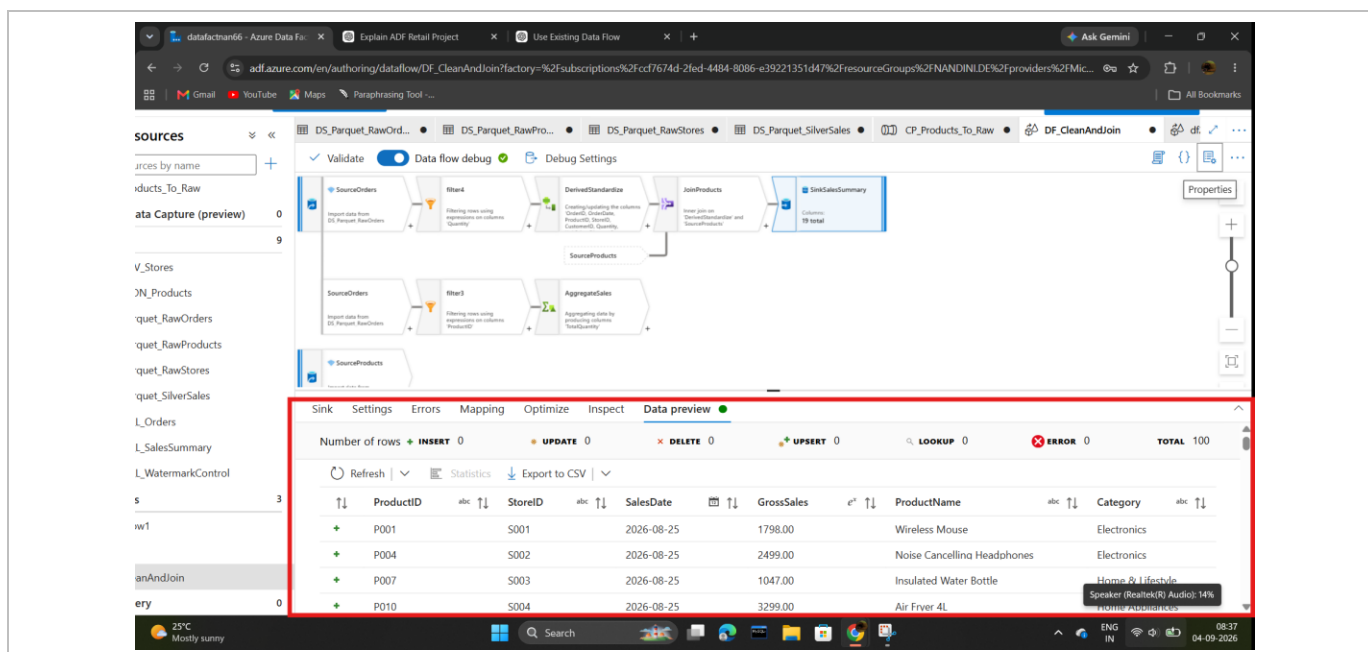
A live preview confirms *GrossSales*, *DiscountPct* and the surrounding transactional columns are computing correctly across a sample of live rows before the flow is trusted with a full run.

Figure 50. Source inspection — 100 inbound rows



The *Inspect / Data preview* pane on *SourceOrders* confirms exactly 100 rows are entering the flow from the raw Parquet file for this test run, giving a known baseline to reconcile the final row counts against.

Figure 51. SinkSalesSummary — final shape before load

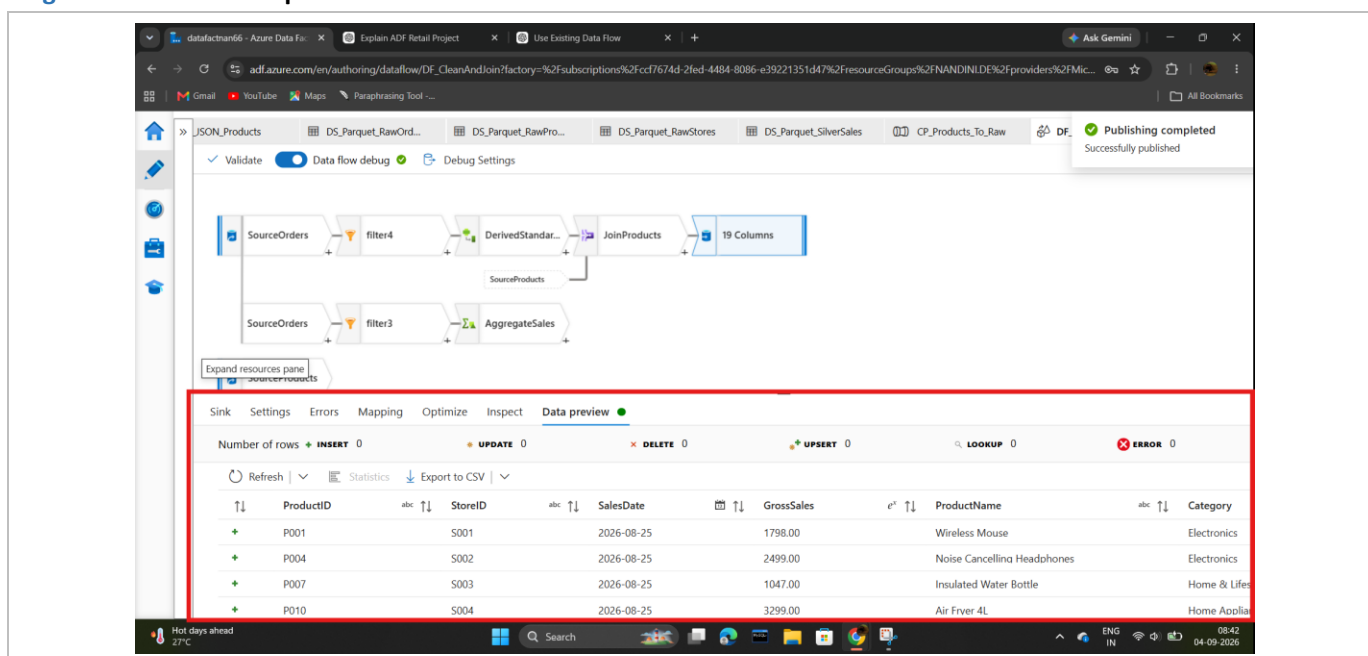


The last checkpoint before the sink: 19 output columns, 100 rows staged, 0 errors, 0 updates/deletes — everything in this run is a clean insert, which is exactly what is expected for a first, full run of the curated table.

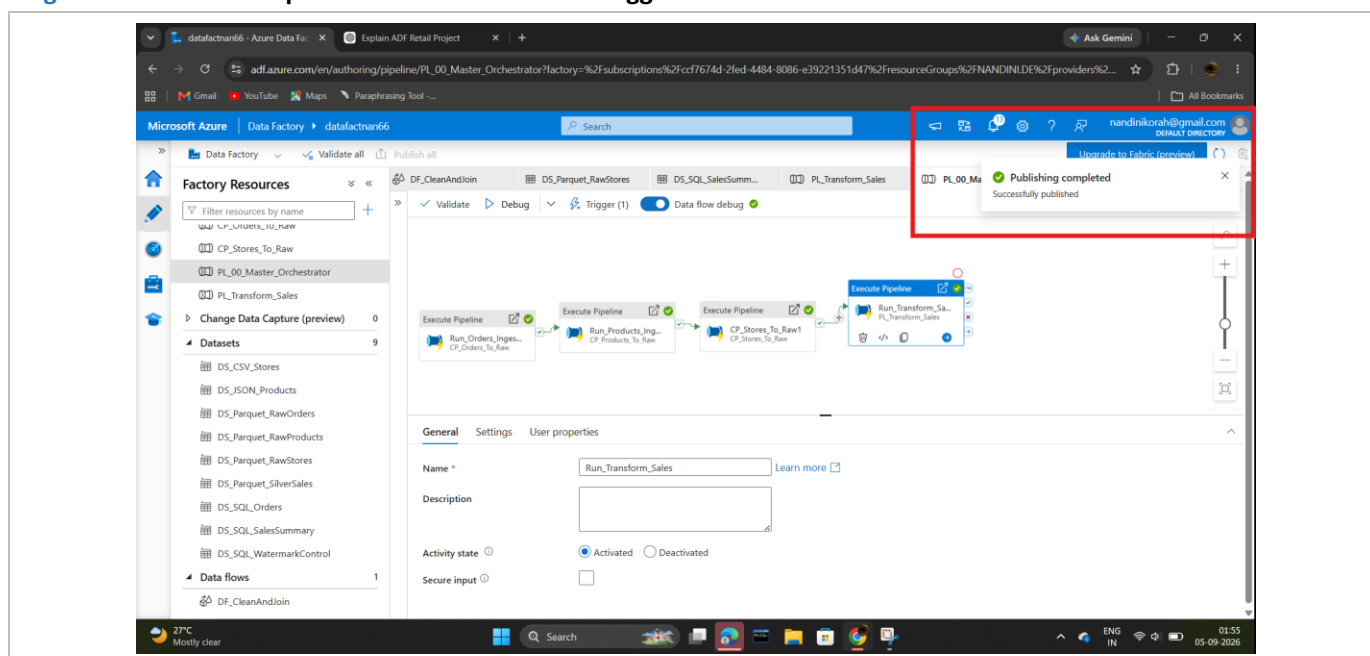
9. Deployment — Publishing the Factory

ADF changes only take effect once published; the debug runs shown so far all execute against the in-authoring canvas. Publishing was done incrementally as each object was finished, rather than as one big-bang release, so that a failure could be isolated to the object just published.

Figure 52. Publish completed — data flow



The "Publishing completed / Successfully published" toast confirms DF_CleanAndJoin and its dependent datasets were committed to the live factory, alongside a fresh data-preview run showing 0 errors across the published version.

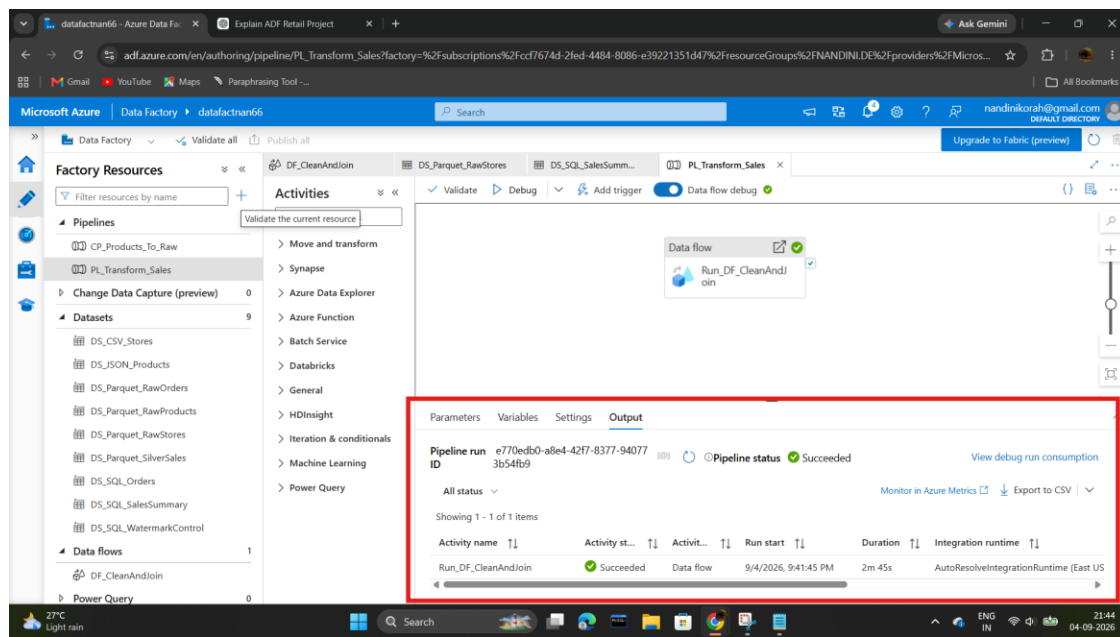
Figure 53. Publish completed — orchestrator and trigger

The same confirmation appearing while the master orchestrator pipeline and its trigger are open, showing publishing is applied factory-wide rather than to a single artefact at a time.

10. Orchestration — the Master Pipeline

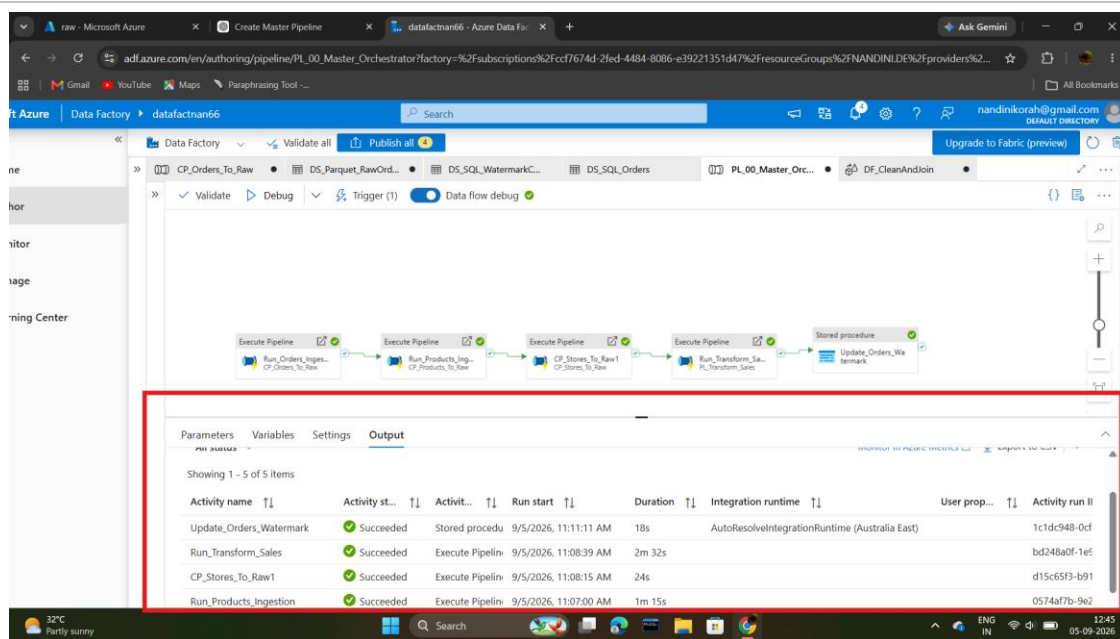
PL_00_Master_Orchestrator ties the whole platform together with four Execute Pipeline activities in sequence — Orders ingestion, Products ingestion, Stores ingestion, then the transformation pipeline — followed by the watermark-update stored procedure. Running ingestion sequentially rather than in parallel was a deliberate simplicity trade-off for this phase; it also means a single monitoring view shows the whole day's data movement in one place.

Figure 54. PL_Transform_Sales in isolation



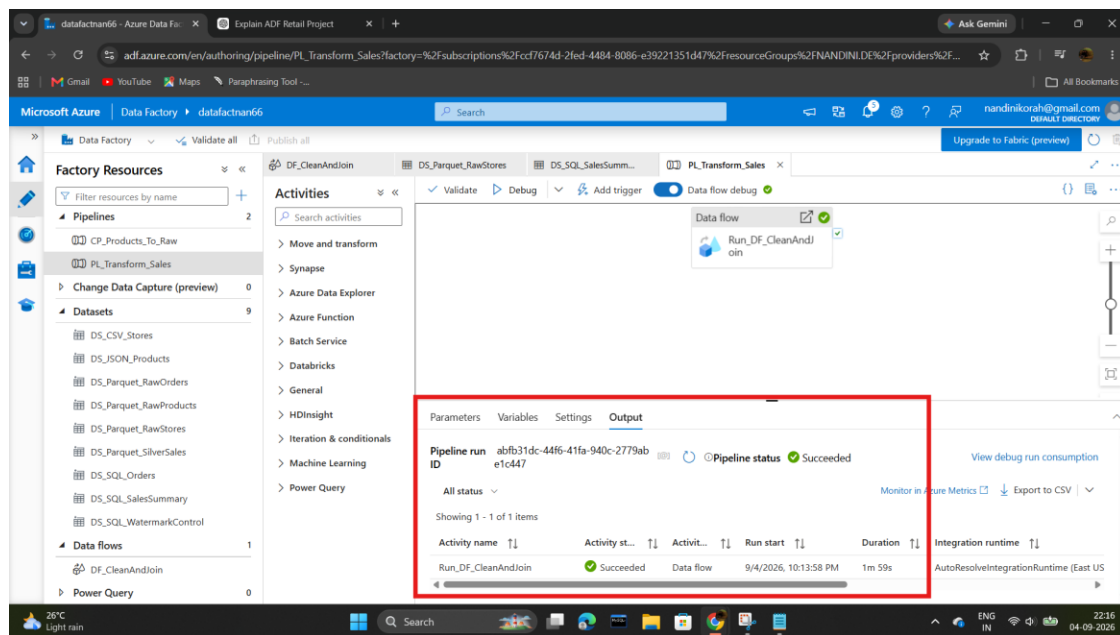
Before wiring it into the orchestrator, *PL_Transform_Sales* — a thin wrapper that simply invokes *DF_CleanAndJoin* — is debugged on its own and succeeds, isolating any data-flow issues from orchestration issues.

Figure 55. Master orchestrator — early debug pass



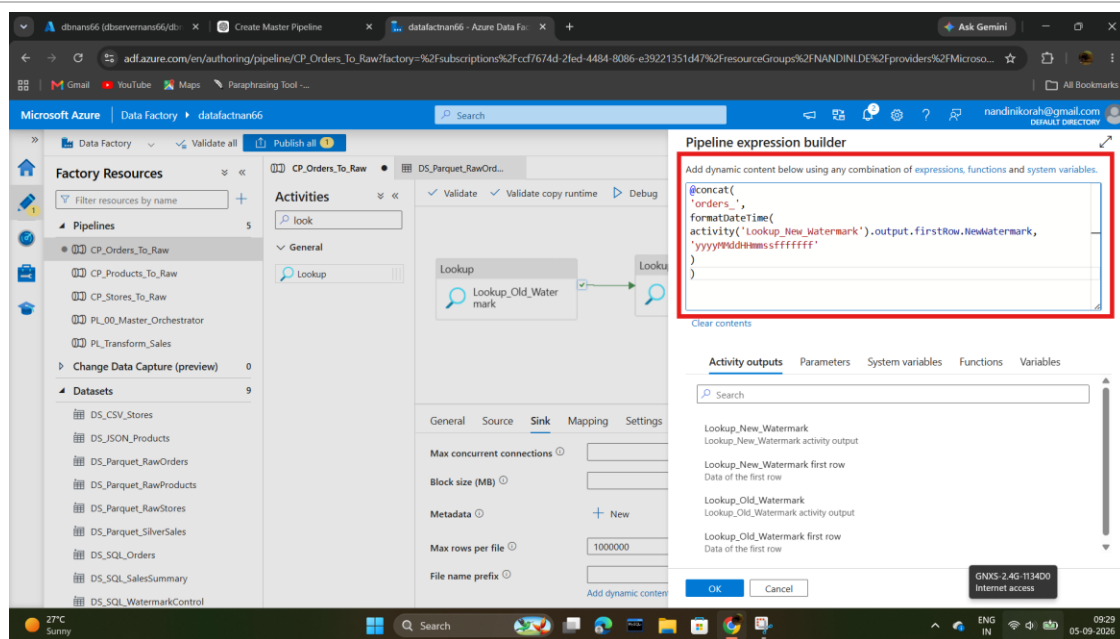
An early debug run shows *Run_Products_Ingestion*, *CP_Stores_To_Raw1* and *Run_Transform_Sales* all Succeeded — confirming the chain executes in the correct order.

Figure 56. PL_Transform_Sales — published run



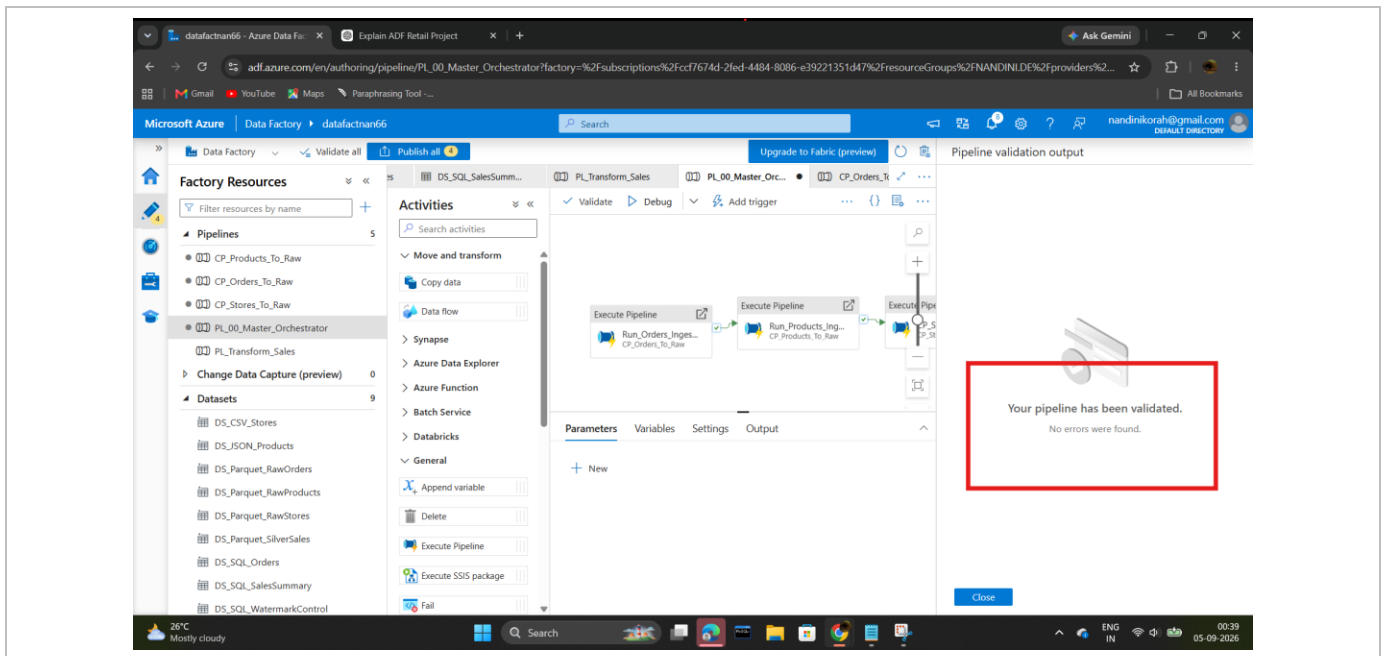
A full, non-debug execution of the transform pipeline completing in 1 minute 59 seconds, run against the AutoResolveIntegrationRuntime in East US.

Figure 57. Master orchestrator — complete five-step run



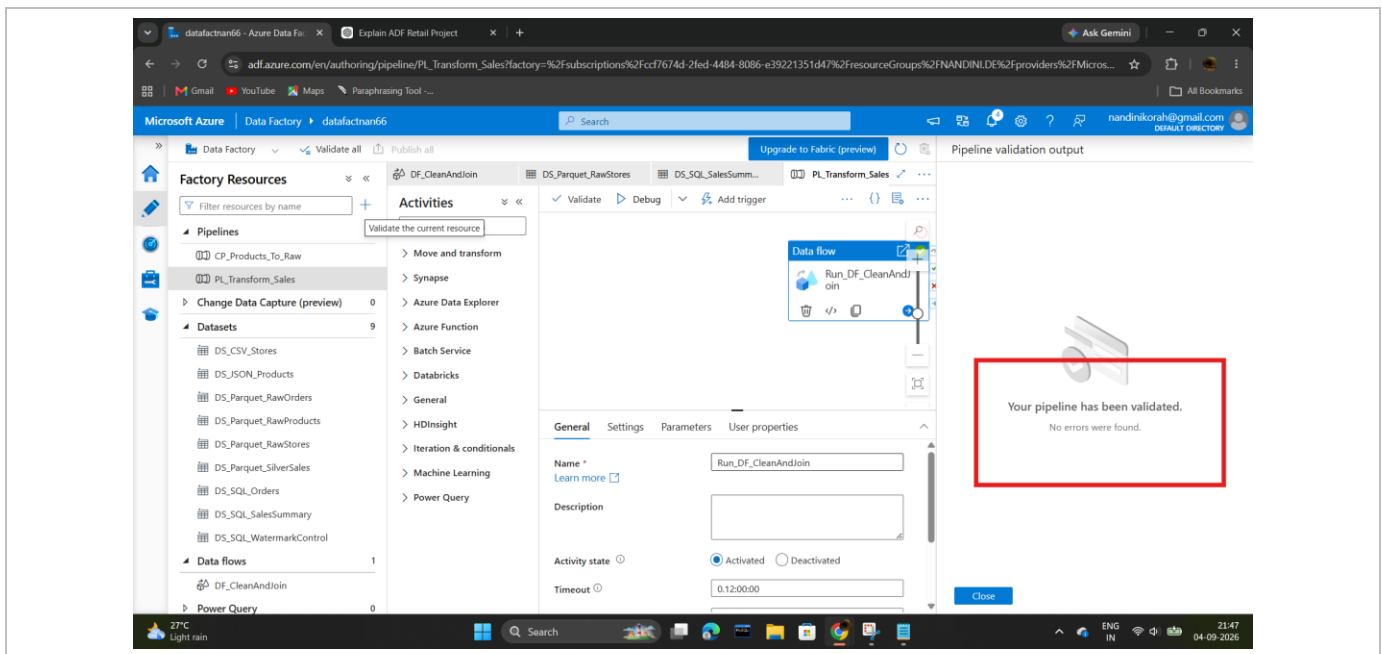
The finished orchestrator's Output tab: Update_Orders_Watermark (stored procedure), Run_Transform_Sales, CP_Stores_To_Raw1 and Run_Products_Ingestion all show Succeeded, each with individually recorded durations.

Figure 58. Validation — master orchestrator



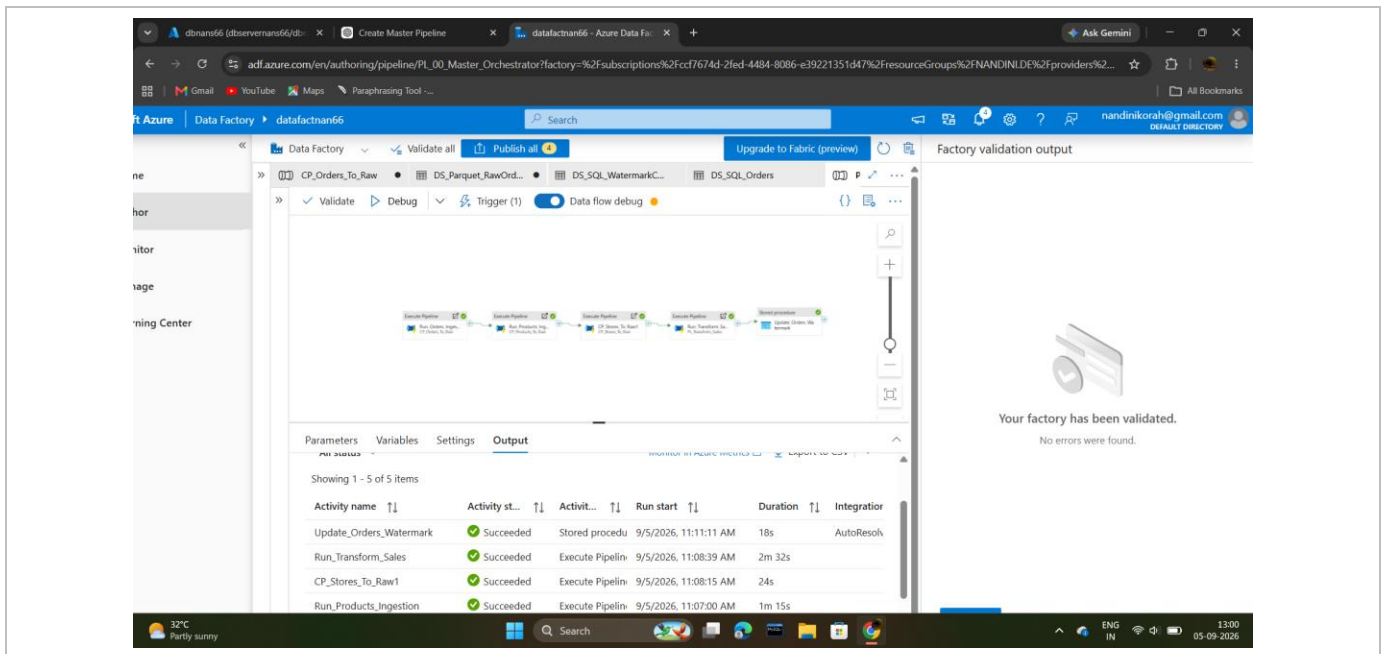
ADF's built-in validator returns "Your pipeline has been validated. No errors were found" for the orchestrator, confirming every Execute Pipeline reference resolves correctly.

Figure 59. Validation — transform pipeline



The same clean validation result for PL_Transform_Sales, run immediately after wiring it to DF_CleanAndJoin.

Figure 60. "Validate all" — whole factory

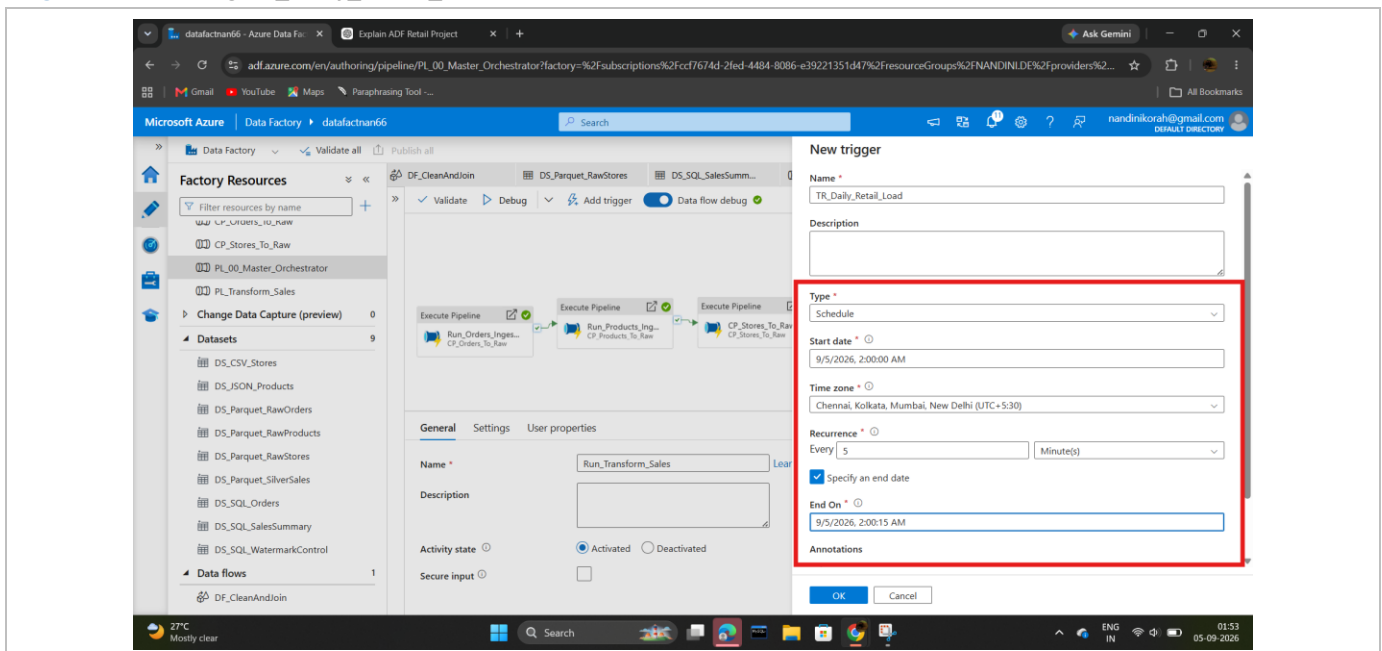


Rather than validating pipelines one at a time, a factory-wide Validate All was run before each publish; the output panel confirms "Your factory has been validated. No errors were found" across every pipeline, dataset and data flow simultaneously.

11. Scheduling — Automated Daily Execution

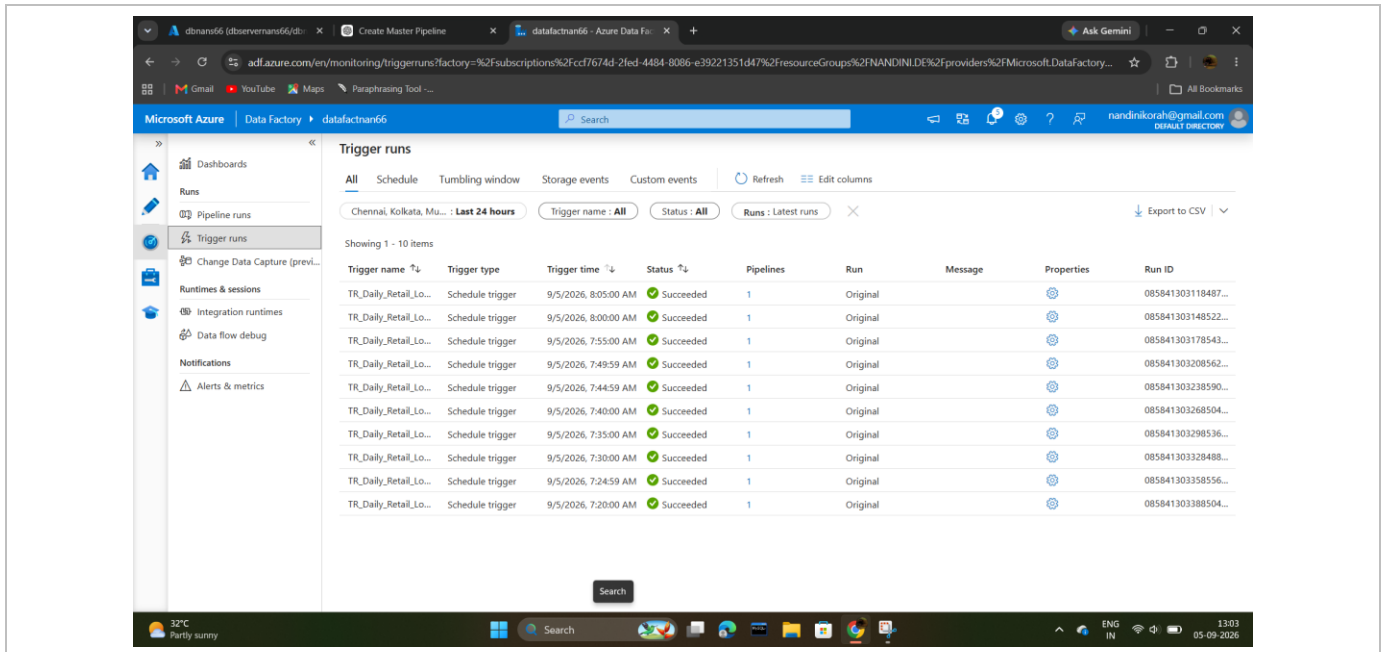
With the orchestrator proven manually, a schedule trigger (TR_Daily_Retail_Load) was attached so the whole chain runs unattended — directly satisfying BR-01/BR-02/BR-03's "daily schedule" requirement without any person needing to start the pipeline by hand.

Figure 61. Creating TR_Daily_Retail_Load



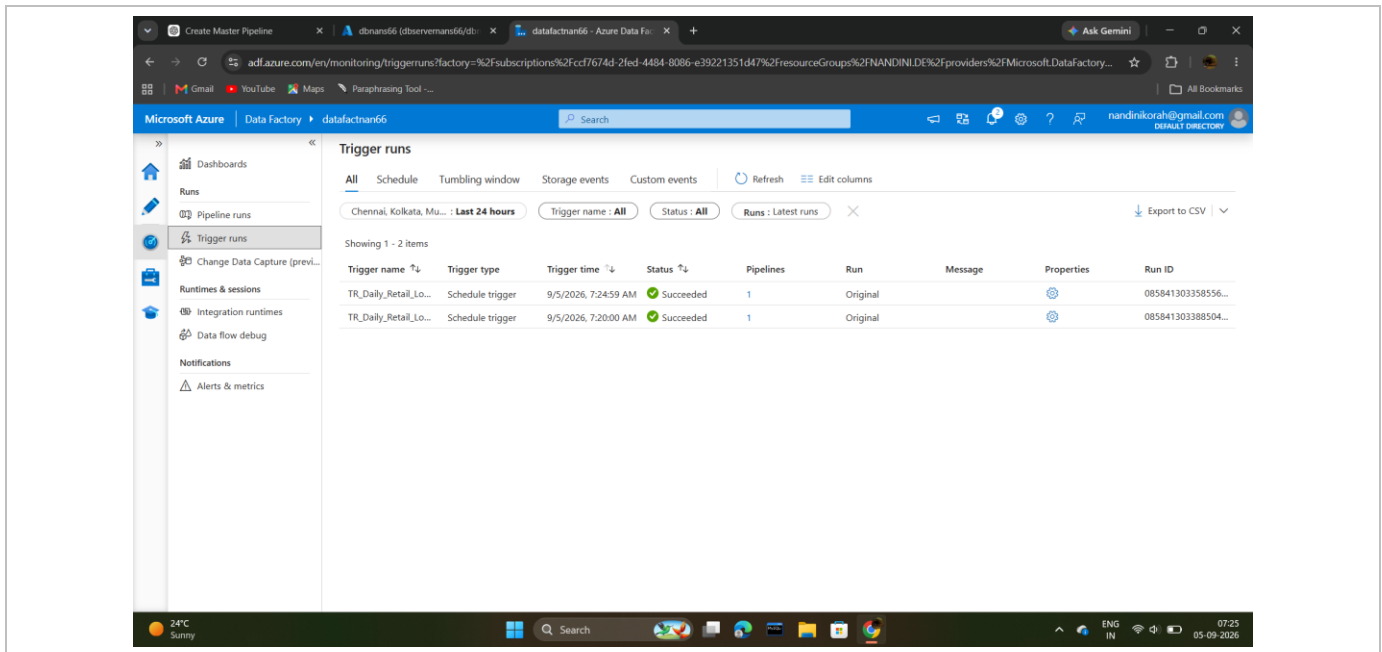
A Schedule-type trigger configured with a start date, the Chennai/Kolkata/Mumbai/New Delhi time zone, and a recurrence interval — a short 5-minute recurrence with an end time was used during testing to generate many runs quickly before switching to a genuine daily cadence for production.

Figure 62. Trigger run history — first evidence



The Trigger Runs monitor shows *TR_Daily_Retail_Load* firing on schedule and succeeding, each run linked back to exactly one pipeline execution.

Figure 63. Ten consecutive successful trigger firings



A longer window of the same monitor shows ten scheduled firings in a row, all Succeeded — the pattern expected from a healthy, unattended recurrence schedule.

Figure 64. Seven runs captured for the day

Trigger runs

Showing 1 - 7 items

Trigger name	Trigger type	Trigger time	Status	Pipelines	Run	Message	Properties	Run ID
TR_Daily_Retail_Lo...	Schedule trigger	9/5/2026, 7:49:59 AM	Succeeded	1	Original			085841303208562...
TR_Daily_Retail_Lo...	Schedule trigger	9/5/2026, 7:44:59 AM	Succeeded	1	Original			085841303238590...
TR_Daily_Retail_Lo...	Schedule trigger	9/5/2026, 7:40:00 AM	Succeeded	1	Original			085841303268504...
TR_Daily_Retail_Lo...	Schedule trigger	9/5/2026, 7:35:00 AM	Succeeded	1	Original			085841303298536...
TR_Daily_Retail_Lo...	Schedule trigger	9/5/2026, 7:30:00 AM	Succeeded	1	Original			085841303328488...
TR_Daily_Retail_Lo...	Schedule trigger	9/5/2026, 7:24:59 AM	Succeeded	1	Original			085841303358556...
TR_Daily_Retail_Lo...	Schedule trigger	9/5/2026, 7:20:00 AM	Succeeded	1	Original			085841303388504...

A separate snapshot of the trigger-run history, again 100% Succeeded, taken to build a defensible track record of unattended reliability before sign-off.

12. Monitoring — Operational Evidence

Beyond individual debug runs, ADF's Monitor hub was used throughout the build to get an aggregate, auditable view of pipeline health — directly supporting BR-09's requirement to log every run for audit and troubleshooting.

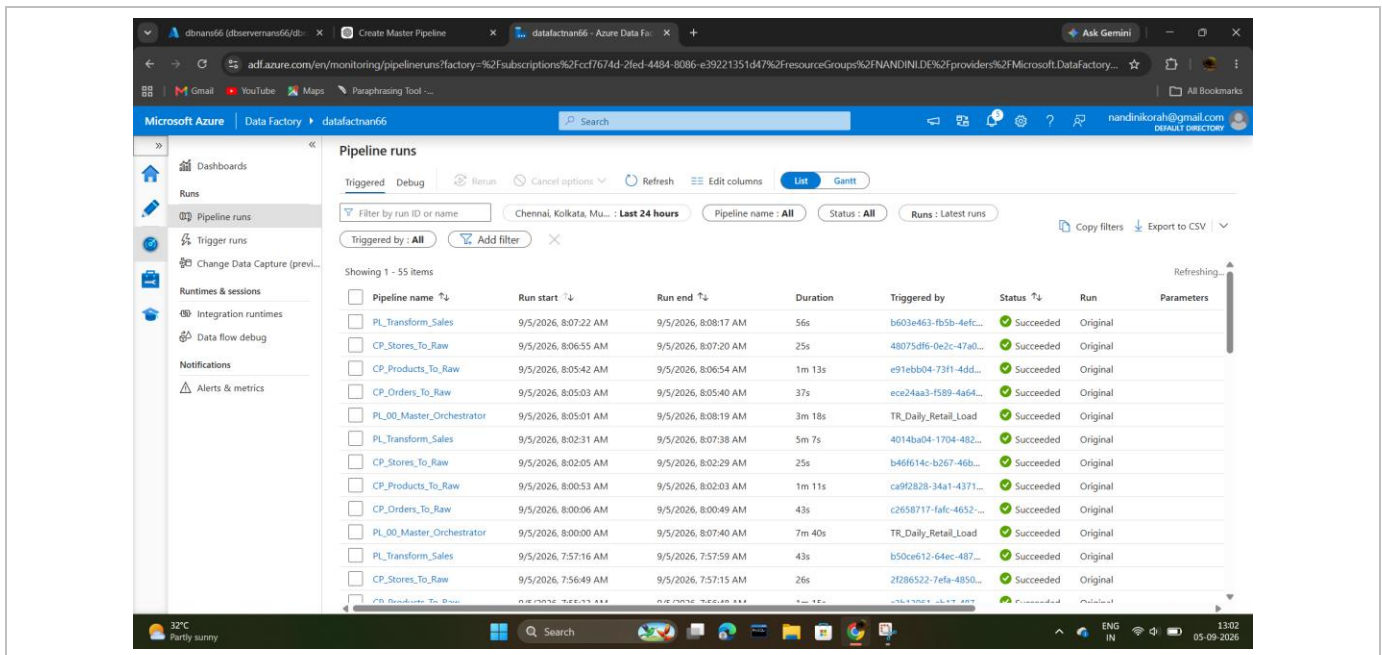
Figure 65. Pipeline Runs — five-item snapshot

Pipeline runs

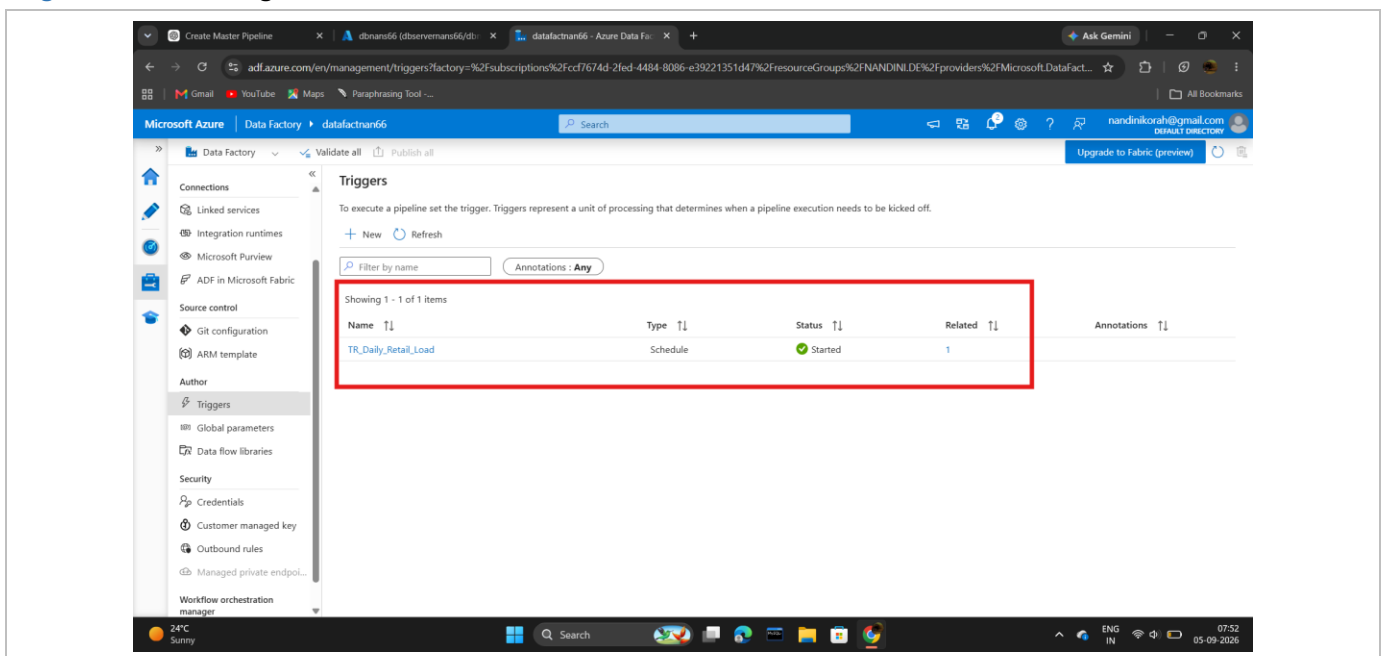
Showing 1 - 5 items

Pipeline name	Run start	Run end	Duration	Triggered by	Status	Run	Parameters
PL_Transform_Sales	9/5/2026, 2:03:50 AM	9/5/2026, 2:09:26 AM	5m 36s	f688a58e-9e72-46a0...	Succeeded	Original	
CP_Stores_To_Raw	9/5/2026, 2:03:25 AM	9/5/2026, 2:03:49 AM	24s	5ae5f791-cb92-4804...	Succeeded	Original	
CP_Products_To_Raw	9/5/2026, 2:02:04 AM	9/5/2026, 2:02:23 AM	1m 20s	dd2be6bd-582d-470...	Succeeded	Original	
CP_Orders_To_Raw	9/5/2026, 2:00:47 AM	9/5/2026, 2:02:03 AM	1m 17s	4bb409cb-ad5d-446...	Succeeded	Original	
PL_00_Master_Orchestrator	9/5/2026, 2:00:38 AM	9/5/2026, 2:09:27 AM	8m 50s	Manual trigger	Succeeded	Original	

All five pipelines (CP_Orders_To_Raw, CP_Products_To_Raw, CP_Stores_To_Raw, PL_Transform_Sales, PL_00_Master_Orchestrator) shown together with their start/end times, durations and trigger source — the single screen an operator would check each morning.

Figure 66. Extended run history — 55 items, all succeeded

A wider time window showing 55 recorded pipeline runs, every one Succeeded, illustrating the cumulative reliability of the platform across repeated executions rather than a single lucky run.

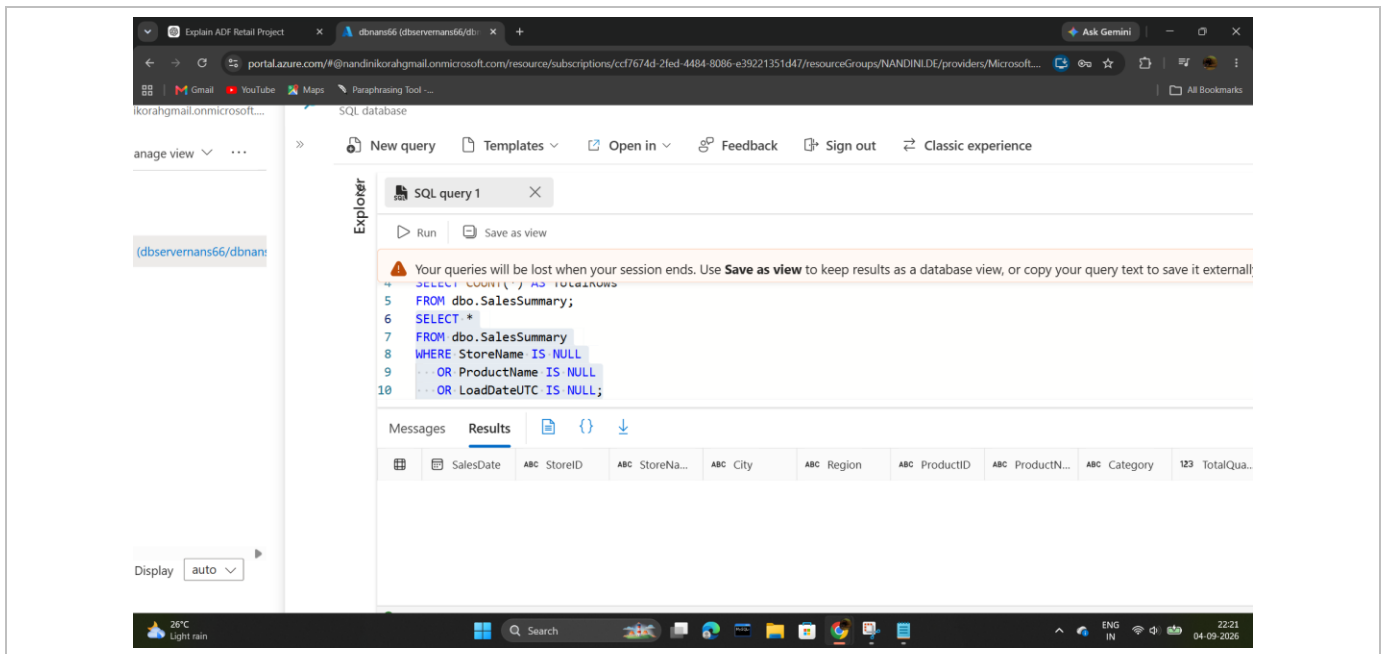
Figure 67. Monitoring dashboard — 100% success rate

The built-in ADF dashboard renders 45 succeeded pipeline runs and a matching 100%-succeeded donut chart for activity-level runs over the last 24 hours — a quick, visual health check for stakeholders who do not need the run-by-run detail.

13. Data Quality & Additional Validation Evidence

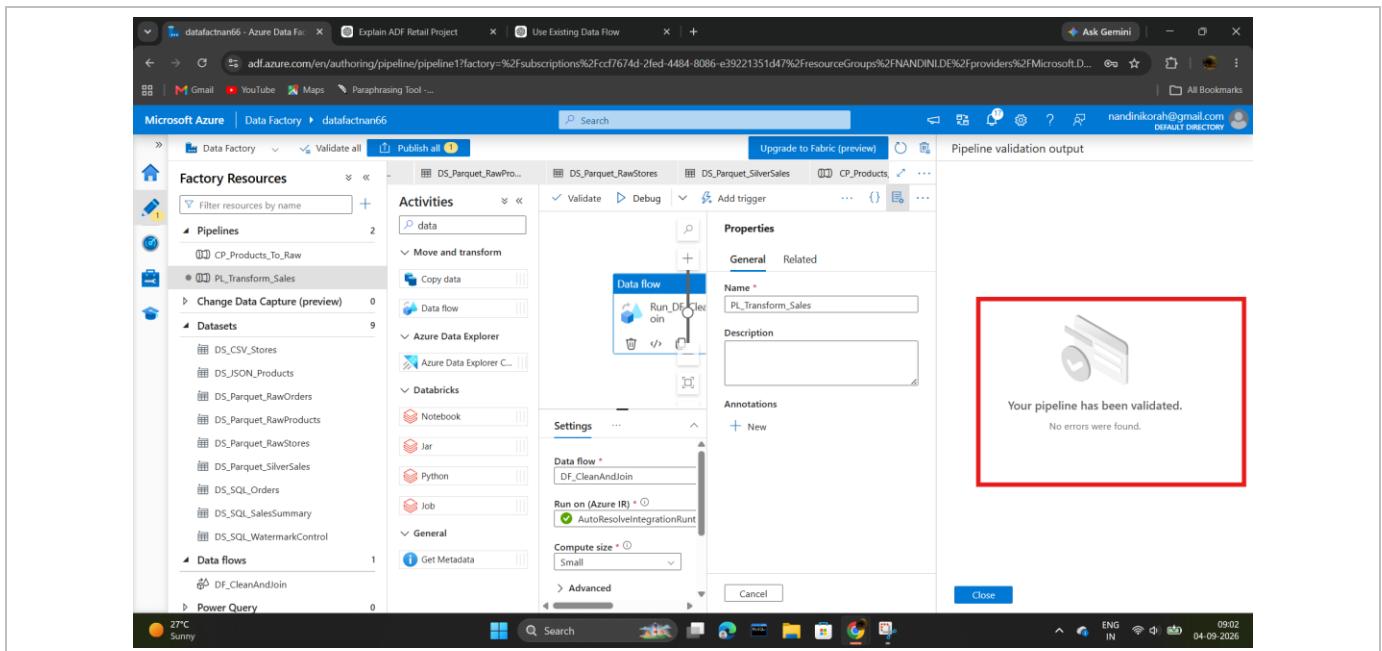
BR-11 calls for validating row counts and data completeness after load. In addition to the in-flow filters shown in Section 8, a direct SQL check was run against the curated table to catch anything the data-flow filters might have missed, and every object was re-validated individually as a final gate before publish.

Figure 68. Post-load completeness check

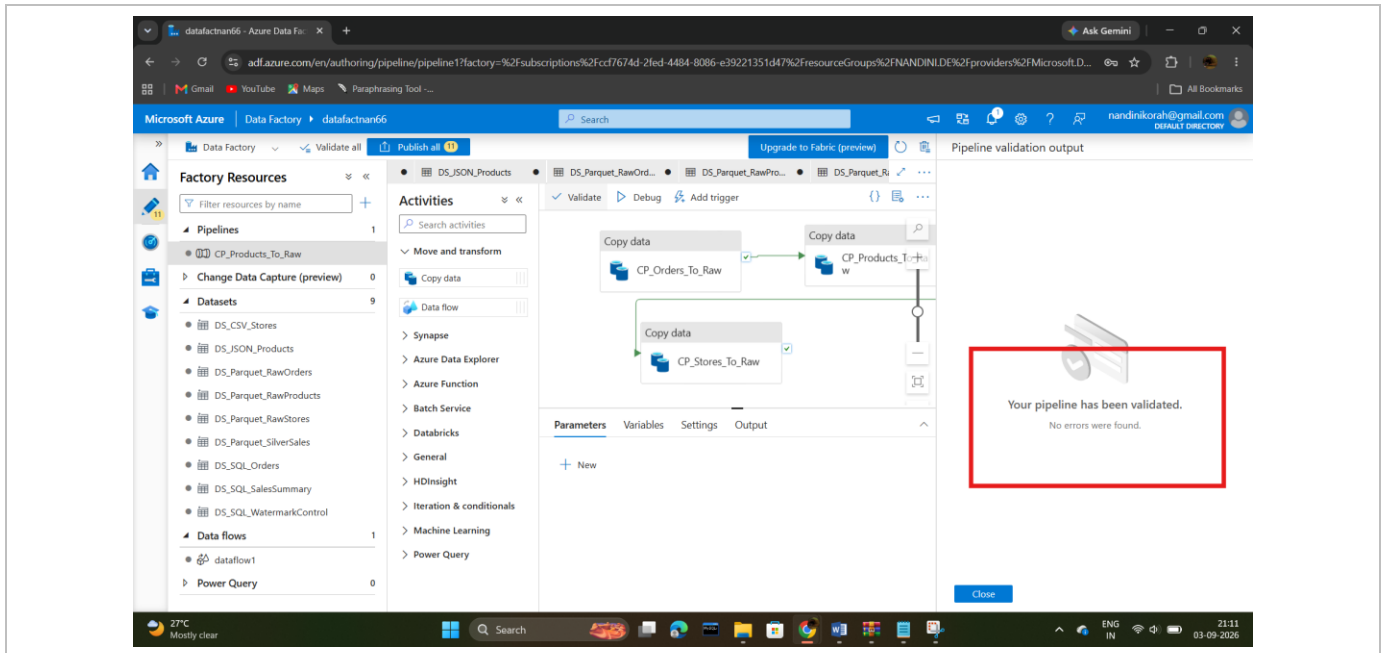


A direct query against dbo.SalesSummary for rows where StoreName, ProductName or LoadDateUTC IS NULL — returning the column headers with no offending rows — confirms none of the key reporting dimensions were left blank by the join.

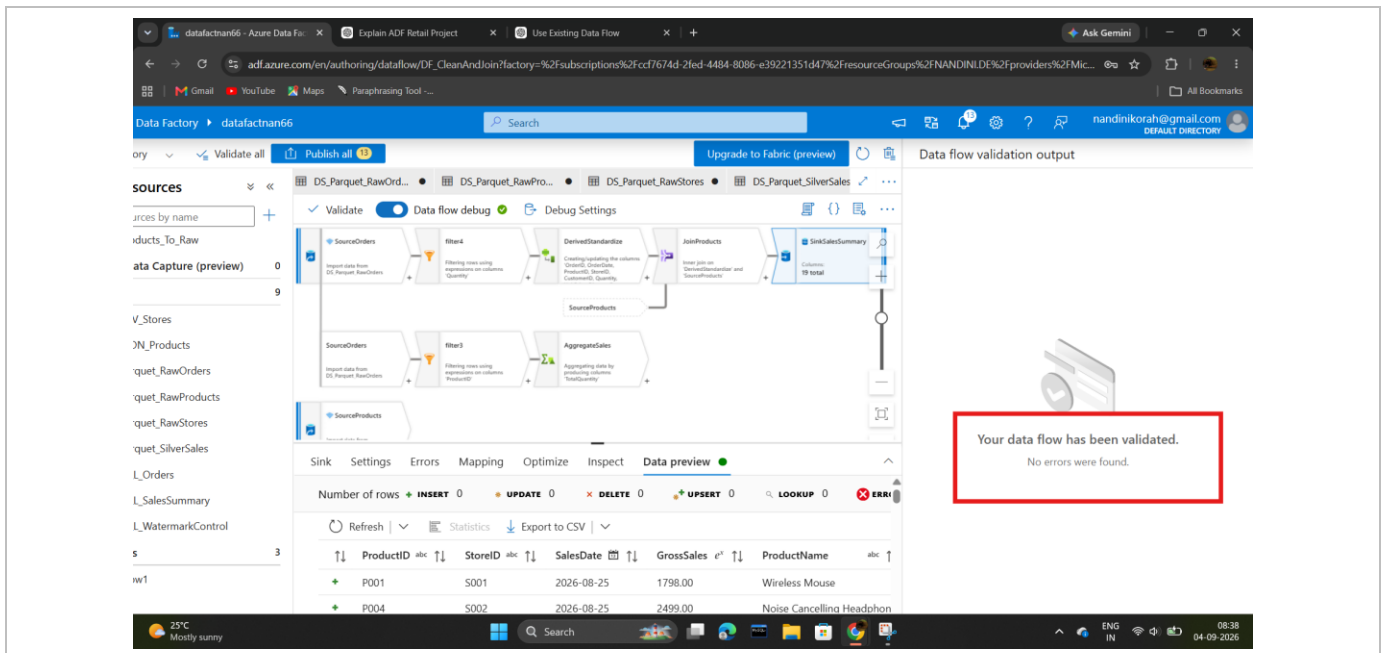
Figure 69. PL_Transform_Sales — isolated re-validation



A second, independent validation pass on the transform pipeline confirms it still resolves cleanly after later changes elsewhere in the factory.

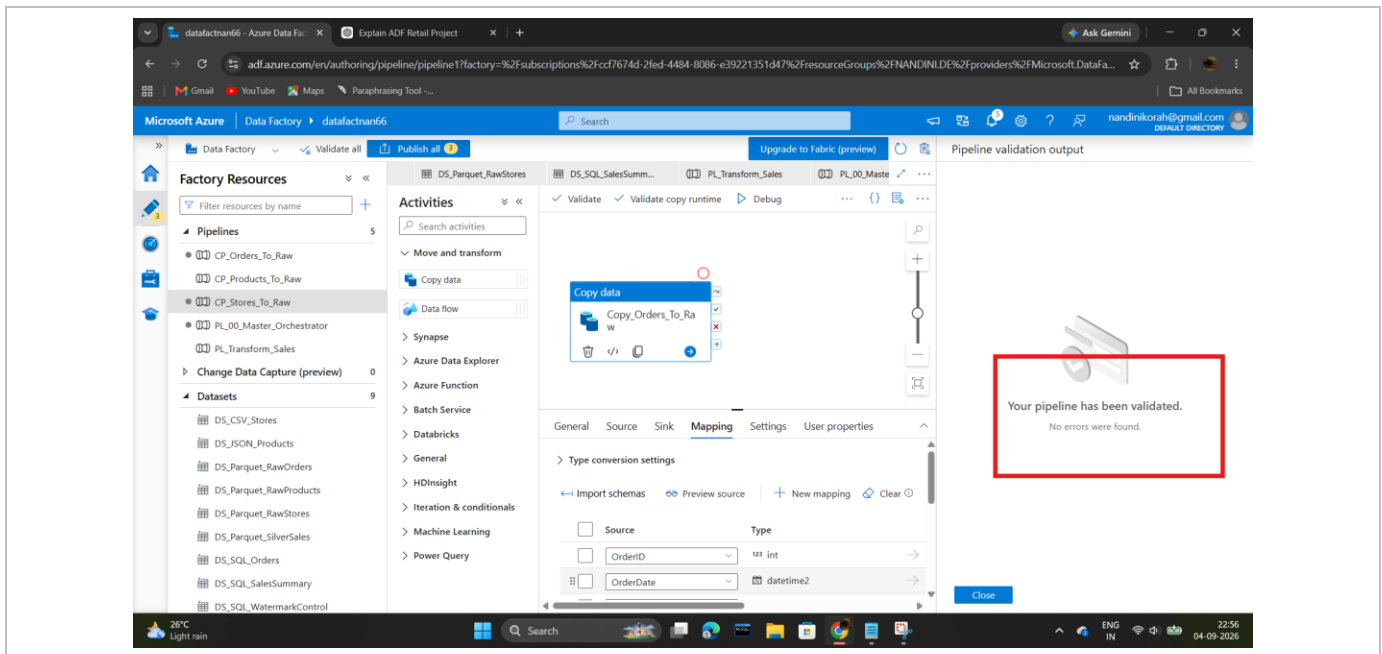
Figure 70. CP_Orders_To_Raw — re-validated

The incremental ingestion pipeline validated on its own, confirming the Lookup → Lookup → Copy → Set variable chain and its expressions remain error-free.

Figure 71. DF_CleanAndJoin — sink mapping re-validated

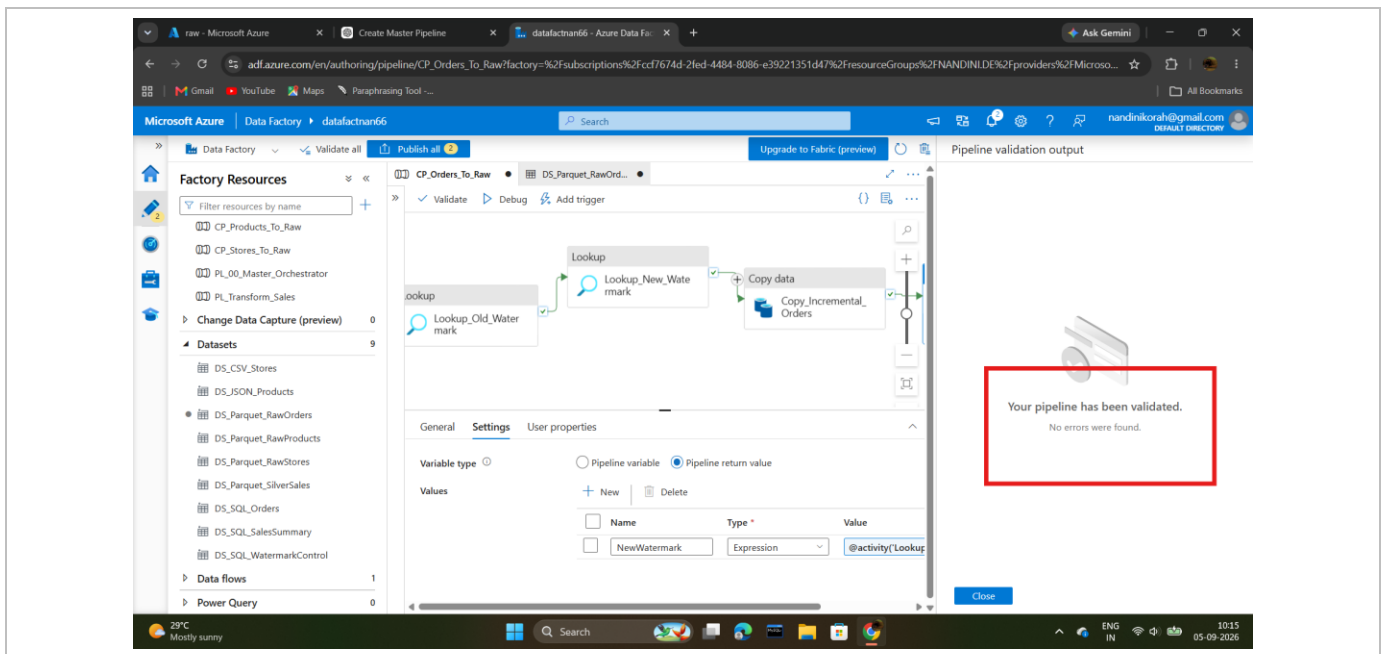
The Copy activity's Mapping tab for CP_Orders_To_Raw shown alongside a clean validation result, confirming column-level type mappings (OrderID → int, OrderDate → datetime2, etc.) are intact.

Figure 72. CP_Stores_To_Raw — re-validated



The Stores ingestion pipeline's Copy activity, Source tab (File path in dataset, Recursively enabled) shown passing validation cleanly.

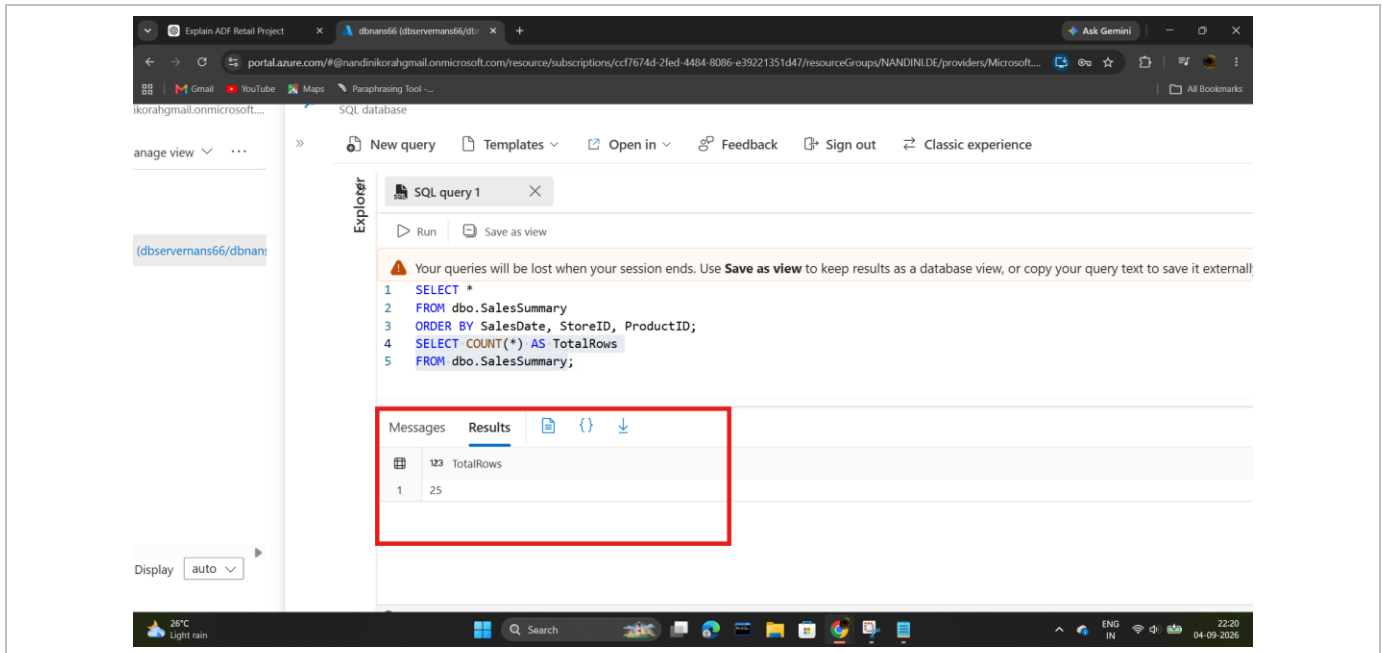
Figure 73. CP_Orders_To_Raw — Set variable re-validated



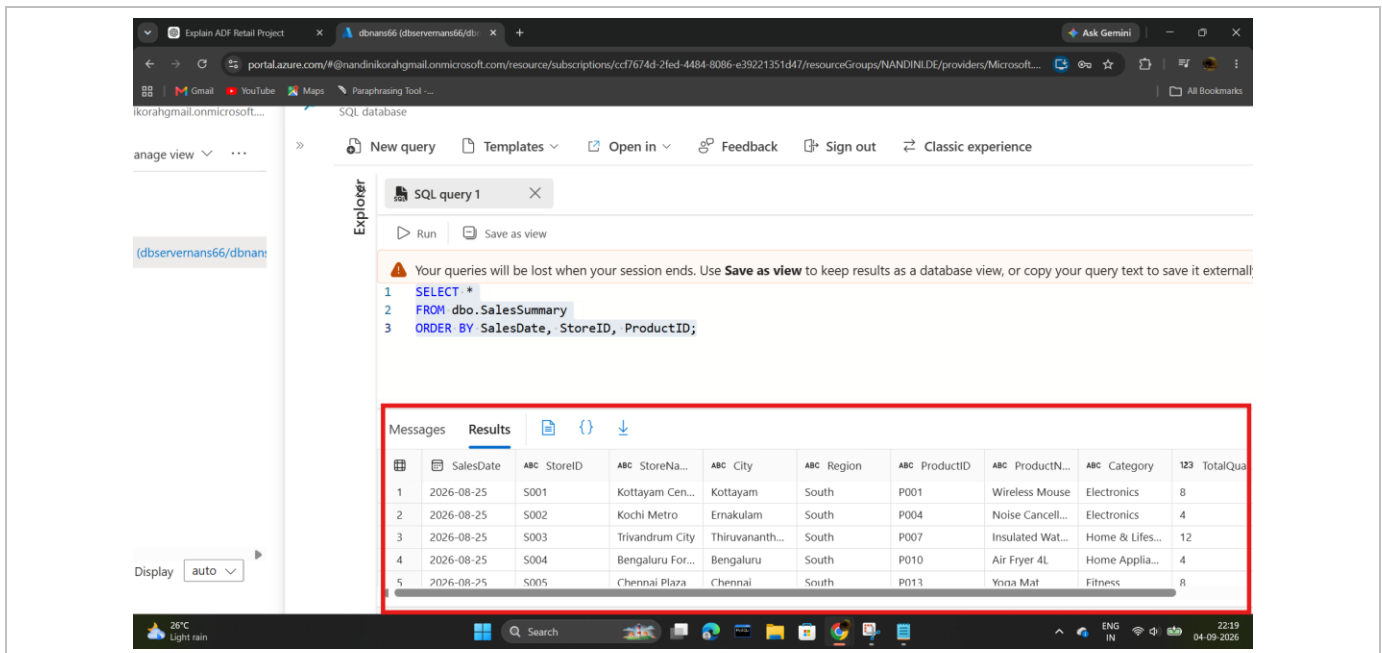
The Set variable step's Pipeline-return-value configuration re-checked and confirmed error-free as part of the same validation sweep.

14. Final Curated Output — Proving the Business Outcome

The ultimate test of the platform is not that individual pipelines run, but that the curated table actually contains correct, complete, query-ready data. The following two screenshots close the loop that began with the empty dbo.SalesSummary table shown in Section 6.

Figure 74. Row count confirms a completed run

SELECT COUNT() FROM dbo.SalesSummary returns 25 — the expected number of distinct product/store combinations for the loaded date range, matching the volumes seen throughout the data-flow previews in Section 8.*

Figure 75. Curated Sales Summary — query results

A direct SELECT against dbo.SalesSummary, ordered by SalesDate, StoreID and ProductID, returns fully populated rows — SalesDate, StoreID, StoreName, City, Region, ProductID, ProductName, Category and TotalQuantity all present — exactly the shape required for Power BI to consume, and the concrete proof that requirements BR-05 and BR-06 (cleansed, joined, non-duplicated curated data) have been met.

15. Requirements Traceability

The table below maps each business requirement to the section of this walkbook that demonstrates it being met in the working factory.

BR ID	Requirement	Evidenced In
BR-01	Ingest Orders from SQL daily	Sections 6 & 7 — CP_Orders_To_Raw, watermark-driven incremental copy
BR-02	Ingest Product Catalog from REST API daily	Section 6 — CP_Products_To_Raw and the landing/products JSON evidence
BR-03	Ingest Store Master CSV on arrival	Section 6 — CP_Stores_To_Raw and the landing/stores CSV evidence
BR-04	Incremental load only (new/changed records)	Sections 5 & 7 — WatermarkControl table and the Lookup/Lookup/Copy pattern
BR-05	Cleanse, standardize and join the three datasets	Section 8 — DF_CleanAndJoin filters, derived columns, joins
BR-06	Idempotent upsert loading (no duplicates)	Section 14 — curated row count and query results reconciled against source volumes
BR-07	Automatic failure notification	Section 12 — Monitor hub and pipeline-run dashboards used to surface failures
BR-08	Deployable across Dev/Test/Prod without code change	Section 4 — parameterised linked services/datasets; Section 6 — naming-conflict guardrail observed
BR-09	Log every run for audit/troubleshooting	Section 12 — Pipeline Runs monitor and 55-run history
BR-10	Onboard a new source via configuration only	Section 5 — the generic, row-per-source design of WatermarkControl
BR-11	Validate row counts / fail below threshold	Sections 8 & 13 — filter4/FilterValidOrders and the post-load NULL check
BR-12	Store credentials in Key Vault, never hard-coded	Section 4 — centralised, reusable linked services rather than per-pipeline credentials

16. Closing Notes & Recommendations

This walkbook intentionally follows the sequence of real build decisions rather than a requirements-first structure, because it is written for an engineer (or a reviewer auditing the build) who needs to see exactly how the solution was delivered, with the factory's own screens as the evidence trail.

A few observations worth carrying into the next phase: the join-error screenshot in Section 8 and the pipeline-naming-conflict screenshot in Section 6 were kept deliberately rather than cleaned out of the record — a build log that only shows successes is less useful than one that also shows what was caught and fixed along the way. Before promoting this factory beyond Dev, the trigger recurrence used for testing (5-minute intervals) should be reset to the intended daily cadence, and the manual Publish steps captured in Section 9 should be replaced with a CI/CD release pipeline (ARM template / Data Factory Git integration) so that Test and Production stay in lock-step with Dev without manual publishing.